

“Unlike all the other books that start at the beginning, this one will get you to the good stuff, fast. Jennifer will explain every step you need, including some very advanced concepts.”

—JEN SIMMONS, MOZILLA AND W3C CSS WORKING GROUP

**Early
Release**

**RAW &
UNEDITED**

Learning Web Design

A BEGINNER'S GUIDE TO HTML, CSS,
JAVASCRIPT, AND WEB GRAPHICS

Jennifer Niederst Robbins

Whether you're a beginner or bringing your skills up to date, this book gives you a solid footing in modern web production. I teach each topic visually at a pleasant pace, with frequent exercises to let you try out new skills. Reading it feels like sitting in my classroom! —Jennifer Robbins

Learning Web Design

SIXTH EDITION

A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

Jennifer Robbins

O'REILLY®

Learning Web Design

by Jennifer Robbins

Copyright © 2025 Littlechair, Inc.. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North,
Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

- Acquisitions Editor: David Michelson
- Development Editor: Jeff Bleiel
- Production Editor: Jonathon Owen
- Interior Designer: David Futato
- Cover Designer: Karen Montgomery
- Illustrator: Kate Dullea

- March 2001: First edition
- June 2003: Second edition
- June 2007: Third edition
- August 2012: Fourth edition
- May 2018: Fifth edition
- June 2025: Sixth Edition

Revision History for the Early Release

- 2025-05-01: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781098137687> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. Learning Web Design, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author(s) and do not represent the publisher's views. While the publisher and the author(s) have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author(s) disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at

your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-13762-5

[LSI]

Brief Table of Contents (*Not Yet Final*)

Part 1: Getting Started

Chapter 1: Teams and Tools (available)

Chapter 2: How the Web Works (available)

Chapter 3: Some Big Concepts You Need to Know (available)

Part 2: HTML for Structure

Chapter 4: Creating a Simple Page (available)

Chapter 5: Marking Up Text (available)

Chapter 6: Adding Links (unavailable)

Chapter 7: Adding Images (unavailable)

Chapter 8: Table Markup (unavailable)

Chapter 9: Forms (unavailable)

Chapter 10: Embedded Media (unavailable)

Part 3: CSS for Presentation

Chapter 11: Introducing Cascading Style Sheets (unavailable)

Chapter 12: Formatting Text (unavailable)

Chapter 13: Colors and Backgrounds (unavailable)

Chapter 14: Selectors (unavailable)

Chapter 15: Thinking Inside the Box (unavailable)

Chapter 16: Floating and Positioning (unavailable)

Chapter 17: Getting in Line with Flexbox (unavailable)

Chapter 18: Grid Layout (unavailable)

Chapter 19: Responsive Web Design (unavailable)

Chapter 20: Transitions, Transforms, and Animation (unavailable)

Chapter 21: More CSS Techniques (unavailable)

Part 4: JavaScript for Behavior

Chapter 22: Getting Started with JavaScript (unavailable)

Chapter 23: Functions, Conditionals, and Loops (unavailable)

Chapter 24: Working with the DOM and Events (unavailable)

Chapter 25: Next-Level JavaScript (unavailable)

Part 5: Web Images

Chapter 26: Web Image Basics (unavailable)

Chapter 27: Image Production Techniques (unavailable)

Chapter 28: SVG (unavailable)

Chapter 1. Teams and Tools: Working in Web Design

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 1st chapter of the final book.

If you’d like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at jbleiel@oreilly.com.

In this chapter:

- Design specialties
- Frontend development
- Backend development
- Content-related disciplines
- Recommended equipment
- Web-related software

The web has been around for more than 30 years now, experiencing euphoric early expansion, an economic-driven bust, an innovation-driven rebirth, and constant evolution along the way. One thing is certain: the web as a communication and commercial medium is here to stay.

This book is a good place to start learning what makes the web tick. My goal is to give you a solid foundation in writing code and producing images according to best practices. I'll also provide important context on *why* we do things the way we do. We'll get to writing code very soon, but first, let's take a look at the web design profession.

Maybe you are reading this book as part of a full course on web design and development. Maybe you bought it to expand your current skill set, to open up job possibilities. Whatever your motivation, it will help to be familiar with the many roles in the web design and development field, so you can consider which role is right for you. You don't have to learn to do it *all* (unless you want to!).

SHARE WHAT YOU LEARN

The web design community has thrived on the sharing of information. For many, the author of this book included, sharing what we learn, as we learn it, has led to new roles and opportunities. Even as a beginner, there are people out there who will benefit from your perspective on what you've learned. Sharing is a contribution to the community.

In this chapter, we'll look at a variety of jobs that are essential to the making of websites:

- Designers, who are responsible for making the site easy to use and great-looking
- Developers, who write the code for the pages themselves as well as the programs that make things work
- Content managers and information architects, who focus on the content of the site and how it is generated, managed, and organized

I'll also introduce the equipment and software commonly used for web-related tasks—in other words, the tools of the trade.

Getting Started in Web Design

There are many ways to get involved in web design, from building a small site for yourself to making it a full-blown career. You may enjoy being a “full-stack” web developer or just specializing in one skill. There are a lot of ways you can go.

If you are interested in pursuing web design or production as a career, you'll need to bring your skills up to a professional level. Employers may not require a web design degree or certification, but they will expect to see a portfolio of working sample sites that demonstrate your skills and experience. These might be sites you've created for class assignments, on a

volunteer basis, as personal projects, or for a small business or other organization. What's important is that they look professional and have well-written, clean HTML, style sheets, and scripts that show what you can do.

If your involvement is at a smaller scale—say, you just have a site or two you'd like to publish—you may find using a template on an online website service gives you a great head start (see the sidebar “I Just Want My Own Site”). Most allow you to tweak the underlying code, so what you learn in this book will help you customize the template to your liking.

I JUST WANT MY OWN SITE

You don't necessarily need to become a web designer or developer to start publishing on the web. There are many website hosting services that provide templates and drag-and-drop interfaces that make it easy to build a site without knowing how to code. They can be used for anything from full-service ecommerce solutions to small, personal sites (although some services are better suited to one or the other).

Here are a few of the most popular site-building services as of this writing:

- WordPress (wordpress.com)
- Squarespace (squarespace.com)
- Wix (wix.com)

There are many similar services available, some using AI to generate content and designs, so it's worth searching the web to find the one that's right for you.

Roles and Responsibilities

When I look at a site, I see the multitude of decisions and areas of expertise that went into building it. Sites usually begin with a business plan or other defined mission, ideally backed up with careful research. All of the content must be created and organized. Designers plan and design the overall

structure of the site, as well as every page type and each component they contain, keeping in mind the user's experience moving through it and a consistent presentation of the organization's brand identity. And of course, the code must be written and coordinated with what's happening on the server to bring it all to fruition.

Big, well-known sites are created by teams of dozens, hundreds, or even thousands of contributors. There are also sites that are created and maintained by much smaller teams, with just a handful of members. In fact, it's absolutely possible to create a respectable site with a team of only yourself. That's the beauty of the web.

You may end up specializing in just one area of expertise, or working as part of a team of specialists. If you are designing sites on your own, you will need to wear many hats. Consider that the day-to-day upkeep of your household requires you to be part-time chef, housecleaner, accountant, diplomat, gardener, and construction worker—but to you, it's just the stuff you do around the house. As a solo designer, you'll handle many web-related disciplines, but it will just feel like the stuff you do to make a website. In this section, I'll introduce you to the various disciplines that contribute to the creation of a site, including roles related to design, code, and content.

All Manner of Design

Ah, *design*! It sounds fairly straightforward, but even this simple requirement has been divided into a number of specializations. We'll take a look at a few of the tasks related to designing a site here, but bear in mind that the disciplines often overlap and that the person calling themselves the “designer” often is responsible for more than one (or perhaps even all) of these areas.

DO DESIGNERS NEED TO LEARN TO CODE?

In short, yes. Familiarity with HTML and CSS, as well as a general understanding of how content behaves in browsers, is now a requirement of anybody joining a web design team. You may not be responsible for creating the final production code for the site, but as HTML and CSS are the native languages of your medium, you need to know your way around them. Some designers also learn JavaScript, but others draw the line there and let a developer handle the programming.

Code has become central to the visual designer's workflow. Whereas once Photoshop was all you needed to mock up web page designs to send to production, mockups fall short for communicating how designs will look and behave across a wide range of devices and screen sizes. For that reason, working prototypes are preferred and often fall under the designer's responsibility. Fortunately, UI design tools like Figma have been designed to communicate the flexible nature of web pages (they can even generate code for you). It's not uncommon for designers to work out their designs in code (HTML and CSS), or to start the sketching process in an image editor but move to code very quickly to refine the details.

User experience and interaction design

Commonly, when people think of design, they think about how something looks. On the web, the first matter of business is designing how the site

works. Before you pick colors and fonts, it's important to identify the site's goals, how it will be used, and how visitors will move through it. These tasks fall under the disciplines of *user experience (UX) design* and *interaction design (IxD)*. There's a lot of overlap between these responsibilities, and it's not uncommon for one person or team to handle all of them.

The user experience designer takes a holistic view of the design process, ensuring the entire experience with the site is favorable. UX design is based on a solid understanding of users and their needs, derived through observations and interviews, balanced with business goals. According to Donald Norman (who coined the term), UX design includes “all aspects of the user's interaction with the product: how it is perceived, learned, and used.” For a website or application, that includes the visual design, the user interface, the quality and message of the content, and even how quickly the pages load. The experience must be in line with the organization's brand and business goals in order to be successful.

The goal of the *interaction designer* is to make the site as easy, efficient, and delightful to use as possible. An interaction designer makes decisions about the arrangement of components (content areas, search fields, buttons, calls to action, etc.) on each page type. They also think about pathways through the site and the steps and feedback in processes such as making a purchase or signing up for a service. They make sure the site makes it clear what can be done and how to do it.

The following are deliverables that UX and interaction designers produce:

User research and testing reports

Understanding the needs, desires, and limitations of users is central to the success of the design of the site or web application. The approach of designing around the user's needs is referred to as *user-centered design* (UCD), and it is central to contemporary web design. Site designs often begin with user research, including interviews and observations, in order to gain a better understanding of how the site can solve problems or how it will be used. The site's design should also be *inclusive*; that is, it should not alienate any segment of the population who may visit it.

Designers may do a round of user testing at each phase of the design process to ensure the usability of their designs. If users are having a hard time figuring out where to find content or how to move to the next step in a process, then it's back to the drawing board.

Wireframe diagrams

A wireframe diagram shows the structure of a web page using only outlines for each content type and widget ([Figure 1-1](#)). The purpose of a wireframe diagram is to indicate how the screen real estate is divided and where functionality and content such as navigation, search boxes, form elements, and so on are placed. Colors, fonts, and other visual identity elements are deliberately omitted so as not to distract from the structure of the page. These diagrams are often annotated with

instructions for how things should work, so the development team knows what to build. Wireframes may be created in drawing programs or they might be working prototypes designed with code, which have the advantage of communicating basic interactivity.

LOGO

[contact](#) | [store locator](#) | [support](#) | [CART](#)

[Log in](#) or [Create Account](#)

Category ▼

SEARCH

All categories

Category1

Category2

Category3

Category4

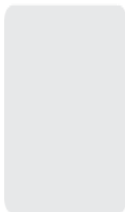
Category5

Category6

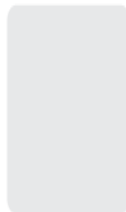
[PROMOTIONAL IMAGES ROTATE HERE]

1 2 3 4

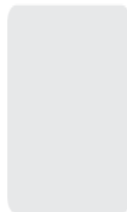
Today's Specials



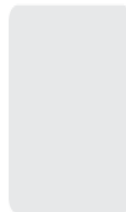
[Product 1](#)



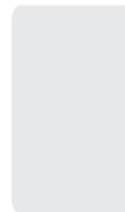
[Product 2](#)



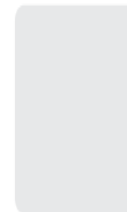
[Product 3](#)



[Product 4](#)



[Product 5](#)



[Product 6](#)

ABOUT US

[Company](#)
[News](#)
[Jobs](#)
[Policies](#)
[Contact](#)

SOCIAL

[Facebook](#)
[Twitter](#)
[Try our app](#)

SERVICE

[FAQ](#)
[Live support](#)
[Site map](#)

copyright statement

Figure 1-1. A wireframe diagram

Site diagram

A site diagram indicates the structure of the site as a whole and how individual pages relate to one another. [Figure 1-2](#) shows a very simple site diagram. Some site diagrams fill the walls of entire hallways!

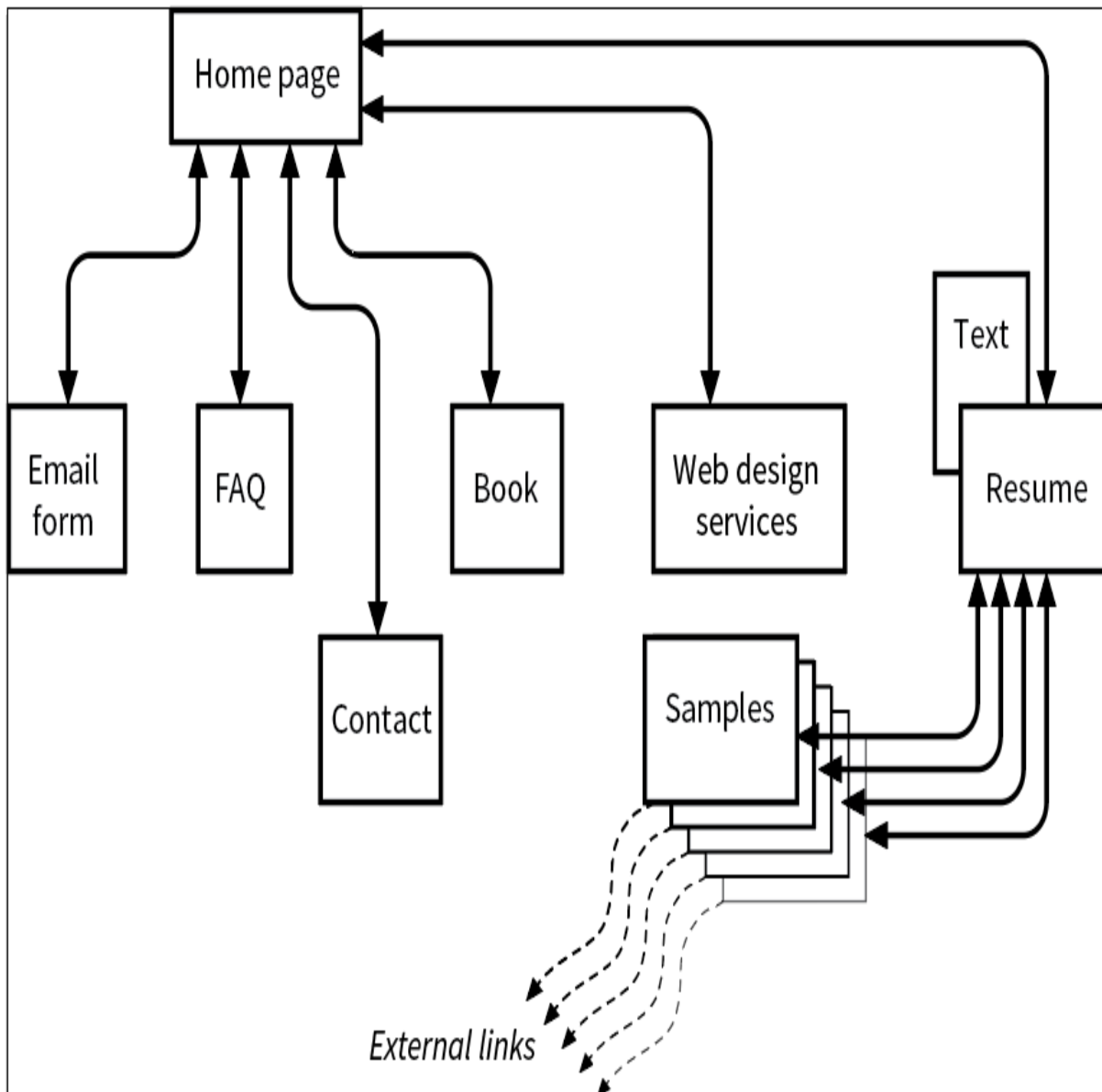


Figure 1-2. A simple site diagram

Storyboards and user flow charts

A storyboard traces the path through a site or application from the point of view of a typical user (a *persona* in UX lingo). It usually includes a script and “scenes” consisting of screen views or the user interacting with the screen. Storyboards aim to demonstrate the steps required to

accomplish tasks; they outline possible options and introduce some standard page types. [Figure 1-3](#) shows a simple storyboard. A user flow chart is another method for showing how the parts of a site or application are connected, but it tends to focus on technical details rather than telling a story: for example, “when the user does *this*, it triggers *that* function on the server.” User flow charts are helpful for documenting the steps in a process such as member registration or online payments.

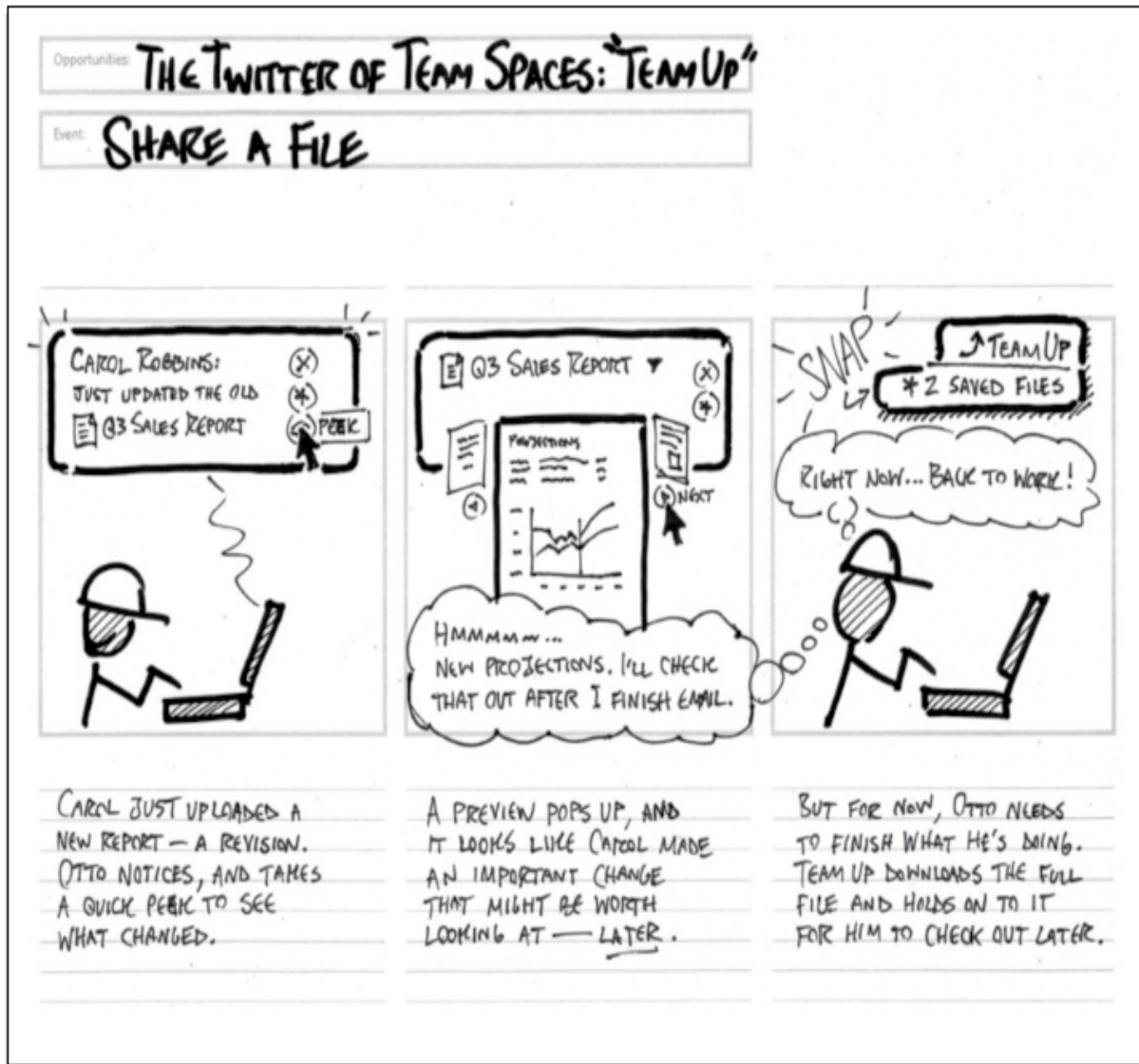


Figure 1-3. A typical storyboard (courtesy of Adaptive Path and Brandon Schauer)

There are many books on UX and interaction design, but these are a few of the classics to get you started:

- About Face: The Essentials of Interaction Design, 4th edition, by Alan Cooper, Robert Reimann, David Cronin, and Christopher Noessel (Wiley)

- Designing Interfaces: Patterns for Effective Interaction Design, 3rd edition, by Jenifer Tidwell, Charles Brewer, and Aynne Valencia (O'Reilly)
- 100 Things Every Designer Needs to Know About People, 2nd edition, by Susan Weinschenk (New Riders)
- Designing User Experience: A Guide to HCI, UX and Interaction Design, 4th edition, by David Benyon (Pearson)
- Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability, 3rd edition, by Steve Krug (New Riders)
- The Design of Everyday Things by Don Norman (Basic Books)
- The Elements of User Experience: User-Centered Design for the Web and Beyond, 2nd edition, by Jesse James Garrett (New Riders)

User interface and visual design

Because the web is a visual medium, web pages require attention to their visual presentation. First impressions are everything. The “look and feel” of a site ensures that it makes a good first impression and is consistent with the brand and message of the organization it represents. These days, the visual design is usually handles by a *user interface (UI) designer* or *visual designer*.

UI design focuses on crafting the visual experience that website users interact with. In addition to designing the layout of components on pages at various sizes, designers apply the organization's brand identity (colors,

logos, typography, and other styles) to those components in order to create websites that are impactful, effective, and accessible.

The creative team may also have a dedicated *graphic designer* who produces image assets and other collateral, or an *information designer* who specializes in visual assets meant to convey specific information, such as infographics, charts, and even interactive presentations.

There are many methods and deliverables that can be used to present a visual design to clients and stakeholders. The most traditional are sketches, often called *comps* or *mockups*, of the way the site might look, such as the home page comps shown in [Figure 1-4](#). We'll talk about popular tools for designing site interfaces later in this chapter.

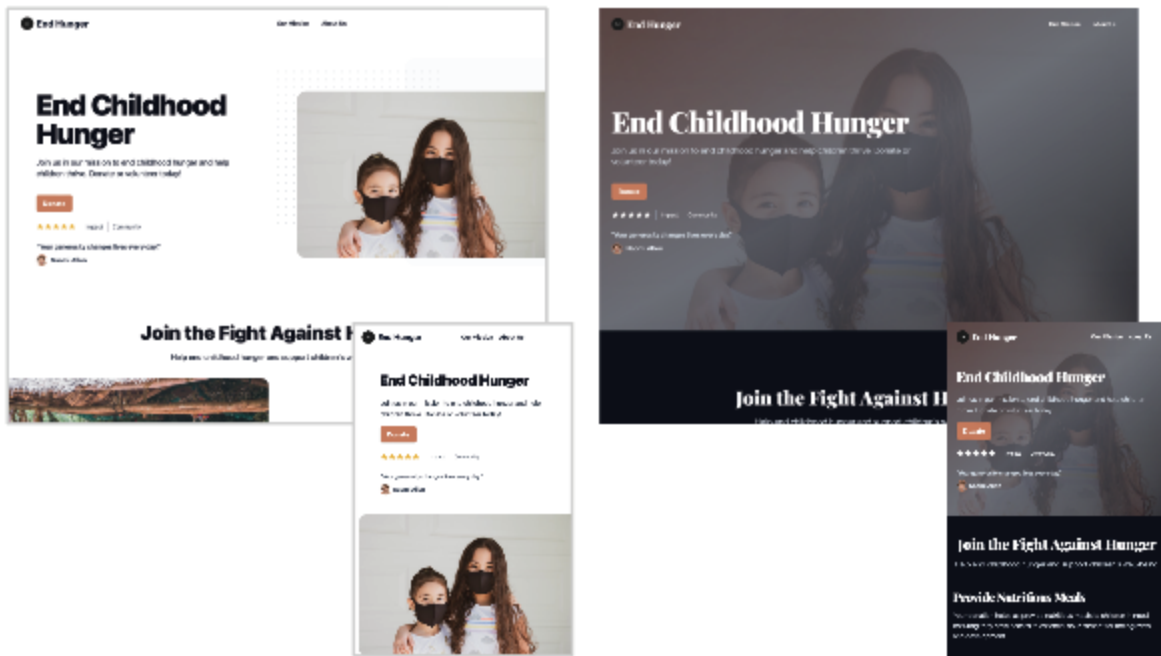


Figure 1-4. Look-and-feel comps for a simple site (designs generated on Mixo.io)

Now that sites appear on screens of all sizes, the standard practice is to take a component-driven approach to design. Page components such as page headers, search fields, navigation, calls to action, product galleries, and so on need to adapt and rearrange based on the available screen space, so UI designers must think about how every page component performs in different contexts. It's about building a visual language for the site that can be applied to various page types at various sizes.

To be sure the components are used consistently by the whole team, they are commonly stored and documented in a design system. A *design system* is a collection of reusable site components, such as UI widgets, content containers, brand styles, and more, that can be shared by a team (I'll talk more about design systems in Chapter 19, Responsive Web Design). Each component generally includes a name, image, description, and code that can be copied and pasted. Creating and managing a design system and its UI component library has become central to the responsibilities of the modern UI designer.

Some designers prefer to get sign-off on the visual identity (colors, fonts, image style, etc.) in a way that isn't tied to a specific layout like the comps shown in [Figure 1-4](#). The designer may choose to create a deliverable similar to a mood board that gives an overview of the visual design at a glance, such as the *style tiles* approach introduced by Samantha Warren ([Figure 1-5](#)). Learn more about it at styletil.es.

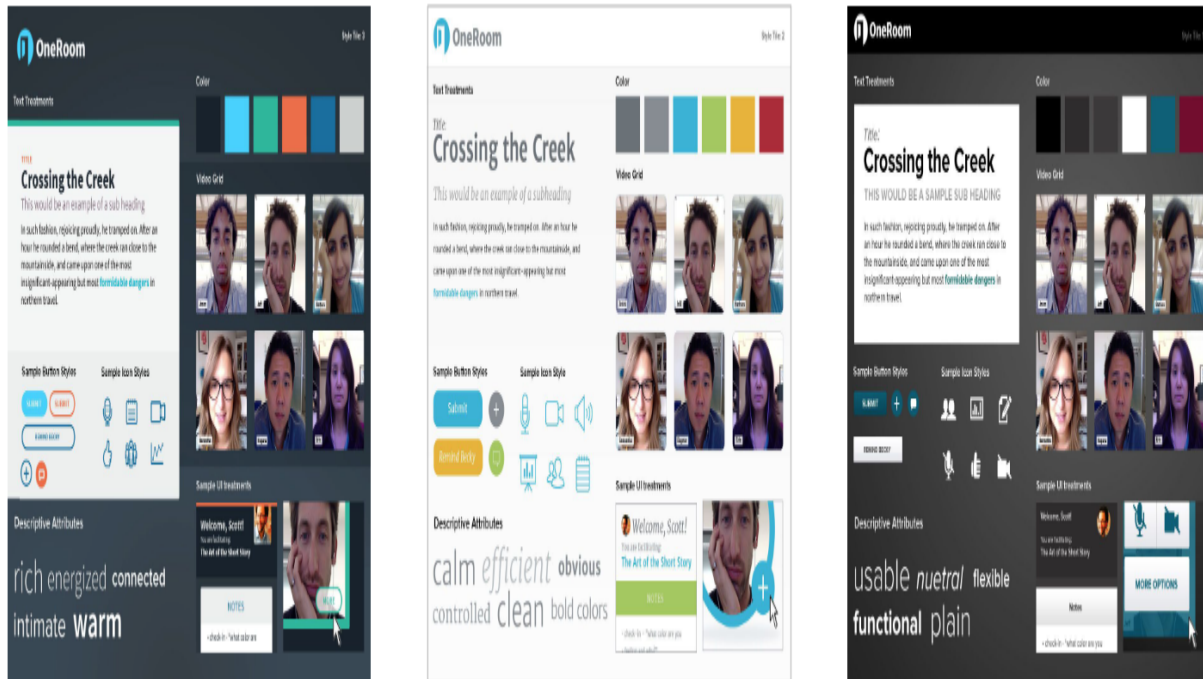


Figure 1-5. The style tiles technique, introduced by Samantha Warren

Designers may also need to produce the image assets for the site, so knowledge of how to optimize images for fastest delivery is a key skill. For large sites with lots of images, it is more common for the development team or a third-party service to handle image optimization, but this is a skill every visual designer should have. We'll discuss image optimization in Chapter 26, Image Production Strategies.

WHAT EMPLOYERS LOOK FOR IN A UX/UI DESIGNER

User experience design is in high demand these days, as businesses acknowledge the effect that thoughtful, user-centered design has on the success of their products and services. Although it is possible to get a degree in UX design from a university, UX designers come from a wide variety of backgrounds. Many start their careers as visual or product designers; some have backgrounds in technology or human–computer interaction (HCI); others come at it with experience in psychology or marketing.

If you are interested in getting into UX design, you might consider attending a UX bootcamp, which is an accelerated program that gets your skills up to speed quickly. There are many of them out there, so shop around and compare.

Bachelor's degrees in areas such as design, computer science, information architecture, HCI, or psychology are good starting points on the path to UX design. In addition to a bachelor's degree, employers typically look for the following:

- Experience working on web and mobile products, preferably in a design role (or product management for UX design)
- Familiarity with industry design tools such as Figma and the Adobe Suite
- User research and testing experience

- Experience with UX deliverables such as site diagrams, personas, flowcharts, etc.
 - An understanding of business metrics (especially for UX design)
 - A working knowledge of HTML, CSS, and JavaScript
 - Familiarity with site-building tools like Bootstrap, Tailwind CSS, or WordPress
 - Strong multitasking and organizational skills, and attention to detail
 - Clear communication skills
-

Writing Code

A large share of the website building process involves creating and troubleshooting the documents, style sheets, scripts, databases, and images that make up a site. The team that handles the creation of the files that comprise the site (or templates for pages that get assembled dynamically) is usually called the *development* or *production department*.

Development falls under two broad categories: frontend development and backend development (but see the sidebar “The Frontend and Backend Lines Have Blurred”). Once again, these tasks may fall to specialists, but it is just as common for one person or team to handle both responsibilities.

THE FRONTEND AND BACKEND LINES HAVE BLURRED

In truth, the divide between frontend and backend development isn't as clean as it once was. For example, JavaScript can now be used for just about everything, on both the frontend and the backend. For the purposes of this discussion, I've introduced JavaScript in the frontend category based on its original intent, but know that JavaScript skills are central to what has traditionally been considered backend programming as well. The lines have blurred.

Brad Frost gives his take on how development roles have evolved in his article "Front-of-the-Front-End and Back-of-the-Front-End Web Development" (bradfrost.com/blog/post/front-of-the-front-end-and-back-of-the-front-end-web-development).

Frontend development

Frontend developers are responsible for all code that appears in or relates directly to the web browser. That includes HTML, CSS, JavaScript, and related production frameworks. You will need to have *intricate* knowledge of these technologies if you want a job as a web developer. Let's take a quick look at each.

FRONTEND DEVELOPMENT

Frontend development includes the following web technologies:

- HTML
 - CSS
 - JavaScript and DOM scripting, including JavaScript-based frameworks
-

Authoring/markup (HTML)

HyperText Markup Language (HTML) is the authoring language used to create web page documents. *Authoring* is the process of preparing content for delivery on the web, or more specifically, marking up the content with HTML tags that describe it.

HTML is not a programming language; it is a *markup language*, which means it's a system for identifying and describing the various components of a document, such as headings, paragraphs, and lists. The markup indicates the document's underlying *structure* (you can think of it as a detailed, machine-readable outline). You don't need programming skills to write HTML, just patience and common sense.

The best way to learn HTML is to type out some pages by hand, as we will be doing in the exercises in Part II, HTML for Structure.

Styling (CSS)

While HTML is used to describe the content of a web page, *Cascading Style Sheets* (CSS; see *Note*) describes how that content should be displayed. The way the page looks is referred to as its *presentation*. Fonts, colors, background images, line spacing, page layout, margins, and so on are all controlled with CSS. You can even use it to add special effects and basic animation to your pages.

NOTE

When this book uses the term “style sheets,” it always refers to Cascading Style Sheets, the standard style sheet language for the World Wide Web. Style sheets (including what “cascading” means!) are discussed further in Part III, CSS for Presentation.

The CSS specification also provides methods for controlling how documents will be presented in contexts other than a browser, such as in print or when read aloud by a screen reader; however, I won’t be covering them much here.

Although it is possible to publish web pages using HTML alone, you’ll probably want to get comfortable using style sheets so you’re not stuck with the browser’s default styles. If your goal is designing websites professionally, either as a designer or as a developer, proficiency with CSS is mandatory.

THE WORLD WIDE WEB CONSORTIUM

The *World Wide Web Consortium* (called the *W3C* for short) is the organization that oversees the development of web technologies such as HTML, CSS, XML, and many more, as well as standards for web accessibility, internationalization, privacy, and security. The group was founded in 1994 by Tim Berners-Lee, the inventor of the web, at the Massachusetts Institute of Technology (MIT).

In the beginning, the W3C concerned itself mainly with the HyperText Transfer Protocol (HTTP) and the development of HTML. Now, it's laying a foundation for the future of the web by developing dozens of technologies and protocols that must work together in a solid infrastructure. For the definitive answer to any web technology question, the W3C site (w3.org) is the place to go. For more information on the W3C and what it does, see w3.org/about/.

The HTTP protocol is now managed by the Internet Engineering Task Force (IETF) HTTP Working Group (httpwg.org). In 2019, the W3C handed over development and maintenance of HTML to the *Web Hypertext Application Technology Working Group* (WHATWG; whatwg.org), which was founded by representatives of leading browser vendors in 2004, but the spec is published on the W3C site as well.

JavaScript and DOM scripting

JavaScript is a scripting language that was originally developed to add interactivity and behaviors to web pages, which is why I'm introducing it in the context of frontend development. However, over the years it has become increasingly popular and important for backend programming as well. And not only that: JavaScript has had a huge impact beyond the web as well! It powers many popular applications, such as Figma, Photoshop, Netflix, social media platforms like Facebook, and many more.

In the context of frontend web development, here are just a few interactions handled by JavaScript:

- Checking form entries for valid entries
- Swapping out styles for an element or an entire site
- Loading scrolling feeds with more content automatically
- Making the browser remember information about users
- Building interface widgets, such as embedded video players or special form inputs
- Adding animation effects

You may also hear the term *DOM scripting* used in relation to JavaScript. DOM stands for *Document Object Model*, and it refers to the standardized list of web page elements that can be accessed and manipulated using JavaScript (or another scripting language).

Frontend developers should also be familiar with JavaScript frameworks (such as React, Vue, Angular, Node.js, and others) that provide helpful structure and conventions that make writing JavaScript easier. They simplify and streamline the production process.

Web scripting definitely requires some traditional computer programming prowess. While many web developers have degrees in computer science, it is also common for developers to be self-taught. A few developers I know started by copying and adapting existing scripts, then gradually added to their scripting skills with each new project. Still, if you have no experience with programming languages, the initial learning curve may be a bit steep.

If you want to be a web developer for a living, JavaScript is a basic requirement. Designers will benefit from understanding what JavaScript can do, but may not need to learn to write it if they are working with a development team. Chapter 22, Introduction to JavaScript, will get you started.

Backend development

BACKEND DEVELOPMENT

The following technologies are typically in the domain of the backend developer:

- Server software (Apache, Microsoft IIS, nginx, and more)
 - Web application languages (Node.js, PHP, Ruby, Python, JSP, ASP.NET, JavaScript)
 - Database software (MySQL, Oracle, SQL Server, Postgres, NoSQL)
-

Backend developers focus on the server, including the applications and databases that run on it. They may be responsible for installing and configuring the server software (we'll be looking more at servers in Chapter 2, *How the Web Works*). They will certainly be required to know one or more server-side programming languages, such as PHP, Ruby, Node.js, .NET (or ASP.NET), and Python, in order to create applications that provide the functionality required by the site. It is worth noting that JavaScript is used for applications that run on the server side as well. Server-side applications handle tasks and features like forms processing, content management systems (CMSs), and online shopping, just to name a few.

Additionally, backend developers need to be familiar with configuring and maintaining databases that store all of the data for a site, such as the content that gets poured into templates, user accounts, product inventories, and

more. Some common database languages include MySQL, Postgres, NoSQL, Oracle, and SQL Server.

Backend development is well beyond the scope of this book, but it's important to know the sorts of tasks that get taken care of at the server level. You should be aware that it is possible to get functionality like shopping carts, mailing lists, and so on as prepackaged solutions from your hosting company without needing to program it from scratch.

WHAT EMPLOYERS LOOK FOR IN A WEB DEVELOPER

As you may expect, employers want to know that prospective web developers are well trained in web-related technologies. You can get that training as part of a traditional computer science college degree, through a web development bootcamp, or even through prior experience in the workplace. There are also web development certification programs available out there (for a fee, of course); although certification certainly doesn't hurt, it is not often a requirement in job listings.

Specific software and programming language requirements vary by company (for example, some are all-in on Microsoft products), but here is a list of requirements that typically appear in web developer job posts:

- Deep understanding of HTML and CSS
- Experience with modern programming languages such as JavaScript, Java, C++, or Python
- Experience with modern web development platforms such as React or Angular
- Comfortable with version control management tools, such as Git
- Microsoft .NET experience or certification (depending on context)
- Database programming experience (depending on context)
- Problem solving and analytical skills
- Clear communication skills and ability to work in a team

When looking for a job in web development, you will frequently see posts looking for “*full-stack*” *developers*, meaning people who are fluent in both frontend (HTML, CSS, JavaScript) and backend (server software, databases) technologies, as well as possibly having additional UX design skills.

Content Wrangling

Anyone who uses the title “web designer” or “web developer” needs to be aware that everything we do supports the process of getting our content, message, or functionality to our users. Furthermore, good writing can help the user interfaces we create be more effective, from button labels to error messages.

Of course, someone needs to create all that content and maintain it—don’t underestimate the resources required to do this successfully. Good writers and editors are an important part of the site creation team. In addition, I want to call your attention to two content-related specialists in modern web development: the information architect (IA) and the content strategist.

Information architecture

An *information architect* organizes the content logically and for ease of findability. They may be responsible for search functionality, site diagrams

(such as the one shown in [Figure 1-2](#)), and how content and data are organized on the server. Information architecture is inevitably entwined with UX and UI design as well as content management. If you like organizing or are enthusiastic about taxonomies, information architecture may be the job for you. For more information on this field as it relates to the web, the books *Information Architecture: For the Web and Beyond*, 4th edition, by Louis Rosenfeld, Peter Morville, and Jorge Arango (O'Reilly) and *Everyday Information Architecture* by Lisa Maria Marquis (A Book Apart) will serve you well.

Content strategy

When the content isn't right, a site can't be fully effective. A *content strategist* makes sure that every bit of text on a site, from long explanatory text down to the labels on buttons, supports the brand identity and marketing goals of the organization. Content strategy may also extend to data modeling and content management on a large and ongoing scale, such as planning for content reuse and updating schedules. The content strategist's responsibilities might also include how the organization's voice is represented on social media. For more reading about content strategy, try *From Solo to Scaled: Building a Sustainable Content Strategy Practice* by Natalie Marie Dunbar (Rosenfeld Media), or search for books written by Kristina Halvorson and Margot Bloomstein.

WHAT EMPLOYERS LOOK FOR IN A WEB CONTENT STRATEGIST

If guiding a company's content to make sure it's consistent with its brand identity and business goals sounds like an exciting way to make a living, you may want to become a content strategist. Although job requirements vary by company and position, employers generally look for some combination of the following qualifications:

- A bachelor's degree in subjects like (but not limited to) English, technical writing, marketing, communications, or user experience
- Experience in marketing and branding
- Experience with project or account management
- Strong organizational skills and attention to detail
- Communication skills, both speaking and writing
- Problem solving and strategic thinking
- Leadership skills and the ability to work well as part of a creative team

Other Roles

Not surprisingly, there are a myriad of other roles that contribute to the creation and maintenance of a site, all of which benefit from some understanding of the basics of web development as taught in this book. These include:

Product manager

The product manager of a website or application guides its design and development in a way that meets business goals. This member of the team must have a thorough understanding of the target market as well as the processes involved in the creation of the site itself. Product managers develop the overall strategy for the site from a marketing perspective, including how and when it gets released.

Project manager

The project manager coordinates the designers, developers, and everyone else who is working on the site. They manage things like timelines, development approaches, deliverables, and so on. The project manager works with the product manager and other product owners to make sure that the project gets done on time and on budget.

SEO specialist

A website or application isn't much good if nobody knows it exists, so it is crucial that a site be easily found by search engines. *Search engine optimization (SEO)* is a discipline focused on tweaking the site structure and code in a way that increases the chances that it will be highly ranked in search results. There may be an SEO specialist on the in-house team, or a company may choose to hire an outside SEO firm. SEO is sometimes perceived as a dark art, but there are many ways to improve findability that are not underhanded. In fact, the number one technique for improving SEO is simply having good content with savvy HTML markup.

Multimedia producers

One of the cool things about the web is that you can add multimedia elements to a site, including sound, video, animation, and even interactive games. Creating multimedia elements is generally best left to artists and technicians in those fields, although they may be part of the web team if video, animation, or interactivity is core to the site's mission.

That concludes our stroll through the virtual village of workers involved in the creation of a website. The larger the site, the more likely it is that each team member will have a narrow specialization and a job title like “UX Lead for Error Messages.” More likely, everybody on the team will possess a spectrum of skills, and the lines between disciplines will blur. For example, I do interaction and user interface design, visual design, HTML, and CSS, but I don't write JavaScript or work on the server.

NOTE

This book should give you a foundation in the frontend technologies that will prepare you for a number of roles.

(SOFT) SKILLS EVERY WEB DESIGNER NEEDS

We've focused on quite a few technical skills that will be helpful in building websites. I'd like to mention a few more (often overlooked) skills that are just as critical to your success:

Communication skills

In your work, you will need to communicate in person, on the phone, in email, and in text messaging tools with clients, team members, and superiors. Be clear, proactive, and straightforward with what you have to say. Good communication requires not only that you express yourself clearly, but also that you be a good listener. Make sure that you understand the issues being discussed, and don't be afraid to ask for clarification if you don't.

Flexibility

It's important to be able to change direction quickly, because not only do web technologies evolve rapidly, but you will no doubt be thrown curveballs in your day-to-day work as well. For example, you may arrive at work one day to find that the client has changed your priorities completely. You might even find that they've cancelled your project entirely. You may be asked to learn new skills and shift positions in the team. Staying adaptable is the key to survival.

Critical thinking and good judgment

Problem solving is central to all of the disciplines related to web design, so you need to be able to use critical thinking skills to come up with solutions and always employ basic common sense.

A good attitude

Creating sites means being part of a team, even if you work at home as a freelancer. Be mindful that the attitude with which you approach your work is contagious, so strive to be a positive and friendly team member.

Collaboration

Remaining open-minded to new ideas and willing to compromise for the greater good are keys to successful collaboration.

Tools of the Trade

It should come as no surprise that professional web designers require a fair amount of gear, both hardware and software. One question frequently asked is, “What do I need to buy?” I can’t tell you specifically, but I will provide an overview of the typical tools of the trade.

Equipment

For a comfortable web development environment, I recommend the following equipment:

A solid, up-to-date computer

Use whatever you have and are comfortable with—macOS, Windows, or Linux is fine. Design departments tend to be Mac-based. For development work, Windows and Linux are popular (although many developers choose to use Macs as well). Although it's nice to have a super-fast machine, the files that make up web pages are small and tend not to be too taxing on computers. Unless you're getting into sound and video editing, don't worry if your current setup is not the very latest and greatest.

A large monitor

Though it's not a requirement, a large monitor makes life easier. The more monitor real estate you have, the more windows and control panels you can have open at the same time. If you're using a large monitor, just make sure you design for users with smaller screens in mind.

A second computer for testing

Many designers and developers find it useful to have a test computer running a different platform than the computer they use for development (i.e., if you design on a Mac, test on a PC). Because sites may work differently on Macs than on Windows machines, it's critical to test your pages in as many environments as possible. If you are a hobbyist web designer working at home, you can install an alternate operating system on your Mac or Windows PC, as discussed in the “Run Windows on

Your Mac (and Vice Versa)” sidebar. Or go visit a friend and check your pages on their machine.

RUN WINDOWS ON YOUR MAC (AND VICE VERSA)

You may not need to use a separate computer to have access to both Windows and macOS. There are ways to install the alternate operating system on your machine with virtual machine (VM) software.

On newer Macs with M1 or M2 chips, you can use a software program called Parallels (parallels.com) to run Windows 11 (the latest version as of this writing) in a separate partition, so you can switch between operating systems. Macs with Intel chips can run Boot Camp to install Windows, but they can only boot up in one OS or the other, not switch back and forth. All VM products for Mac require that you purchase a copy of Microsoft Windows, but it sure beats buying a whole machine.

There are also several methods for installing macOS on computers running Windows 10 or 11. One popular method is to download and install Oracle VirtualBox (virtualbox.org) and create a partition to run macOS. The full instructions are longer than I can address here, but a quick web search will turn up several helpful tutorials.

Mobile devices for testing

It is absolutely critical that you test the appearance and performance of your site on browsers on smartphones and tablet devices.

Web Production Software

There's no shortage of software available for creating web pages. In the early days, we just made do with tools originally designed for print. Today, there are wonderful tools created specifically with web design in mind that make the process more efficient. It's a delicate business listing software in a book such as this, because a) there are *so many* programs, b) everyone has their personal favorite, and c) new tools come along so rapidly that as you're reading this chapter there are likely already newer, cooler options available that didn't exist as I wrote it.

That said, here is a general overview of the types of software that comprise the tools of our trade, along with a few specific mentions of the most popular in each class (at the time of writing).

NOTE

To do the exercises in this book, all you'll need is the text editor that came with your operating system and free image creation software. There is no need to purchase anything to follow along.

Coding tools

Although you can get by with the simple text editors that come with your computer, a dedicated code editor makes the task of writing HTML, CSS, and JavaScript much easier. Code editors understand the syntax of the code you write, so they can do things for you like color coding, error detection, code validation, and automatically finishing simple tasks like closing HTML tags. Some provide page previews so you can view the results of your code as you work.

Here are just a few of the better-known code editors for web production that are worth exploring:

- Visual Studio Code (code.visualstudio.com; free)
- Phoenix Code (phcode.dev; a free browser-based code editor with preview)
- Sublime Text (sublimetext.com)
- Adobe Dreamweaver (adobe.com/products/dreamweaver.html)
- CodeKit (codekitapp.com; *Mac only*)

[Figure 1-6](#) shows how an HTML document looks in the free Visual Studio Code editor.

Figure 1-6. The Visual Studio Code editor

Another great tool is CodePen (codepen.io), an easy-to-use online code editor that shows a preview of your page as you work. CodePen is much more than just a code editor, though; it's also a community where people share their work and make it available to others. For beginners, it's a great place to learn from code examples. It's free to get started, but if you want to host images and other files and get access to advanced features, there is a monthly or annual fee.

User interface and layout tools

There is a new breed of user interface design tools made specifically for websites and other applications. Because they have been designed from the ground up with interface design in mind, they seem to anticipate a web designer's every need. Interface design tools make it easy to design multiple layouts (such as layouts at various screen sizes) and to export images and code for use in production. Some allow basic interactivity, such as clicks and swipes, so your prototypes can be shared online and used for basic interface testing.

Figma (figma.com), shown in [Figure 1-7](#), is by far the most popular design tool at the time of writing because of its prototyping capabilities, developer mode, cross-platform support, and ease of collaboration. Other options include the following:

- Sketch (sketch.com; *Mac only*)
- UXPin (uxpin.com)

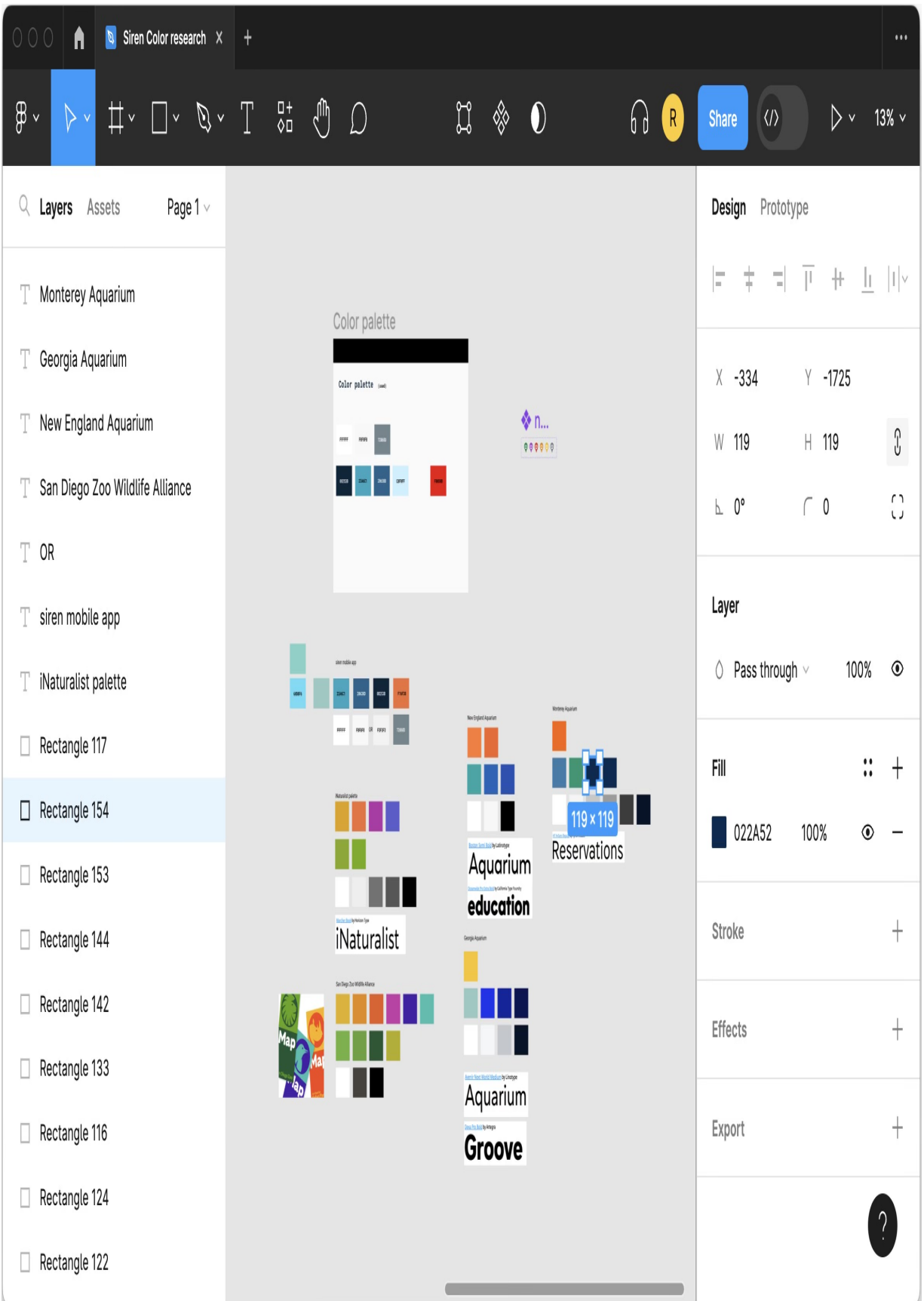


Figure 1-7. The Figma user interface design tool

Image editing tools

It's certainly possible to create all the images you need for a site by using one of the interface design tools just listed. There are also programs that focus solely on image creation that can export files in web-appropriate formats. For professional designers, the Adobe Creative Cloud (adobe.com) suite of tools, which includes Photoshop ([Figure 1-8](#)), Illustrator, and other high-end design tools, may be worth the investment.



Figure 1-8. Adobe Photoshop: the professional standard for image editing

If the Adobe monthly subscription fee is out of reach, you can try lower-cost alternatives that provide many of the same features. The number of graphics tools out there is dizzying, so I'll mention just a few here:

- GIMP (free, open source; gimp.org; *all platforms*)
- Corel PaintShop Pro (for photo editing; paintshoppro.com; *Windows only*)
- Corel Draw (for vector drawing; coreldraw.com; *Windows only*)
- Pixelmator Pro (pixelmator.com; *Mac only*)

The following image editors work right in your browser, without the need to download a program:

- SumoPaint (sumopaint.com; free online tool; paid Pro version also available)
- Pixlr (pixlr.com; free)
- Photopea (photopea.com; free)

A variety of browsers

One of the biggest challenges for web designers is that our sites may look and behave differently from browser to browser. For this reason, it is critical that we test our designs early and often on the widest range of browsers

possible, both desktop and mobile. These are the most popular browsers, and the ones designers and developers typically keep around for testing:

Desktop

- Chrome (google.com/chrome)
- Safari (apple.com/safari; *Mac only*)
- Microsoft Edge (microsoft.com/edge; see Note)
- Firefox (mozilla.org)
- Opera (opera.com)

NOTE

Although once dominating browser statistics with over 95% of web traffic, Internet Explorer usage has reduced to less than 1% now that Microsoft has discontinued it in favor of Edge. There may be special circumstances under which you need to support Internet Explorer, and if that is the case, you will find many articles online describing what it does and doesn't support, as well as workarounds. In general, I will not be providing Internet Explorer-related information in this book, with a few exceptions.

Mobile

- Chrome
- Safari iOS (Apple App Store only)
- Samsung Internet (Google Play only)
- Opera Mini
- Firefox

- Microsoft Edge

Unless otherwise noted, these mobile browsers are available for download on the Apple App Store and Google Play.

File management and transfer tools

Web design and development involves a lot of moving files around, particularly from the computer where you do your work to the server computer that hosts the site. There are various tools that come in handy here, for version control and file transfer.

Git and GitHub

These days, most web development teams use a version control management program called Git. Git keeps copies of every version of a document in a “repository” and allows you to roll back to previous iterations. It also makes it very easy for teams to share and contribute to a central codebase. Many organizations and individuals use the online service GitHub to host their Git repositories (you’ll see github.com in the links for many resources in this book). In the modern web development process, files are most often deployed to the server via Git, and it has become central to the development process.

Knowledge of Git is critical if you are on a professional team and useful even if you are working alone. I recommend taking the time to read my

brief introductory article “Version Control with Git and GitHub,” available at learningwebdesign.com/articles.

FTP

The traditional way to move files across the internet is to use an *FTP* (short for *File Transfer Protocol*) program. If you are not working on a team using Git or more elaborate build and deployment tools, you can use a dedicated FTP application to move your files to the server. You will find that many hosting services offer their own FTP tools for uploading your files. Many of the code editors listed earlier also include built-in FTP functionality. A few popular standalone FTP programs include:

- Filezilla (filezilla-project.org; free; *all platforms*)
- Cyberduck (cyberduck.io; *Mac and Windows*)
- WinSCP (winscp.net; free; *Windows only*)
- Transmit (panic.com/transmit/; *Mac only*)

Command-line tools

You may also find it useful to have a *terminal application* (command-line tool) that allows you to type Unix commands for setting file permissions, moving or copying files and directories, or managing the server software. Command-line tools have a number of uses in web design and development workflows. For a quick introduction, see my article “An Introduction to the

Command Line,” available at learningwebdesign.com/articles. The following are examples of terminal applications:

- Terminal (installed with macOS; shown in [Figure 1-9](#))
- Cygwin (cygwin.com; Linux emulator for Windows that includes a command-line tool)

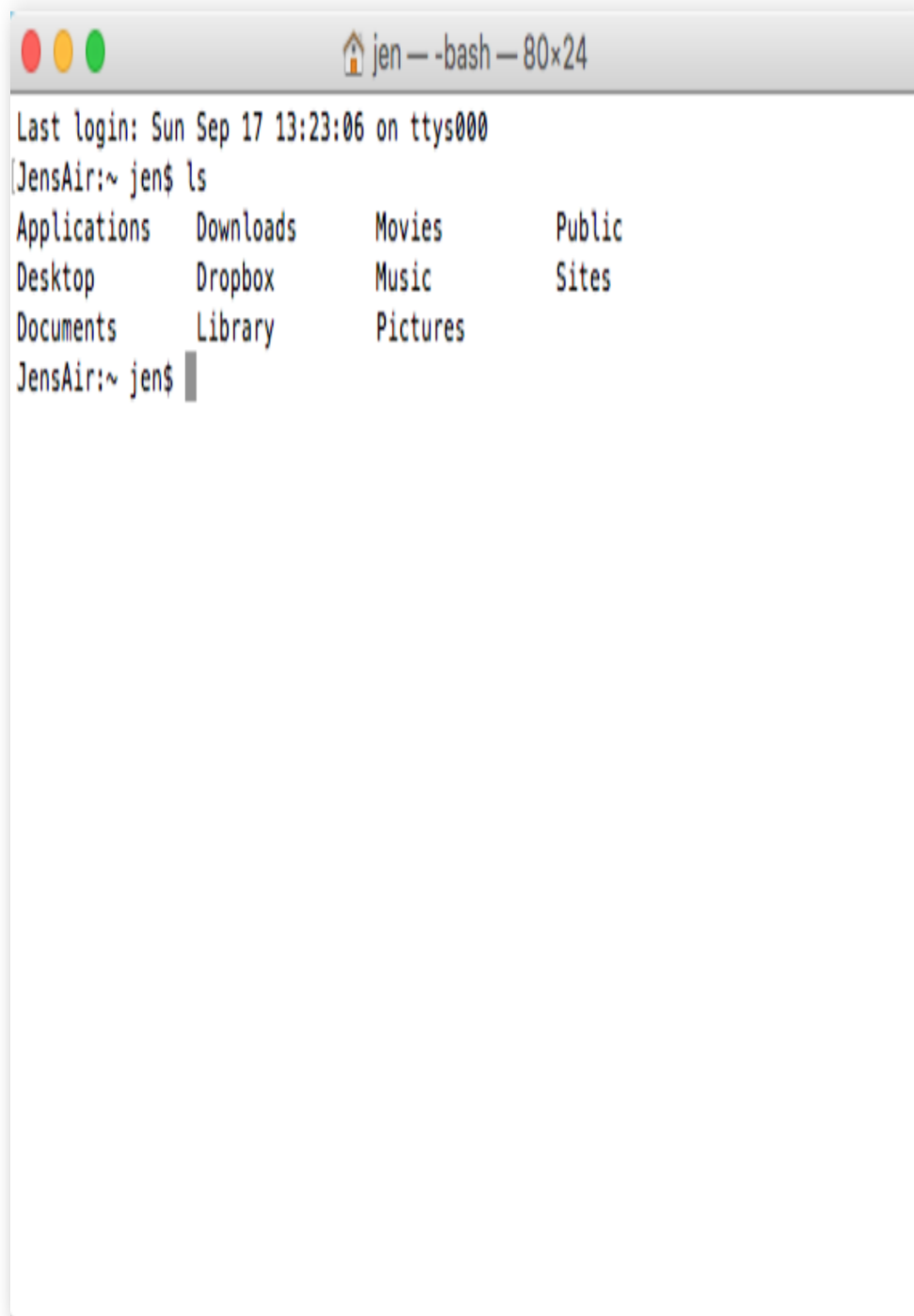


Figure 1-9. The Terminal command-line tool for macOS

What You've Learned

This chapter has given you an overview of the many roles and responsibilities that contribute to the creation of websites. Hopefully, you've come away realizing that you don't need to learn everything—and even if you do want to learn everything eventually, you don't need to learn it all at once. So relax, and don't worry. The other good news is that, while many professional tools exist, it is possible to create a basic website and get it up and running without spending any money by using freely available tools and your existing computer setup.

As you'll soon see, it's easy to get started making web pages—you will be able to create simple pages by the time you're done reading this book. From there, you can continue adding to your bag of tricks and find your particular niche in the website creation ecosystem. In the meantime, try answering the questions in Exercise 1-1.

EXERCISE 1-1. TAKING STOCK

Now that you're taking that first step in learning web design, it might be a good time to take stock of your assets and goals. Using the lists in this chapter as a general guide, try jotting down answers to the following questions:

- What are your web design goals? To become a professional web designer? To make personal websites only? To work in a management position?
 - Which aspects of web design interest you the most? Writing code? Creating the look and feel and visual assets? Organization? Everything?!
 - What current skills do you have that will be useful in creating web pages?
 - Which skills will you need to brush up on?
 - Which hardware and software tools do you already have for web design?
 - Which tools do you need to buy? Which tools would you like to buy eventually?
-

Chapter 2. How the Web Works

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 2nd chapter of the final book.

If you’d like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at jbleiel@oreilly.com.

In this chapter:

- An explanation of the web as it relates to the internet
- The role of the server
- The role of the browser
- URLs and their components
- The anatomy of a web page

I got started in in web design in early 1993—pretty close to the start of the web itself. That’s a roughly a third of a century ago (gasp!), but I still distinctly remember the first time I looked at a web page. It was difficult to tell where the information was coming from and how it all worked.

This chapter sorts out the pieces and introduces some important terminology. We'll start with the big picture and work down to specifics.

Here is what you can expect to learn in this chapter:

- The difference between the internet and the web
- The role of the server and browser
- The structure of URLs (web page addresses)
- How HTML and style sheets work together
- How the browser communicates with the server when you go to a web page

The Internet Versus the Web

No, it's not a battle to the death, just an opportunity to point out the distinction between two words that are often used interchangeably.

The *internet* is an international network of connected computers. No company owns the internet; it is a cooperative effort governed by a system of standards and rules. The purpose of connecting computers together, of course, is to share information. There are many ways information can be passed between computers, including via email (POP3/IMAP/SMTP), file transfer (FTP), Secure Shell (SSH), and various other specialized modes upon which the internet is built. These standardized methods for transferring data or documents over a network are known as *protocols*.

NOTE

The web is a subset of the internet. It is just one of many ways information can be transferred over networked computers.

The *web* (originally called the World Wide Web, thus the “www” in site addresses) is just one of the ways information can be shared over the internet. It is unique in that it allows documents to be linked to one another via *hypertext* links, thus forming a huge “web” of connected information. The web uses a protocol called *HTTP* (*HyperText Transfer Protocol*). That acronym should look familiar because it is the first four letters of nearly all website addresses, as we’ll discuss in an upcoming section.

A BRIEF HISTORY OF THE WEB

The web was born in a particle physics laboratory (CERN) in Geneva, Switzerland, in 1989. There, a computer specialist named Tim Berners-Lee first proposed a system of information management that used a “hypertext” process to link related documents over a network. He and his partner, Robert Cailliau, created a prototype and released it for review. For the first several years, web pages were text-only. By 1992, the world had only about 50 web servers, total.

The real boost to the web’s popularity came in 1992 when the first graphical browser (NCSA Mosaic) was introduced, and the web broke out of the realm of scientific research into commercial mass media. The ongoing development of web technologies is overseen by the World Wide Web Consortium (W3C).

If you want to dig deeper into the web’s history, check out the W3C’s history archives at w3.org/History.html.

Fun fact: If you look at that page, you’ll see a July 1993 entry for the first “WWW Wizards Workshop.” Although I did not attend that meeting, I *did* design the commemorative t-shirt!

Serving Up Your Information

Let's talk more about the computers that make up the internet. Because they “serve up” documents upon request, these computers are known as *servers*. More accurately, the server is the software (not the computer itself) that allows the computer to communicate with other computers; however, it is common to use the word “server” to refer to the computer as well. The role of server software is to wait for a request for information, and then retrieve and send that information back as quickly as possible.

In order for a computer to be part of the web, it must be running special web server software that allows it to handle HTTP transactions. Web servers are also called *HTTP servers*.

There are many server software options out there, but the two most popular are Apache and nginx (both *open source* software; see sidebar). Both Apache and nginx are freely available for Unix-based computers and Windows, though they're most often run on Unix-based machines. Apache also comes installed by default on Macs running macOS.

OPEN SOURCE

Open source software is developed as a collaborative effort with the intent to make its source code available to other programmers for use and modification. By allowing anyone to access, modify, and distribute the source code, open source fosters collaboration and a sense of community among developers worldwide. Because developers can build on each other's work rather than starting from scratch, it accelerates the pace of development. It also leads to improved software quality, as many eyes can identify bugs.

Open source programs are usually free to use, so startups and individuals can create and use powerful tools without the high cost of proprietary solutions. Examples of open source software include the Linux operating system, the Bootstrap web development framework, the Mozilla Firefox browser, the Git version control management system, the Drupal content management system, and many more.

Anyone can contribute to an open source project. It's a great way to learn, discover, and be a part of the tech community.

Every computer and device (router, smartphone, car, etc.) connected to the internet is assigned a unique numeric *Internet Protocol (IP) address*. For example, as I write this, the computer that hosts *oreilly.com* has the IP address 199.27.145.64. All those numbers can be dizzying, so fortunately,

the *Domain Name System (DNS)* was developed to allow us to refer to that server by its *domain name*, “oreilly.com”, as well. The numeric IP address is useful for computer software, while the domain name is more accessible to humans. Matching the text domain names to their respective numeric IP addresses is the job of a separate *DNS server*.

It is possible to configure a web server so that more than one domain name is mapped to a single IP address, allowing several sites to share a single server.

INTRANETS AND EXTRANETS

When you think of a website, you generally assume that it is accessible to anyone surfing the web. However, many organizations take advantage of the awesome information sharing and gathering power of websites to exchange information just within their own networks. These private web-based networks are called *intranets*. They are created and function like ordinary websites, but they use special security devices called *firewalls* that prevent the outside world from seeing them. Intranets have lots of uses, such as sharing human resource information or providing access to inventory databases.

An *extranet* is like an intranet, but it allows access to select users outside of the organization. For example, a manufacturing company may provide its customers with passwords that allow them to check the status of their orders in the company's orders database. Passwords determine which slice of the company's information is accessible.

A Word About Browsers

You now know that the server does the serving, but what about the other half of the equation? The software that does the requesting is called the *client*. People use desktop browsers, mobile browsers, and other assistive technologies (such as screen readers) as clients to access documents on the

web. The server returns the documents for the browser (also referred to as the *user agent* in technical circles) to display.

The requests and responses are handled via the HTTP protocol, mentioned earlier. Although we've been talking about "documents," HTTP can be used to transfer images, movies, audio files, data, scripts, and all the other web resources that commonly make up websites and applications.

It's common to think of a browser as a window on a computer monitor with a web page displayed in it. These are known as *graphical browsers* or *desktop browsers*, and for a long time, they were the only web-viewing game in town. The most popular desktop browsers as of this writing include Chrome, Safari, Edge, and Firefox (Opera and old versions of Internet Explorer make up a tiny percentage of usage).

SERVER SIDE AND CLIENT SIDE

Often in web design, you'll hear references to "client-side" or "server-side" applications. These terms are used to indicate which machine is doing the processing. *Client-side* applications run on the user's machine (also referred to as the *frontend*), while *server-side* applications and functions use the processing power of the server computer (the *backend*).

These days, however, more than half of web traffic comes from mobile browsers on smartphones and tablets such as Safari on iOS, Android and

Chrome browsers on Android devices, Opera Mini, and a myriad of other default and installable mobile browsers (see en.wikipedia.org/wiki/Mobile_browser for a complete list). Navigating the web on a touchscreen is the new normal.

In this book and in the HTML/CSS specs, you will commonly see the more technical term *viewport* to refer to the area in which a web page is displayed. The viewport may be a resizable browser window on a computer monitor, or the entire screen of a handheld device displaying a browser application.

It's also important to keep alternative web experiences in mind. Users with impaired sight may be listening to a web page read by a screen reader (or simply make their text extremely large). Users with limited mobility may use assistive technology such as joysticks or voice commands to access links and enter content. The sites we build must be accessible and usable for all users, regardless of their browsing experiences.

The web is also finding its way onto smart TVs and gaming systems, where users access our pages with TV remotes or Xbox controllers. You never know where the web will pop up next!

BROWSER RENDERING ENGINES

The program that is responsible for converting HTML, CSS, and JavaScript into what you see rendered on the screen is called a rendering engine (a.k.a. browser engine or layout engine). Browsers that you use on desktop computers and mobile devices are made up of rendering engines as well as other code used for their own user interfaces and functionality. Although I talk a lot about which browsers support particular functions in this book, I’m technically referring to the browser’s rendering engine. Multiple browsers often share a rendering engine; for example, the Blink engine powers Chrome, Microsoft Edge, Opera, and a variety of Android browsers. [Table 2-1](#) lists the rendering engines used by the most popular web browsers today. For more information, search *Wikipedia.org* for “comparison of web browser engines” and “comparison of web browsers.”

Table 2-1. Current browsers and their rendering engines

Browser	Rendering engine
Chrome 28+	Blink (forked from WebKit)
Firefox (all)	Gecko (except Firefox for iOS, which uses WebKit)
Safari and Safari iOS (all)	WebKit
MS Edge (79+)	Blink (forked from WebKit)
Opera 15+	Blink (forked from WebKit)
Internet Explorer 4– 11	Trident

The reality is that pages may look and perform differently from browser to browser. This is due to varying support for web technologies, varying device capabilities, and the users' ability to set their own browsing preferences. It is one of the most challenging aspects of designing and

developing for our medium. The “Statistics and Support” sidebar presents some online tools that help.

SUPPORT AND STATISTICS

New HTML, CSS, and other web technology features are being added all the time, and it takes browsers a while to support them. And even when new browsers add support, it takes the older browsers some time to fade away. You’ll see me mention browser support frequently in this book. One online resource that is critical for navigating the support landscape is Can I Use ([CanIUse.com](https://caniuse.com)). You can search for the feature you want to use, and the site provides support information for all desktop and mobile browsers. There is also a view that shows support by year rather than browser version.

If you are curious about which browsers are most common and which are still hanging on, you can get up-to-date browser statistics at Statcounter (gs.statcounter.com). As the old web developer saying goes, however, “The only browser stats that matter are your own,” so while the global stats provide a good starting point, paying attention to your own web traffic is the best way to make decisions about cutting-edge features.

NOTE

Pages may look and perform differently from browser to browser.

Web Page Addresses (URLs)

Every page and resource on the web has its own special address called a *URL*, which stands for *Uniform Resource Locator*. It's nearly impossible to get through a day without seeing a URL (pronounced "U-R-L," not "erl") plastered on the side of a bus, printed on a business card, or broadcast on a television commercial. Web addresses are fully integrated into the modern vernacular.

Some URLs are short and sweet. Others may look like crazy strings of characters separated by dots (periods) and slashes, but each part has a specific purpose. Let's pick one apart.

The Parts of a URL

A complete URL is generally made up of three components: the protocol, the site name, and the absolute path to the document or resource, as shown in [Figure 2-1](#).

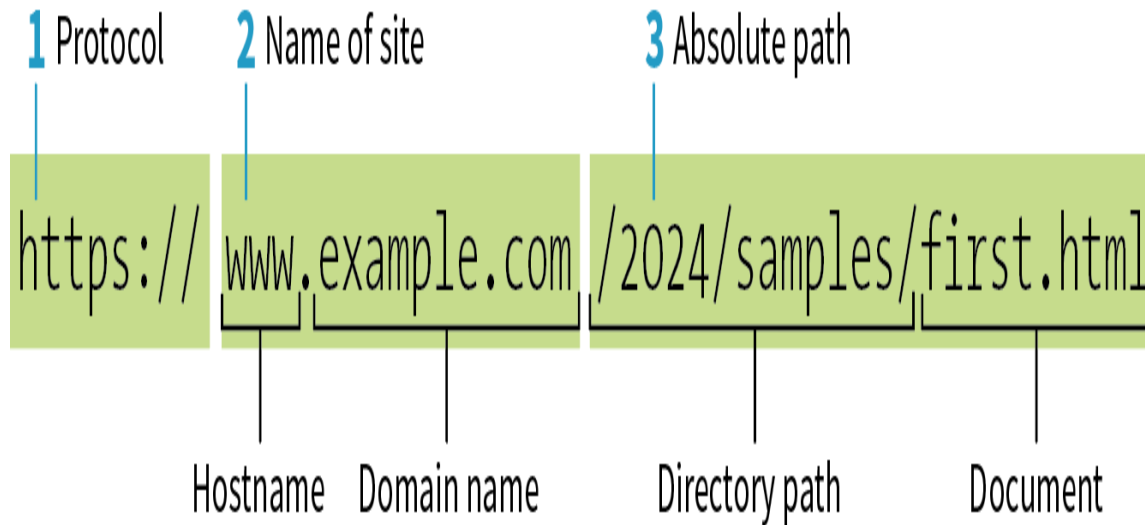


Figure 2-1. The parts of a URL

Let's break this example down:

1 `https://`

The first thing the URL does is define the protocol that will be used for that particular transaction. The letters “https” let the server know to use *HTTPS (HyperText Transfer Protocol Secure)*, or get into secure “web mode” (my made-up term). This is one of many protocols used to transfer information over the internet. Defining the protocol as HTTPS indicates the client is requesting the transfer of encrypted text and data for a web page.

You may also see a URL begin with `http://` (no “s”), but HTTP is rapidly fading away in favor of the security provided by HTTPS. You can learn a little more in the “Moving from HTTP to Secure by Default” sidebar.

2 `www.example.com`

The next portion of the URL identifies the website by its *domain name*. In this example, the domain is “example.com.” The “www.” part at the beginning is the particular *hostname* at that domain. The hostname “www” has become a convention, but it’s not a rule. In fact, sometimes the hostname may be omitted. There can be more than one website at a domain (called *subdomains*). For example, there might also be sites at development.example.com, clients.example.com, and so on.

3 `/2024/samples/first.html`

This is the absolute path through directories on the server to the requested HTML document, first.html. The words separated by slashes are the directory names, starting with the root directory of the host (as indicated by the initial `/`). Because the internet originally comprised computers running the Unix operating system, our current way of doing things still follows Unix rules and conventions, hence the `/` separating directory names.

To sum it up, the URL in [Figure 2-1](#) says it would like to use the HTTPS protocol to connect to a web server on the internet called “www.example.com” and to request the document first.html, located in the samples directory, which is in the 2024 directory.

Since the dawn of time, or at least the dawn of the web, the vast majority of traffic was served over the HTTP protocol. HTTPS was a rarity used only for particularly sensitive traffic, such as shopping online with a credit card or using a banking site.

They're very similar—in fact, HTTPS is an extension of HTTP—with the notable exception that HTTPS encrypts the data when it is sent between the user's client and the server, whereas in HTTP, data is sent as plain text.

As of this writing, around 80% of pages (and growing!) use HTTPS, and for good reason. Not only is it a good idea to keep your users' data secure in transit, but browser vendors have started to push much stronger incentives to use HTTPS. A lot of newer web technologies are available only if your site is served over HTTPS. Each major browser now also prominently displays some sort of “not secure” indicator in the address bar for any site using HTTP. On top of this, Google looks at whether or not a site uses HTTPS when factoring in how to rank the site in its search engine.

HTTPS works in tandem with another protocol, SSL (for Secure Sockets Layer), which needs to be enabled on the server for secure transactions to work. While there used to be an added cost for enabling SSL, today most hosting companies have options for enabling it for free, and the Let's Encrypt project (letsencrypt.org) provides free SSL certificates for anyone.

Keep in mind that while HTTPS protects data as it is sent to the server, that doesn't mean your site is "secure" and safe from hackers. It's a start, but not a complete solution.

Simplified URLs

Obviously, not every URL you see is so lengthy. To get to O'Reilly's site, for example, you'd expect to type `oreilly.com` instead of `https://www.oreilly.com/index.html`. Here's why that works.

Skipping the protocol

Because nearly all web pages use HTTPS, the `https://` part is often just implied. This is the case when site names are advertised in print or on TV, as a way to keep the URLs easy to remember.

Additionally, browsers are programmed to add `https://` automatically as a convenience to save you some keystrokes. It may seem like you're leaving it out, but it is being sent to the server behind the scenes.

When we begin using URLs to create hyperlinks in HTML documents in Chapter 6, Adding Links, you'll learn that it is necessary to include the protocol when making a link to a web page on another server.

Pointing to default files

Many addresses do not include a filename, but simply point to a directory, like these:

```
https://www.oreilly.com  
https://www.example.com/resume/
```

When a server receives a request for a directory name rather than a specific file, it looks in that directory for a default document, typically named `index.html`. So when someone types the previous URLs into their browser, these are the files they will actually see:

```
https://www.oreilly.com/index.html  
https://www.example.com/resume/index.html
```

The name of the default file (also referred to as the *index file*) may vary, and depends on how the server is configured. In these examples, it is named `index.html`, but some server software uses the filename `default.htm`. If your site uses server-side programming to generate pages, the index file might be named `index.php` or `Default.aspx`. Check with your server administrator or the tech support department at your hosting service to make sure you give your default file the proper name.

Another thing to notice is that in the first example, the original URL did not have a trailing slash to indicate it was a directory. If the slash is omitted, the server checks to see if the request is a file or a directory. If it is a directory, the server asks the browser to send the request again with a slash. In the end, the slash is included for directories, even if it isn't included the first time it is entered.

NOTE

Include the trailing slash in the URLs in links to avoid a needless second trip to the server (a small hit to performance).

The index file is also useful for security. Some servers (depending on their configuration) display the contents of the directory if the default file is not found. Figure 2-2 shows how the documents in the housepics directory are exposed as the result of a missing default file. Your server administrator may add protections to prevent your directories from displaying in the browser if the index file is missing.

 <https://example.com/housepics>



Providing the URL for a directory (rather than a specific filename) prompts the server to look for a default file, typically called [index.html](#).

Some servers are configured to return a listing of the contents of that directory if the default file is not found.

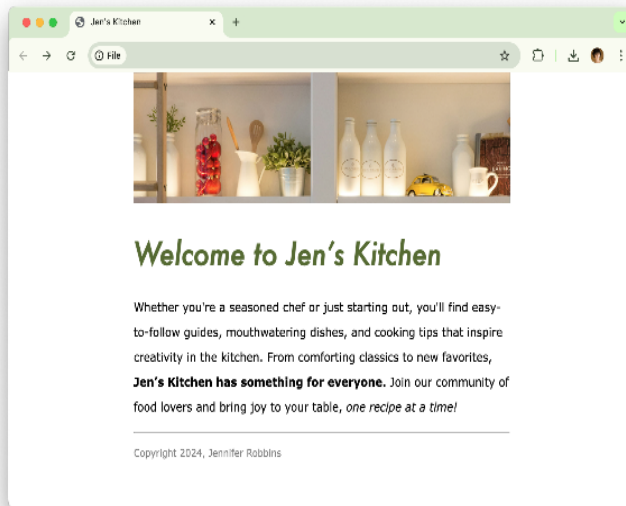
Index of /housepics				
Name	Last modified	Size	Description	
 Parent Directory	18-Mar-2000 21:40	-		
 blank.html	07-Feb-2000 11:23	1k		
 br1.html	07-Feb-2000 11:23	1k		
 br2.html	07-Feb-2000 11:23	1k		
 br3.html	07-Feb-2000 11:22	2k		
 br4.html	07-Feb-2000 11:22	1k		
 br5.html	07-Feb-2000 11:22	1k		
 br6.html	07-Feb-2000 11:22	1k		
 dir1.html	07-Feb-2000 11:22	2k		
 dir2.html	07-Feb-2000 11:22	1k		
 dir3.html	07-Feb-2000 11:22	1k		

Figure 2-2. A server displaying the contents of the directory because the default file is missing

The Anatomy of a Web Page

We're all familiar with what web pages look like in the browser window, but what's happening behind the scenes?

At the top of Figure 2-3, you see a minimal web page as it appears in a graphical browser. Although you see it as one coherent page, it is actually assembled from three separate files: an HTML document (index.html), a style sheet (kitchen.css), and an image (shelf.jpg). The HTML document is running the show.



The web page shown in this browser window consists of three separate files:

- an HTML text document (*index.html*)
- a style sheet (*kitchen.css*)
- one image (*shelf.jpg*)

Tags in the HTML source document give the browsers instructions for how the text is structured and where the image should be placed.

index.html

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Jen's Kitchen</title>
  <link rel="stylesheet" href="kitchen.css">
</head>
<body>



<h1>Welcome to Jen's Kitchen</h1>

<p>Whether you're a seasoned chef or just starting out, you'll find easy-to-follow guides, mouthwatering dishes, and cooking tips that inspire creativity in the kitchen. From comforting classics to new favorites, <strong>Jen's Kitchen has something for everyone.</strong> Join our community of food lovers and bring joy to your table, <em>one recipe at a time!</em></p>

<hr>
<small>Copyright 2024, Jennifer Robbins</small>
</body>
</html>
```

kitchen.css

```
body {
  font: normal 1em/2em Verdana, sans-serif; margin: 0 20%; }

h1 { font: italic bold 2.5rem Futura, sans-serif;
  color: darkolivegreen; }

small { color: gray; }
img { width: 100%; height: auto; }
```

shelf.jpg



Figure 2-3. The source file, style sheet, and images that make up a simple web page (photo by Jason Leung on Unsplash.com)

HTML Documents

You may be surprised to learn that the graphically rich and interactive pages we see on the web are generated by simple, text-only documents. The text file that displays in the browser viewport is referred to as the *source document*.

Take a look at `index.html`, the source document for the Jen’s Kitchen web page in Figure 2-3. You can see that it contains the text content of the page, plus special *tags* (indicated with angle brackets, `<` and `>`) that describe each element on the page. Adding descriptive tags to a text document is known as “marking up” the document. This document has been marked up with tags from the HTML standard, which was created especially for documents with hypertext links.

NOTE

Adding descriptive tags to a text document is known as “marking up” the document.

HTML defines dozens of text elements that make up documents, such as headings, paragraphs, emphasized text, and of course, links. There are also

elements that add information about the document (such as its title), media such as images and videos, and widgets for form inputs, just to name a few.

NOTE

There are other markup languages out there that are used to describe specific types of content, such as the parts of math equations (MathML), vector images (SVG), and more.

You can view the HTML source document for any web page. Exercise 2-1 gives you some prompts and pointers.

There have been several versions of the HTML specification since its birth in 1989. In the past, each specification was locked in and given a version number. Today, the HTML specification is considered a “living standard” and evolves without versioned releases. In your studies, you may run across references to HTML5, which was the last numbered version of HTML maintained by the W3C. It was big shift in the specification, cleaning up obsolete tags and adding new ones that reflected modern development practices and the more interactive nature of websites. “HTML5” became a marketing buzzword meaning “the latest and greatest web technologies.” In this book, when I refer to HTML, I mean the most current, unnumbered standard maintained by the WHATWG. I’ll use the term “HTML5” when it is necessary to refer to that particular standard. You can find a complete history of HTML, all its versions, and an overview of what made HTML5

such a breakthrough in my article “From HTML+ to HTML5,” available at *learningwebdesign.com/articles/*.

EXERCISE 2-1. VIEW SOURCE

You can see the HTML file for any web page by viewing its source in a desktop browser. Most modern browsers keep the View Source function with the developer tools and typically open the source document in a separate window, or in a developer's panel at the bottom of the current window.

WARNING

Keep in mind that while learning from others' work is fine, stealing other people's code is poor form (or even illegal). If you want to use code as you see it, ask for permission, and always give credit to those who did the work.

Here's where to find the View Source function on the major desktop browsers:

- Safari: Develop → Show Page Source
- Chrome: View → Developer → View Source
- MS Edge: Tools → Developer → View Source
- Firefox: Tools → Browser Tools → Page Source

1. Enter this URL into the browser of your choice:

```
www.learningwebdesign.com/6e/kitchen.html
```

You should see the Jen's Kitchen web page from Figure 2-3.

2. Follow the directions for your browser listed above to view the source HTML document for the Jen's Kitchen page. It should be the same as shown in the figure.
 3. To view a page that is a little more complicated, take a look at the source for the learningwebdesign.com home page.
 4. The source for most sites is considerably more complicated than that. View the source of oreilly.com. It's got style sheets, scripts, inline SVG graphics...the works! Don't worry if you don't understand what's going on. Much of it will look more familiar by the time you are done with this book.
-

A Quick Introduction to HTML Markup

You'll be learning the nitty-gritty of markup in Part II, *HTML for Structure*, so I don't want to bog you down with too much detail right now, but there are a few things I'd like to point out about how HTML works and how browsers interpret it.

Read through the HTML document in Figure 2-3 and compare it to the browser results. It's easy to see how the elements marked up with HTML tags in the source document correspond to what displays in the browser window.

First, you'll notice that the text characters within brackets (for example, `<body>` and `<strong>`) do not display in the final page. The browser displays only what's *between* the tags—the *content* of the element. The markup is hidden. The tag provides the name of the HTML element—usually an abbreviation, such as “h1” for “heading level 1,” or “em” for “emphasized text.”

Second, you'll see that most of the HTML tags appear in pairs surrounding the content of the element. `<h1>` indicates that the following text should be a first-level heading; `</h1>` indicates the end of the heading. Some elements, called *void elements* (or *empty elements* in older HTML specs), do not have content. In the Jen's Kitchen source, the `<hr>` tag indicates a void element that tells the browser to “insert a horizontal rule here” as a thematic divider.

PARSING HTML: “BEADS ON A STRING”

Because I was unfamiliar with computer programming when I first began writing HTML in 1993, it helped me to think of the tags and text as “beads on a string” that the browser interprets one by one, in sequence. For example, when the browser encounters an open bracket (`<`), it assumes all of the following characters are part of the tag until it finds the closing bracket (`>`). Similarly, it assumes all of the content following an opening `<h1>` tag is a heading until it encounters the closing `</h1>` tag. This is the manner in which the browser *parses* the HTML document.

Understanding the browser’s method can be helpful when troubleshooting a misbehaving HTML document.

Where Are the Pictures?

Obviously, there are no pictures in the HTML file itself, so how do they get there when you view the final page? You can see in Figure 2-3 that the image is a separate file. Images are placed in the flow of the text with the HTML image element (`img`), which tells the browser the location of the image (its URL). When the browser sees the `img` element, it makes another request to the server for the image file, then places it in the content flow.

The browser also sends requests to the server for style sheets (like `kitchen.css`), JavaScript files (`.js`), and other embedded media like audio and

videos. The browser software (or more specifically, its rendering engine) brings the separate pieces together into the final page.

The assembly of the page generally happens in an instant, so it appears as though the whole page loads all at once. Over slow connections, or if the page includes huge images or media files, the assembly process may be more apparent as images lag behind the text. The page may even need to be redrawn as new images, fonts, and style sheets arrive, although you can write your HTML in a way that prevents this from happening.

Adding a Little Style

Direct your attention to the last key ingredient of our minimal page. Near the top of the HTML document there is a `link` element that points to the style sheet document `kitchen.css`. That style sheet includes a few lines of instructions for how the page should look in the browser. These are style instructions written according to the rules of *Cascading Style Sheets (CSS)*. CSS allows designers to add visual style instructions (known as the document's *presentation*) to the marked-up text (the document's *structure*, in web design terminology). In Part III, *CSS for Presentation*, you'll get to know the power of Cascading Style Sheets.

Figure 2-4 shows the Jen's Kitchen page without (left) and with (right) the style instructions. Browsers come equipped with default styles for every HTML element they support, so if an HTML document lacks custom style

instructions, the browser will use its own. For example, the `h1` heading is bigger and bolder than the paragraph text by default. That's what you see in the screenshot on the left. Even just a few style rules can make big improvements to the appearance of a page.



Figure 2-4. The Jen's Kitchen page without (left) and with (right) custom style rules.

Adding Behaviors with JavaScript

To make elements on the page *do* something, you use a scripting language called *JavaScript* (see Note). There are no scripts on the Jen's Kitchen page because I thought it best to keep things simple this early in the book, but JavaScript is an essential ingredient in modern websites.

Whereas HTML provides the structure and the CSS style sheet alters how things look, JavaScript adds a behavior component that controls how things *work*. Scripts can be standalone files on the server (with the .js suffix) or be written out right in the document. They may be triggered to run immediately when the page loads or be triggered by something the user does, like clicking or hovering on an element or entering something in a form field.

You'll get a basic introduction to JavaScript in [Part IV](#), JavaScript for Behavior.

NOTE

JavaScript is not required for the interactivity of links and web forms, which work using HTML alone.

STATIC VERSUS DYNAMIC SITES

Static websites consist of HTML files with fixed content that display the same information to every visitor. In other words, each page you see in the browser is a view of a single HTML file on the server. This book focuses on the creation of static web pages, as they are straightforward and the best starting place for beginners.

By contrast, dynamic websites are generated with backend programming such as PHP, Node.js, Ruby, or ASP.NET. Each page is generated by the application on the fly by pouring content and data from a database into a page template. The final pages may be customized for each user. For extremely large sites with hundreds or thousands of pages, setting up and maintaining a dynamic site is considerably less work than creating and storing every page as a static HTML document individually.

Putting It All Together

To wrap up this introduction to how the web works, let's trace the typical stream of events that occurs with every web page that appears on your screen (Figure 2-5):

1. You request a web page, by either typing its URL (for example, `https://learningwebdesign.com/6e/kitchen/`) directly

in your browser or clicking a link on a page. The URL contains the information needed to target a specific document on a specific web server on the internet.

2. The browser sends an HTTP request to the server named in the URL and asks for the page. The request also includes information about what languages the user can read and what types of files the browser can accept.
3. The server looks for the requested file and issues an HTTP response in the form of an *HTTP header*. The header includes information about the file, like the last modified date, the length of the file, and its Content-Type (for example, an .html file has the Content-Type “text/html”). If the URL specifies a directory (not a file), the server looks for the default file in that directory—in this case *index.html* in the *kitchen* directory.

If the page cannot be found, the server returns an error message. The message typically says “404 Not Found,” although more hospitable error messages may be provided. Other error types are possible as well (see the sidebar “HTTP Status Codes”).

If the document *is* found, the server retrieves the requested file and returns it to the browser. If the site is dynamic, the server assembles the page from stored data before returning it to the browser.

4. The browser parses the HTML document. If the page contains images (indicated by the HTML `img` element) or other external resources, like

scripts or style sheets, the browser contacts the server again to request each resource specified in the markup.

5. The browser inserts each image in the document flow where indicated by the `img` element, applies styles, and runs scripts. And *voilà!* The assembled web page is displayed for your viewing pleasure.

HTTP STATUS CODES

Servers issue status codes in response to browser requests. The full list of status codes is quite long (you can read about them all at en.wikipedia.org/wiki/List_of_HTTP_status_codes), but here are a few common responses:

200 OK

301 Moved Permanently

302 Moved Temporarily

404 Not Found

500 Internal Server Error

I should note that I've depicted a traditional and simplified scenario here to tell you how web pages are put together. These days, it is common for web pages to be generated by *content management systems (CMSs)* that keep

content in databases and use templates to assemble the data into pages on the fly. In that case, in step 3b, there is a more complicated process of assembling the file from various parts rather than just handing off an existing file.

Browser (Client)

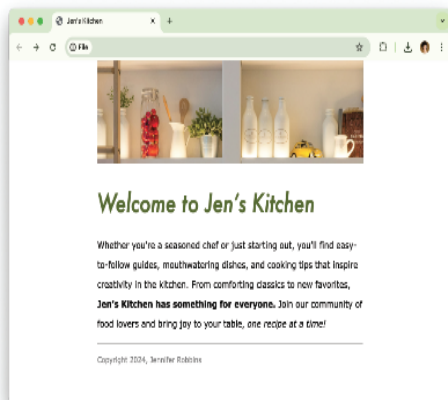
1. Type in a URL or click a link in the browser.

`http://jenskitchensite.com`

2. The browser sends an HTTP request.



4. The browser parses the document. If it has images, style sheets, and scripts, the browser contacts the server again for each resource.



5. The page is assembled in the browser window.

Server

Files on Server



3. The server looks for or assembles the file and responds with an HTTP response.

"I see that you requested a directory, so I'm sending you the default file, index.html. Here you go."

Oops, no file

If the file is not on the server, it returns an error message.

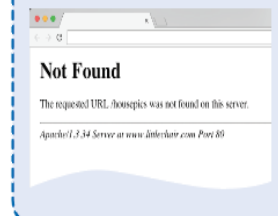


Figure 2-5. How browsers display web pages

GETTING YOUR PAGES ON THE WEB

If you would like more information about registering domain names and finding a server to host your site, see my article “Getting Your Pages on the Web” at learningwebdesign.com/articles/.

Chapter 3. Some Big Concepts You Need to Know

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 3rd chapter of the final book.

If you’d like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at jbleiel@oreilly.com.

In this chapter:

- The web on mobile devices
- The benefits of web standards
- Progressive enhancement
- Responsive web design
- Accessibility
- Site performance

Before we get to the nuts and bolts of HTML, CSS, and JavaScript, I want to introduce some “Big Concepts” that every web designer needs to know. I’ll be referring to the terminology introduced in this chapter frequently, so it’s not to be missed.

The heart of the matter is that as web designers, we never know exactly how the pages we create will be viewed. We don’t know which of the dozens of browsers might be used, whether that browser is on a desktop computer or something more portable, how large the viewport will be, what fonts are installed, whether functionality such as JavaScript is enabled, how fast the internet connection is, whether the pages are being read by a screen reader, and so on. The Big Concepts in this chapter are primarily reactions to—and methods for—coping with the inescapable element of *the unknown* in our medium. They include the following:

- The multitude of devices
- Web standards
- Progressive enhancement
- Responsive web design
- Accessibility
- Site performance

Because we’re just getting started, I’ll keep the descriptions brief and fairly non-technical. The goal is that you have a basic understanding of what I mean by terms like “progressive enhancement” when you encounter them

in lessons later. Many excellent articles and books have been written on each of these topics and their related production techniques, and I'll provide pointers to resources for further reading.

A Multitude of Devices

Until 2007, web designers could be relatively certain that our users were visiting our sites while sitting at their desks, looking at a large monitor, using a speedy internet connection. We had all more or less settled on 960 pixels as a good width for a web page, based on the most common monitor size. Back then, our biggest concern was dealing with the dozen or so desktop browsers and jumping through a few extra hoops to support quirky old versions of Internet Explorer (it was no picnic!).

Although people could access web pages and web content on mobile phones prior to 2007, the introduction of the iPhone and Android smartphones, as well as faster networks, heralded a huge shift in how, when, and where we do our web surfing. Since then, we've seen the introduction of phones and tablets of all different dimensions, as well as web browsers on TVs, gaming systems, and other devices. And the diversity is only going to increase. I think mobile web design expert Brad Frost (bradfrost.com) sums it up nicely in his illustrations in Figure 3-1.



Figure 3-1. Brad Frost sums up the reality of device diversity nicely.

The challenge of designing for all of these devices goes beyond addressing differing screen sizes. There is a world of difference between using a site over a broadband connection and over a slow cell network. Designers need to resist making assumptions about network speed and context based on the screen size. Just because it's a small screen doesn't mean it's a slow connection or that the person is in a hurry—it's not uncommon to leisurely browse the web on a smartphone while sitting on the couch at home with a solid WiFi connection. Conversely, iPads with larger, high-resolution displays may be accessing the internet on slower cell connections. In other words, it's complicated!

NOTE

Resist making assumptions about network speed and context based on the screen size.

For a lot of sites today, more people are visiting via their mobile devices than on a desktop computer. Already, a significant portion of Americans use their mobile phones as their *only* means of access to the internet. That means it's critical to get the design and functionality right. We've made huge strides in serving a pleasing experience to users with handheld devices, and the technology for targeting their needs continues to head in the right direction.

What I want you to learn here is that the way you see your design as you're working on it on your nice desktop machine is not how it will be experienced by everyone. Some will see it much smaller. Some will see it load painfully slowly. Some may be looking at it on a TV across the room. Some may be listening to it with a screen reader. All web design professionals should keep this fact in mind.

Further Reading

- Mobile First by Luke Wroblewski (A Book Apart). Luke was way ahead of the curve in insisting that sites work well on mobile devices, and he shares his perspective in this little book, which is jam-packed with ideas.

Sticking with the Standards

So how do we deal with this diversity? A good start is to follow the standards documented by the World Wide Web Consortium. Sticking with web standards is your primary tool for ensuring your site is consistent on all standards-compliant browsers (that's virtually all browsers in current use). It also helps make your content forward-compatible as web technologies and browser capabilities evolve.

NOTE

Sticking with web standards is your primary tool for ensuring your site is as consistent as possible.

The notion of standards compliance may seem like a no-brainer, but it used to be that everyone, including the browser makers, played fast and loose with HTML and scripting. The price we paid was incompatible browser implementations and the need to create multiple versions of sites to make them work for everyone. I'll talk more about web standards throughout this book, so I won't go into too much detail here. Suffice it to say that the web standards are your friends.

Further Reading

- The W3C site (w3.org/standards) is the primary resource for all web standards documents.
- Designing with Web Standards, 3rd edition, by Jeffrey Zeldman (New Riders) is the bible for standards compliance and how it makes good business sense. It's getting on in years, but the fundamentals are still solid.

Progressive Enhancement

NOTE

Progressive enhancement is a strategy for coping with unknown browser capabilities.

With a multitude of browsers comes a multitude of levels of support for the web standards. In fact, no browser has fully implemented all the standards, and there are always emerging technologies steadily gaining steam.

Furthermore, users can set their own browser preferences, so, for example, they may have a browser that supports JavaScript but have chosen to turn it off. The point here is that we are faced with a wide range of browser capabilities, from basic HTML support to all the bells and whistles.

Progressive enhancement is one strategy for dealing with unknown browser capabilities (see Note). The first rule of progressive enhancement is to start with a baseline experience that makes the content or core functionality

available to even the most rudimentary browsers or assistive devices. From there, you layer on more advanced features for the browsers that can handle them. You might finish with some “nice to have” features, like using drop caps (enlarging the first letter of text and letting the text lines wrap around it) or a cutting-edge image format like AVIF, that enhance the experience for users with the most advanced browsers.

NOTE

The shift to using JavaScript frameworks that render page components on the server has taken some of the responsibility for standards support off of browsers. For that reason, in the current professional web development landscape, progressive enhancement is less in the forefront of developers’ minds than it was a decade or so ago. However, if you are coding a site traditionally, with HTML and CSS (and not a framework), accommodating older browsers with progressive enhancement is still a useful approach. This is why I have chosen to include it as one of the key principles.

Progressive enhancement is an approach that can be applied to all components of page design and production, including authoring in HTML, styling content with CSS, and adding scripted behaviors with JavaScript:

Authoring strategy

When an HTML document is written in a logical order and its elements are marked up in a meaningful way, it will be usable on the widest range of browsing environments, including the oldest browsers, future browsers, and mobile and assistive devices. It may not look exactly the same, but the important thing is that your content is available. This

approach also ensures that search engines like Google will catalog the content correctly. A clean HTML document with its elements accurately and thoroughly described is the foundation of accessibility.

Styling strategy

You can create layers of experience by taking advantage of the way browsers parse style sheet rules. The knowledge that browsers simply ignore properties and rules they don't understand gives you license to innovate without breaking things on older browsers. You just have to be mindful of styling the baseline experience first, then adding improvements once the minimum requirements are met.

Scripting strategy

Although it's not as common as it used to be, users can turn off JavaScript in their browsers. There is also a small chance that an external .js file won't load. Use progressive enhancement to make sure basic functionality—such as linking from page to page or accomplishing essential tasks like data submission via forms—is intact even when JavaScript is off. In this way, you ensure the integrity of the baseline experience, and enhance it when JavaScript is available.

Further Reading

- Adaptive Web Design: Crafting Rich Experiences with Progressive Enhancement, 2nd edition, by Aaron Gustafson (New Riders). There's

no better introduction to the progressive enhancement approach than this book. Aaron is also the author of the chapters in Part IV, JavaScript for Behavior.

- The Uncertain Web: Web Development in a Changing Landscape by Rob Larson (O'Reilly).
- Resilient Web Design by Jeremy Keith (online, self-published).

Responsive Web Design

By default, most browsers on small devices such as smartphones and tablets shrink a web page down to fit the screen and provide mechanisms for zooming and moving around the page. Although this technically works, it's not a great experience. The text is too small to read, the links are too small to tap, and all that zooming and panning around is distracting.

Responsive web design (RWD) is a strategy for providing appropriate layouts to devices based on the size of the viewport (the browser window on a desktop computer, or a full-screen browser application on a mobile device). The key to RWD is serving a single HTML document with one URL to all devices, but applying different styles based on the viewport size in order to provide the most optimized layout for each device. For example, when the page is viewed on a smartphone, it appears in one column with large links for easy tapping. But when that same page is viewed on a large desktop browser, the content rearranges into multiple columns with

traditional navigation elements. It's like *magic*! (Except that it's actually just CSS.)

The web design community has been abuzz about responsive design since Ethan Marcotte first wrote about it and coined the phrase in his article “Responsive Web Design” on A List Apart in 2010 (alistapart.com/articles/responsive-web-design/).

Figure 3-2 shows some examples of responsive sites at the typical dimensions for a smartphone, tablet, and desktop monitor. Try opening one of the responsive sites in your desktop browser and then resizing the window, making it narrower and wider. Watch as the layout changes based on the window size. *Très* cool.

Brad Frost personal site bradfrost.com



NASA nasa.gov

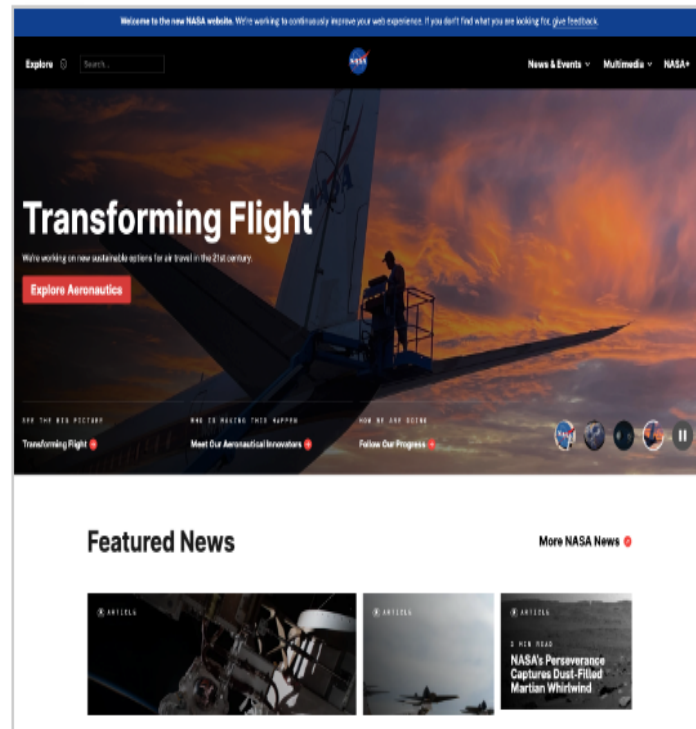
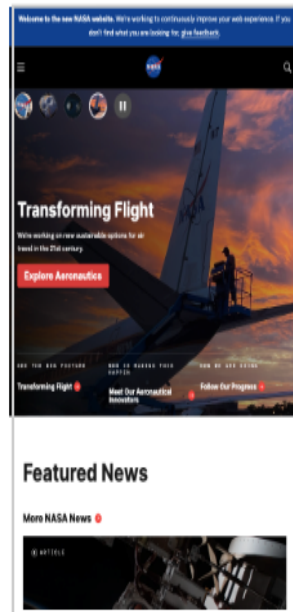
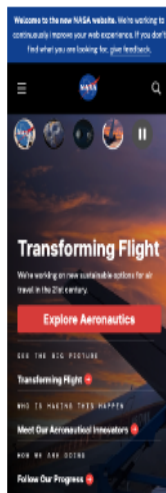


Figure 3-2. A responsive site's layout changes based on the size of the browser window.

Responsive web design helps with matters of layout at different viewport sizes, but it's not a solution to all mobile web design challenges. The fact is that providing the best experiences for your users and their chosen devices may require optimizations that go beyond adjusting the look and feel. You can better address some problems by having the server detect the device and its capabilities and send back appropriately formatted content.

For some sites and services, it may be preferable to build a separate mobile site (see the “M-Dot Sites” sidebar) with a customized interface and feature set that takes advantage of phone capabilities like geolocation. That said, although responsive design won't fix everything, it has become the standard method for building a website and delivering satisfactory experiences on a wide variety of browsers.

NOTE

Responsive web design is a strategy for dealing with unknown screen size.

M-DOT SITES

Some companies and services choose to build an entirely separate site, with a unique URL, just for mobile devices. M-dot sites (so called because their URLs typically begin with “m.” or “mobile.”) usually offer a reduced set of options and may also include mobile-specific features such as geolocation. A dedicated mobile site may be the best solution if you know that your mobile users have very different usage patterns than folks seated at a desk. Facebook, for example, delivers a streamlined interface to mobile browsers with `m.facebook.com`.

The downside of creating a dedicated mobile site is that it’s more than twice the work. It requires additional content planning, design templates, production time, and ongoing maintenance. In general, m-dot sites have faded out of use in favor of RWD. Google has helped to speed this process along by switching to “mobile-first” indexing (developers.google.com/search/docs/crawling-indexing/mobile/mobile-sites-mobile-first-indexing), basing search results on the contents of a site as it appears on a mobile device. If search result rankings are a concern, you will get more mileage from going responsive.

Further Reading

- I’ll cover responsive web design in more detail in Chapter 19, Responsive Web Design, once you have more code experience under

your belt. You'll find plenty of resources there to continue your responsive design education.

Accessibility

I've talked a bit about the daunting number of browsers in use today, but so far, I've only addressed visual browsers controlled with mouse pointers or fingertips. It is critical, however, to keep in mind that people access the web in many different ways—with a keyboard, mouse, voice commands, screen readers, Braille output, magnifiers, joysticks, foot pedals, and so on. Web designers must build pages in a manner that creates as few barriers as possible to getting to information, regardless of the user's ability and the device used to access the web. In other words, you must design for *accessibility*.

NOTE

A11y is a common abbreviation for Accessibility that provides the first (A) and last letter (y) and counts the 11 letters in between.

Although intended for users with disabilities such as poor vision or limited mobility, the techniques and strategies developed for accessibility also benefit other users with less-than-optimum browsing experiences.

Accessible sites are also more effectively indexed by search engines such as Google. Making your site accessible is well worth the extra effort.

There are four broad categories of disabilities that affect how people interact with their computers and the information on them:

Vision impairment

People with low or no vision may use an assistive device such as a screen reader (see Note), Braille display, or screen magnifier to get content from the screen. Low-vision site visitors may also simply use the browser's text zoom function to make the text large enough to read.

NOTE

It's not a bad idea to get your own screen reader to get familiar with how your site will be experienced when read aloud. Windows users can download the NVDA screen reader (nvaccess.org/download/) for free. Mac users can use VoiceOver, the screen reader that is built into the operating system.

Mobility impairment

Users with limited or no use of their hands may use special devices such as modified mice and keyboards, foot pedals, voice commands, or joysticks to navigate the web and enter information.

Auditory impairment

Users with limited or no hearing will miss out on audio aspects of multimedia, so it is necessary to provide alternatives, such as transcripts for audio tracks or captions for video.

Cognitive impairment

Users with memory, reading comprehension, problem solving, and attention limitations benefit when sites are designed simply and clearly. These qualities are helpful to anyone using your site.

The W3C started the *Web Accessibility Initiative (WAI)* to address the need to make the web usable for everyone. The WAI site (w3.org/WAI) is an excellent starting point for learning more about web accessibility. One of the documents produced by the WAI to help developers create accessible sites is the *Web Content Accessibility Guidelines (WCAG)*. You can read about the different versions at w3.org/WAI/standards-guidelines/wcag/. The US government based its Section 508 accessibility guidelines on the Priority 1 points of WCAG (see the sidebar “Government Accessibility Requirements: Section 508”). All sites benefit from these guidelines, but if you are designing a government site, adherence is a requirement.

Another W3C effort is the *WAI-ARIA (Accessible Rich Internet Applications)* spec, which addresses the accessibility of web applications that include dynamically generated content, scripting, and advanced interface elements that are particularly confounding to assistive devices. The ARIA recommendation defines a number of roles for content and

widgets that web developers can explicitly apply using the `role` attribute, including include `menubar`, `progressbar`, `slider`, `timer`, and `tooltip`, to name just a few. For the complete list of roles, go to w3.org/TR/wai-aria/#role_definitions.

Further Reading

The following resources are good starting points for further exploration of web accessibility:

- The Web Accessibility Initiative (WAI), w3.org/WAI/
- WebAIM: Web Accessibility in Mind, webaim.org
- Accessible Web, accessibleweb.com (this company provides accessibility audits and scanning software; there is a fee for their services, but they offer a free trial version)
- Accessibility Handbook: Making 508 Compliant Websites by Katie Cunningham (O'Reilly)
- Universal Design for Web Applications: Web Applications That Reach Everyone by Wendy Chisholm and Matt May (O'Reilly)

US GOVERNMENT ACCESSIBILITY REQUIREMENTS: SECTION 508

If you create a site that receives federal funding from the US government, you are required by law to comply with the Section 508 guidelines, which ensure that electronic information and technology are available to people with disabilities. State and other publicly funded sites may also be required to comply.

The following guidelines, excerpted from the Section 508 Standards at access-board.gov/ict/, provide a good checklist for basic accessibility for all websites:

1. A text equivalent for non-text elements shall be provided (e.g., via the “alt” attribute or in element content).
2. Equivalent alternatives for any multimedia presentation shall be synchronized with the presentation.
3. Web pages shall be designed so that all information conveyed with color is also available without color—for example, from context or markup.
4. Documents shall be organized so they are readable without requiring an associated style sheet.
5. Row and column headers shall be identified for data tables.
6. Markup shall be used to associate data and header cells for tables with two or more levels of row or column headers.
7. Pages shall be designed to avoid causing the screen to flicker with a frequency greater than 2 Hz and lower than 55 Hz.

8. When pages utilize scripting languages to display content, or to create interface elements, the information provided by the script shall be identified with functional text that can be read by assistive technology.
 9. When a web page requires that an applet, plug-in, or other application be present on the client system to interpret page content, the page must provide a link to a plug-in or applet that complies with §1194.21(a) through (l).
 10. When electronic forms are designed to be completed online, the form shall allow people using assistive technology to access the information, field elements, and functionality required for completion and submission of the form, including all directions and cues.
 11. A method shall be provided that permits users to skip repetitive navigation links.
 12. When a timed response is required, the user shall be alerted and given sufficient time to indicate more time is required.
-

INCLUSIVE DESIGN

Accessibility generally addresses physical limitations—compromised sight, hearing, mobility, etc.—that might prevent users from accomplishing their goals. It is one part of a broader design methodology, inclusive design.

Inclusive design takes into account all the ways that people are diverse so that nobody feels unwelcome or left out. It may address age, race, economic situation, neurodiversity, location, language, culture, accessibility, and more.

It is natural to subconsciously assume your users are like you or do things the way you do. The key to inclusive design is making sure that you include diverse perspectives as part of the design and problem-solving process. Or, as practitioners of inclusive design put it: “Design with, not for.”

Even small adjustments can make a site feel more inclusive. For example, a person of mixed race may feel excluded while filling out a form if an input forces them to choose a single race. A “mixed race” option or the ability to choose multiple options would offer less friction and make them feel more welcome.

When people feel like your site or product is not quite for them, it affects the bottom line, which is why inclusive design processes have become a priority in web design. It’s good business. You can learn more with these resources:

- Mismatch: How Inclusion Shapes Design by Kat Holmes (The MIT Press)
 - Inclusive Design Communities by Sameera Kapila (samkapila.com/books/)
 - Microsoft's Inclusive Design resources, including articles and guidebooks (inclusive.microsoft.design)
 - "Inclusive Design," an overview article by Alita Joyce of Nielsen Norman Group (nngroup.com/articles/inclusive-design)
 - The Neurodiversity Design System (neurodiversity.design), design patterns for accommodating a range of neurodiversity
-

The Need for Speed (Site Performance)

While dial-up connections are mostly a thing of the past, networks conditions still vary dramatically by connection type, location, and even time of day (for example, large urban areas can see significant swings in data speeds during peak hours). Devices, too, vary dramatically in how much computational power and memory they have available. Regardless of who you are or where you live, we've all felt the frustration of a slow-loading site at some point or another.

Site performance factors into search engine rankings, and it has repeatedly been shown to have a noticeable impact on how people use sites. In 2006,

Amazon.com showed that reducing page load times by just 100 ms resulted in a 1% increase in revenue (interestingly, Walmart found the exact same impact in its own study).¹ A study by Google in 2009² showed that the addition of just 100 to 400 ms to the load time of their search results page resulted in reduced searches (−0.2 to −0.6%). Other studies have shown that users expect a site to load in under 2 seconds, and nearly a third of your audience will leave your site for another if it doesn't. Furthermore, those people aren't likely to come back. Google has added site speed to its search algorithm, so if your site is a slowpoke, it's not likely to show up in that coveted first screen of results. The takeaway here is that site performance (down to the millisecond!) matters a lot.

TIP

You can find an ever-growing collection of case studies showing the business impact of performance at wpostats.com, curated by Tim and Tammy Everts.

There are many things you can do to improve the performance of your site. Quite a few of them fall beyond the scope of this book, but a solid starting point is to focus on limiting file sizes and reducing the number of requests to the server. The following list only scratches the surface for site optimization, but it gives you a general idea of what can be done:

- Optimize images so they are the smallest file size possible without sacrificing quality.

- Streamline HTML markup, avoiding unnecessary levels of nested elements.
- Minimize the size of HTML and CSS documents by removing extra character spaces and line returns.
- Keep JavaScript to a minimum.
- Add scripts in such a way that they load in parallel with other page assets and don't block rendering.
- Don't load unnecessary assets (such as images, scripts, or JavaScript libraries).
- Reduce the number of times the browser makes requests of the server (known as *HTTP requests*).

Every trip to the server in the form of an HTTP request takes a few milliseconds, and those milliseconds can add up. All those little Instagram widgets, Facebook Like buttons, and advertisements can make dozens of server requests each.

You may be surprised to see how many server requests even a simple site makes. If you'd like to see for yourself, you can use the Network tool available with the Developer tools in Chrome, Safari, or Firefox. The Network tool displays each request to the server and how many milliseconds it took. It works similarly in all the browsers, but here's how you use it in Chrome:

1. Launch the browser and go to any web page.

2. Go to the View menu and select Developer → Developer Tools. A panel will open at the bottom of the browser. If you don't see it, it may be on the right side, but you can change its position with the “Dock Side” setting under the options icon (three stacked dots).
3. Select the Network tab in the tools view. The chart (commonly referred to as a *waterfall chart*) shows you all the requests made and assets downloaded. The columns on the right show the amount of time each request took in milliseconds. At the bottom of the chart, you can see a summary of the number of requests made and the total amount of data transferred.

Figure 3-3 shows the performance waterfall chart for learningwebdesign.com. You can poke around any site on the web this way. It can be very educational.

I won't address site performance in deep technical detail in this book, but I do want you to remember the importance of keeping file sizes as small as possible and eliminating unnecessary server requests in your web design work.

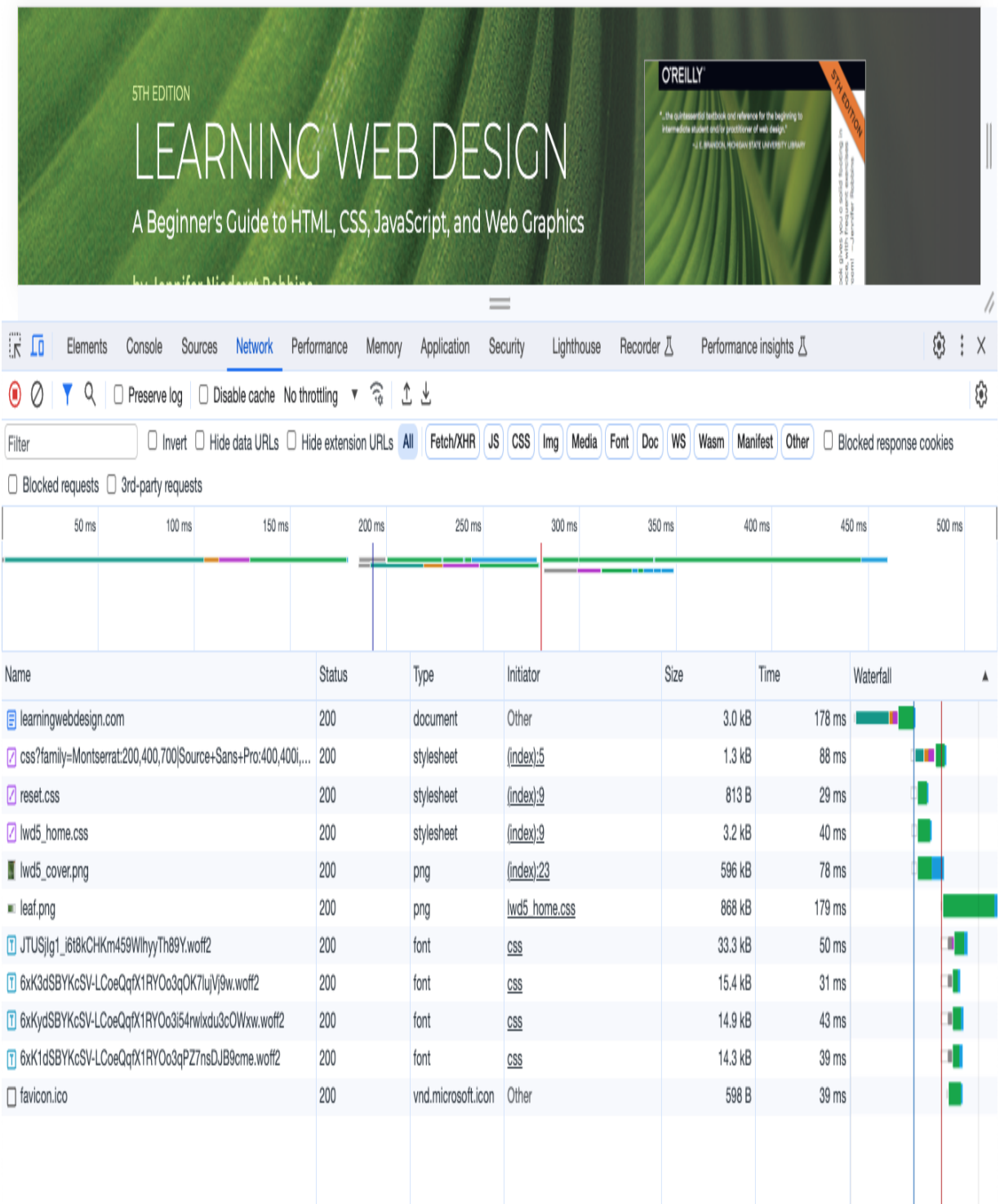


Figure 3-3. Waterfall charts such as this one created by the Chrome Network developer tool show the individual server requests made by a web page and the amount of time each request takes.

Further Reading

There are other techniques that are too technical for this book, and I figure if you're reading this book, you're probably not quite ready to become a site performance wizard. But when you are ready to take on this challenge, here are some resources that should help:

- Lara Hogan has assembled a list of performance-related studies, tools, and resources at larahogan.me/design. You can also read her book *Designing for Performance* (O'Reilly) there for free.
- *High Performance Mobile Web: Best Practices for Optimizing Mobile Web Apps* by Maximiliano Firtman (O'Reilly) covers optimization methods and tools to check your progress.
- *Web Performance in Action: Building Fast Web Pages* by Jeremy Wagner (Manning) is a fantastic introduction to performance optimization techniques.

TESTING PERFORMANCE USING LIGHTHOUSE

Google's Lighthouse is a great starting point for testing your site's performance and seeing what sort of improvements can be made. You can run Lighthouse as a standalone tool, from within Chrome's developer tools, or through Google's PageSpeed Insights (pagespeed.web.dev).

1 Statistic from “Make Data Matter,” PowerPoint presentation by Greg Linden of Stanford University (2006).

2 “Speed Matters,” googleresearch.blogspot.com/2009/06/speed-matters.html.

Chapter 4. Creating a Simple Page:

HTML Overview

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 4th chapter of the final book.

If you’d like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at jbleiel@oreilly.com.

In this chapter:

- An introduction to elements and attributes
- Marking up a simple web page
- The elements that provide document structure
- Troubleshooting broken web pages

Part I, Getting Started, provided a general overview of the web design environment. Now that we’ve covered the Big Concepts, it’s time to roll up our sleeves and start creating a real web page. It will be an extremely

simple page, but even the most complicated pages are based on the principles described here.

In this chapter, we'll create a web page step by step so you can get a feel for what it's like to mark up a document with HTML tags and apply styles. The exercises allow you to work along.

In this chapter you will:

- Get a feel for how markup works, including an understanding of elements and attributes.
- See how browsers interpret HTML documents.
- Learn how HTML documents are structured.
- Get a first glimpse of a style sheet in action.

Don't worry about learning the specific text elements or style sheet rules at this point; we'll get to those in the following chapters. For now, just pay attention to the process, the overall structure of the document, and the new terminology.

A Web Page, Step by Step

You got a look at an HTML document in Chapter 2, How the Web Works, but now you'll get to create one yourself and play around with it in the browser. You'll start with a page of raw text for the fictional Black Goose

Bistro site and turn it into a styled web page in five steps that cover the basics of page production:

Step 1: Start with content

As a starting point, we'll write up some raw text content and see what browsers do with it.

Step 2: Give the document structure.

You'll learn about HTML element syntax and the elements that set up areas for metadata and page content.

Step 3: Identify text elements

You'll describe the content using the appropriate text elements and learn about the proper way to use HTML.

Step 4: Add an image

By adding an image to the page, you'll learn about attributes and void elements.

Step 5: Change how the text looks with a style sheet

This exercise gives you a taste of formatting content with Cascading Style Sheets.

By the time we're finished, you'll have written the document for the page shown in Figure 4-1. It's not very fancy, but you have to start somewhere!



BLACK GOOSE BISTRO

The Restaurant

The Black Goose Bistro offers casual lunch and dinner fare in a relaxed atmosphere. The menu changes regularly to highlight the freshest local ingredients.

Catering

You have fun. *We'll handle the cooking.* Black Goose Catering can handle events from snacks for a meetup to elegant corporate fundraisers.

Location and Hours

Seekonk, Massachusetts;

Monday through Thursday 11am to 9pm;

Friday and Saturday, 11am to midnight

Figure 4-1. In this chapter, we'll write the HTML document for this page in five steps.

We'll be checking our work in a browser frequently throughout this demonstration—probably more than you would in real life, but because this is an introduction to HTML, it's helpful to see the cause and effect of each small change to the source file along the way. Now would be a good time to get set up with a text or code editor, so you can follow along with the exercises.

Launch a Text Editor

In this chapter and throughout the book, we'll be writing out HTML documents by hand, so the first thing you'll need to do is launch a text or code editor.

The text editor that is provided with your operating system, such as Notepad (Windows) or TextEdit (Mac), will do for these purposes. Other text editors are fine as long as you can save plain-text files with the .html extension. The advantages to using a plain text editor are that you have it already, you don't need to download or sign up for anything, and, more importantly, it won't do anything to your code (like automatically closing element tags for you, for example), so you'll have to be more mindful when writing.

If you would like to use a code editor, like Visual Studio Code, Phoenix Code, or the others I talked about in Chapter 1, Teams and Tools, I'm fine with that, too. It will give you a more realistic web developer experience. At this point, though, I would recommend that you avoid any sort of visual drag-and-drop editor, like Dreamweaver, Wix, or similar. I want you to get a feel for marking up a document manually (see the sidebar "HTML the Hard Way").

This section shows how to create HTML documents in Notepad and TextEdit. Even if you've used these programs before, skim through for some special settings that will make the exercises go more smoothly. I'll start with Notepad; Mac users can jump ahead. You can also skip ahead if you would prefer to use a dedicated code editor.

HTML THE HARD WAY

I stand by the method of teaching HTML the old-fashioned way—*by hand*. There’s no better way to truly understand how markup works than typing it out, one tag at a time, and then opening your page in a browser. It doesn’t take long to develop a feel for marking up documents properly.

Although you may choose to use a visual or drag-and-drop web authoring tool or service down the line, understanding HTML will make using that tool easier and more efficient. In addition, you’ll be glad that you can look at a source file and understand what you’re seeing. This is also crucial for troubleshooting broken pages or fine-tuning the default formatting that some web tools produce.

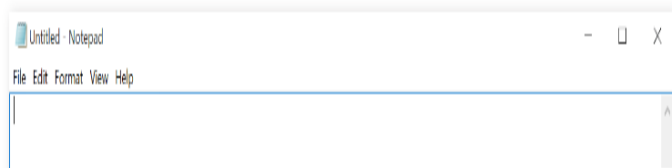
And, for what it’s worth, professional web developers tend to mark up content manually for better control over the code and the ability to make deliberate decisions about what elements to use.

Creating a New Document in Notepad (Windows)

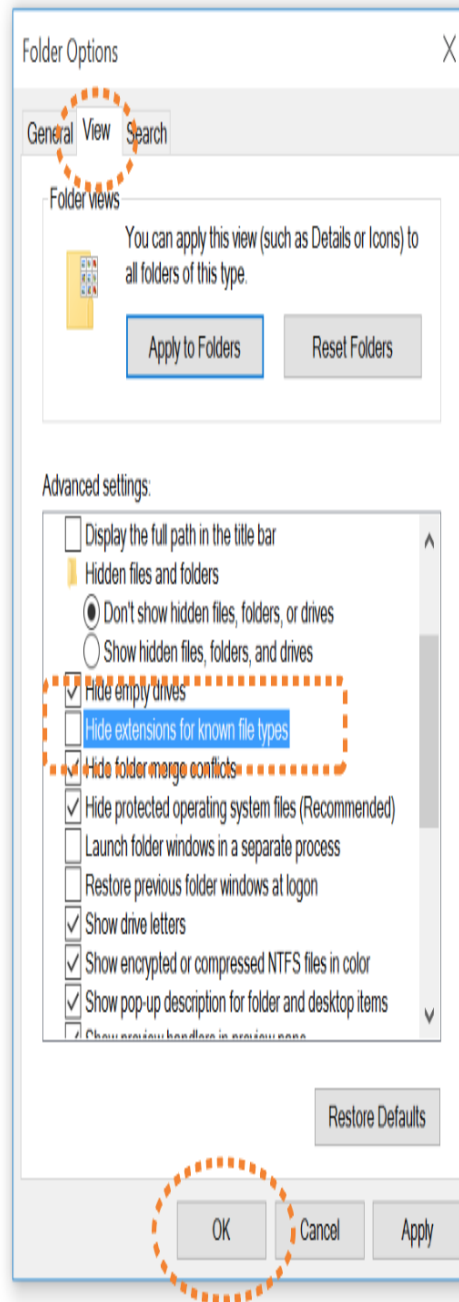
These are the steps to creating a new document in Notepad on Windows 11 (Figure 4-2):

1. Search for “Notepad” to access it quickly. Click on Notepad to open a new document window, and you’re ready to start typing [1].

2. Next, make the extensions visible. This step is not required to make HTML documents, but it will help make the file types clearer at a glance. Open the File Explorer, select the See More button, and then select Options. In the Folder Options panel, select the View tab [2].
3. Find “Hide extensions for known file types” and uncheck that option [3].
4. Click OK to save the preference [4], and the file extensions will now be visible.



1 Click on Notepad to open a new document.



2 Open the File Explorer, select the View tab, and then select the Options button on the right (not shown). Select the View tab.

3 Uncheck “Hide extensions for known file types.”

4 Click OK to save the preference, and the file extensions will now be visible.

Figure 4-2. Creating a new document in Notepad

Creating a New Document in TextEdit (macOS)

By default, TextEdit creates *rich-text* documents—that is, documents that have hidden style-formatting instructions for making text bold, setting font size, and so on. You can tell that TextEdit is in rich-text mode when it has a formatting toolbar at the top of the window (plain-text mode does not).

HTML documents need to be plain-text documents, so we'll need to change the format, as shown in this example (Figure 4-3):

1. Use the Finder to look in the Applications folder for TextEdit. When you've found it, double-click the name or icon to launch the application.
2. In the initial TextEdit dialog box, click the New Document button in the bottom-left corner. If you see the text formatting menu at the top of the Untitled document, you are in rich-text mode [1]. If you don't, you are in plain-text mode [2]. Either way, there are some preferences you need to set.
3. Close that document, and open the Preferences dialog box from the TextEdit menu.

4. Change these preferences:

On the New Document tab, select "Plain text" [3]. Under Options, deselect all of the automatic formatting options except "Smart quotes and dashes in rich text documents only" [4].

On the "Open and Save" tab, select "Display HTML files as HTML code instead of formatted text" [5] and deselect "Add '.txt' extension to plain text files" [6]. The rest of the defaults should be fine.

5. When you are done, click the red button in the top-left corner to close the dialog box.
6. Now create a new document by selecting File → New. The formatting menu will no longer be there, and you can save your text as an HTML document. You can always convert a document back to rich text by selecting Format → Make Rich Text when you are not using TextEdit for HTML.

1 Formatting menu indicates rich text.

2 Plain text documents have no menu.

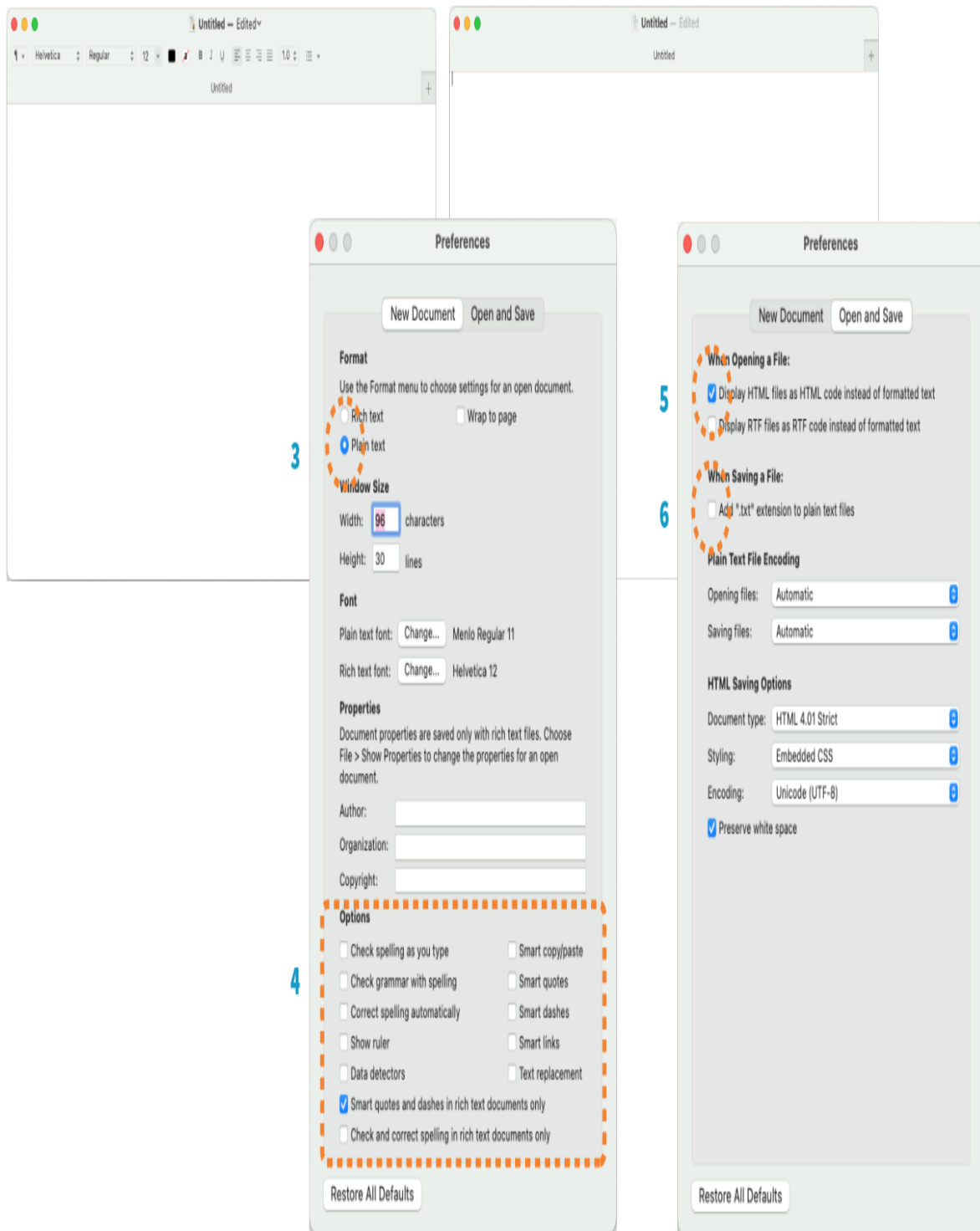


Figure 4-3. Launching TextEdit and choosing “Plain text” settings in the Preferences

Step 1: Start with Content

Now that we have our new document, it's time to get typing. A web page is all about content, so that's where we begin our demonstration. Exercise 4-1 walks you through entering the raw text content and saving the document in a new folder.

EXERCISE 4-1. ENTERING CONTENT

1. Type the home page content below into the new document in your text editor. Copy it exactly as you see it here, keeping the line breaks the same for the sake of playing along. The raw text for this exercise is also available online at learningwebdesign.com/6e/materials/.

```
Black Goose Bistro

The Restaurant
The Black Goose Bistro offers c

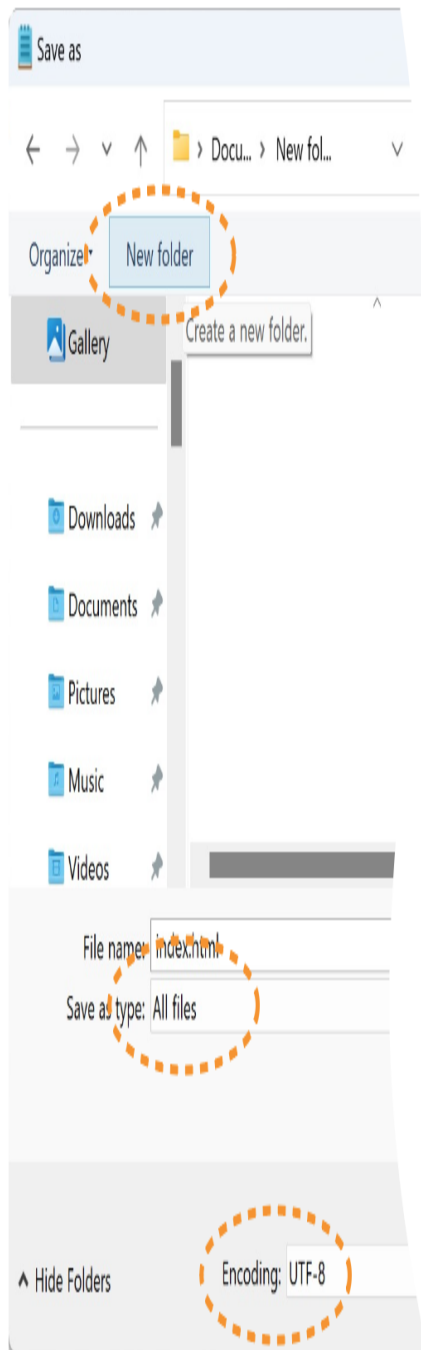
Catering
You have fun. We'll handle the

Location and Hours
Seekonk, Massachusetts;
Monday through Thursday 11am to
```



2. Select Save (or “Save as”) from the File menu to get the “Save as” dialog box (Figure 4-4). The first thing you need to do is create a new folder (click the New Folder button on both Windows and macOS) that will contain all of the files for the site. The technical name for the folder that contains everything is the local root directory.

Windows 11



MacOS

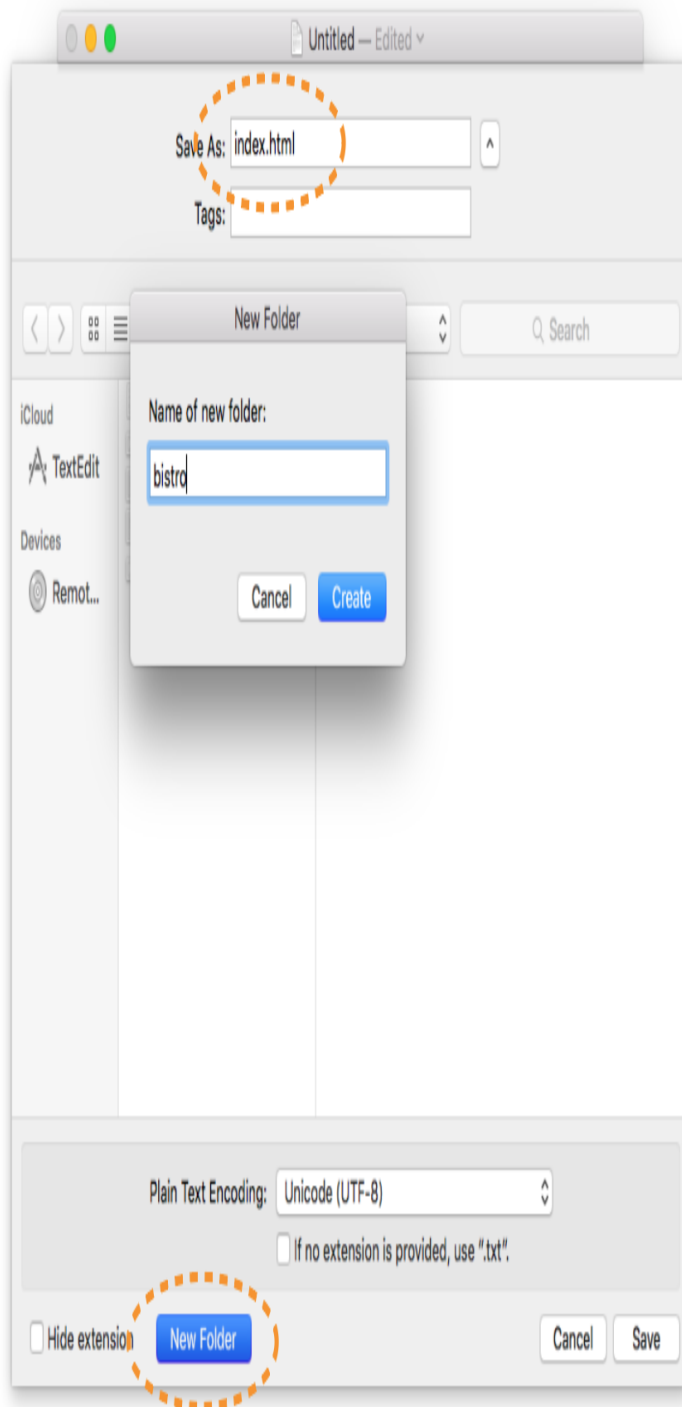


FIGURE 4-4. SAVING INDEX.HTML IN A NEW FOLDER CALLED BISTRO

Name the new folder `bistro`, and save the text file in it as `index.html`. The filename needs to end in `.html` to be recognized by the browser as a web document. See the sidebar “Naming Conventions” for more tips on naming files.

3. Just for kicks, let’s take a look at `index.html` in a browser.

Windows users: Double-click the filename in the File Explorer to launch your default browser, or right-click the file for the option to open it in the browser of your choice.

Mac users: Launch your favorite browser (I’m using Google Chrome) and choose Open or Open File from the File menu. Navigate to `index.html`, and then select the document to open it in the browser.

4. You should see something like the page shown in Figure 4-5. I’ll talk about the results in the following section.

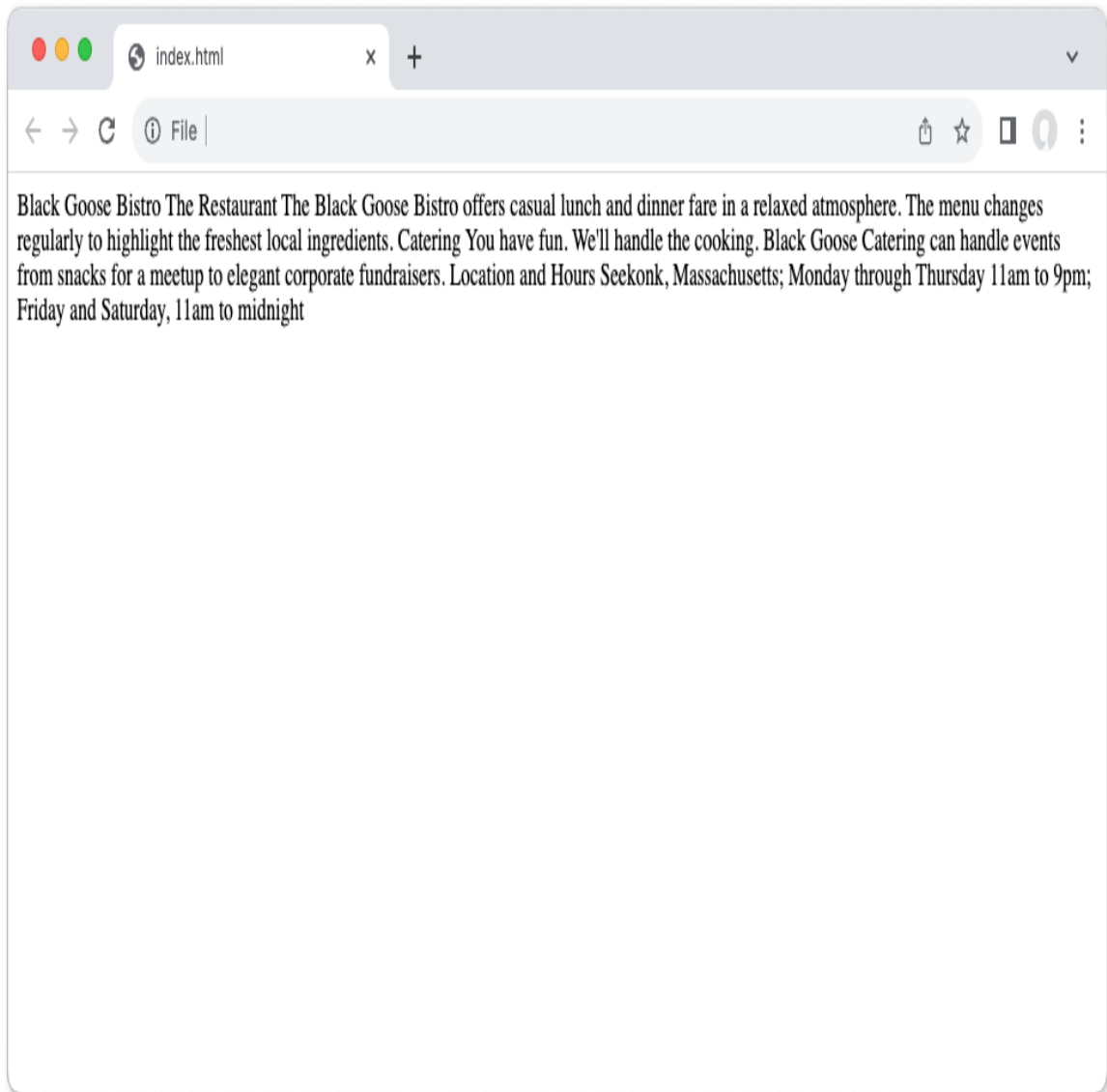


FIGURE 4-5. A FIRST LOOK AT THE CONTENT IN A BROWSER

NAMING CONVENTIONS

It's important that you follow these rules and conventions when naming your files:

- Use proper suffixes for your files. HTML files must end with `.html`. Web images must be labeled according to their file format: `.gif`, `.png`, `.jpg` (`.jpeg` is also acceptable, although less common), or `.svg`.
- Never use character spaces within filenames. It is common to use an underline character or hyphen to visually separate words within filenames, such as `robbins_bio.html` or `robbins-bio.html`.
- Avoid special characters such as `?`, `%`, `#`, `/`, `:`, `;`, `•`, etc. Limit filenames to letters, numbers, underscores, hyphens, and periods (although there may not be multiple periods in a row). It's also best to avoid international characters, such as the Swedish `å`.
- Filenames may be case-sensitive, depending on your server configuration. Consistently using all lowercase letters in filenames, although not required, is one way to make your filenames easier to manage.
- Keep filenames short. Long names are more likely to be misspelled, and short names shave a few extra bytes off the file size. If you really must give the file a long, multiword name, you can separate words with hyphens, as in `a-long-document-title.html`, to improve readability.
- Self-imposed conventions. It's helpful to develop a consistent naming scheme for large sites—for instance, always using lowercase with

hyphens between words. This takes some of the guesswork out of remembering what you named a file when you go to link to it later.

Learning from Step 1

Our page isn't looking so good (Figure 4-5). The text is all run together into one block—that's not how it looked when we typed it into the original document! There are a couple of lessons to be learned here. The first thing that is apparent is that the browser ignores line breaks in the source document. The sidebar "What Browsers Ignore" lists other types of information in the source document that are not displayed in the browser window.

Second, we see that simply typing in some content and naming the document with an .html extension is not enough. While the browser can display the text from the file, we haven't indicated the *structure* of the content. That's where HTML comes in. We'll use markup to add structure: first to the HTML document itself (coming up in step 2), then to the page's content (step 3). Once the browser knows the structure of the content, it can display the page in a more meaningful way.

WHAT BROWSERS IGNORE

The following information in the source document will be ignored when it is viewed in a browser:

Multiple character (white) spaces

When a browser encounters more than one consecutive blank character space, it displays a single space. So, if the document contains

```
long,      long      ago
```

the browser displays:

```
long, long ago
```

Line breaks (carriage returns)

Browsers convert carriage returns to spaces, so following the earlier “ignore multiple character spaces” rule, line breaks have no effect on formatting the page.

Tabs

Tabs are also converted to spaces, so guess what? They’re useless for indenting text on a web page (although they may make your code more readable).

Unrecognized markup

Browsers are instructed to ignore any tag they don't understand or that was specified incorrectly. Depending on the element and the browser, this can have varied results. The browser may display nothing at all, or it may display the contents of the tag as though it were normal text.

Text in comments

Browsers do not display text between the special `<!--` and `-->` tags used to denote a comment. See the “Adding Hidden Comments” sidebar in the discussion of step 3 for more on using comments.

Step 2: Give the HTML Document Structure

We have our content saved in an HTML document—now we're ready to start marking it up.

The Anatomy of an HTML Element

Back in Chapter 2, you saw examples of elements with an opening tag (`<p>` for a paragraph, for example) and a closing tag (`</p>`). Before we start adding tags to our document, let's look at the anatomy of an HTML element (its *syntax*) and firm up some important terminology. A generic container element is labeled in Figure 4-6.

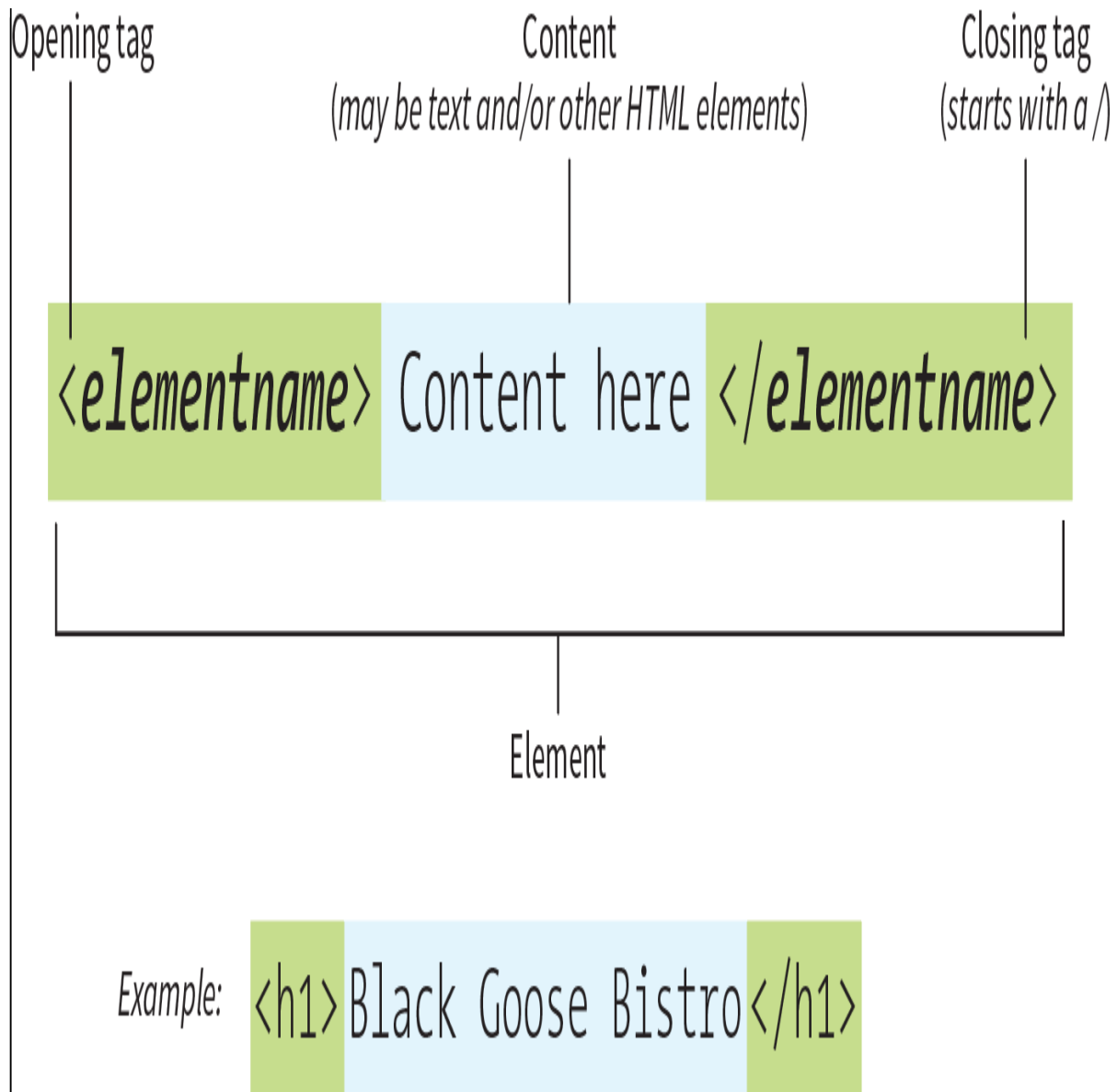


Figure 4-6. The parts of an HTML container element

Elements are identified by tags in the text source. A *tag* consists of the element name (usually an abbreviation of a longer descriptive name) within angle brackets (`< >`). The browser knows that any text within brackets is hidden and not displayed in the browser window.

The element name appears in the *opening tag* (also called a *start tag*) and again in the *closing* (or *end*) *tag*, where it is preceded by a slash (/). The closing tag works something like an “off” switch for the element. Be careful not to use the similar backslash character in end tags (see the tip “Slash Versus Backslash”).

The tags added around content are referred to as the *markup*. It is important to note that an *element* consists of both the content *and* its markup (the start and end tags). Not all elements have content, however. Some are void by definition, such as the `img` element used to add an image to the page. I’ll talk about void elements a little later in this chapter.

One last thing: capitalization. In HTML, the capitalization of element names is not important. That is, it is not case-sensitive (unlike the stricter XHTML; see Note). So, `<img>`, `<Img>`, and `<IMG>` are all the same as far as the browser is concerned. However, most developers prefer the consistency of writing element names in all lowercase, as I will be doing throughout this book.

NOTE

There is a stricter version of HTML called XHTML that requires all element and attribute names to appear in lowercase. XHTML is all but obsolete except for certain use cases when it is combined with other XML languages, but the preference for all-lowercase element names has persisted. Other strict requirements of XHTML, such as always including closing tags and always putting attribute values in quotation marks, also persist as good HTML form even though they are not required. You can learn more about XHTML in the article “HTML+ to HTML5,” available at learningwebdesign.com/articles.

SLASH VERSUS BACKSLASH

HTML tags and URLs use the slash character (/). The slash character is found under the question mark (?) on the English QWERTY keyboard (key placement on keyboards in other countries may vary).

It is easy to confuse the slash with the backslash character (\), which is found under the bar character (|); see Figure 4-7. The backslash key will not work in tags or URLs, so be careful not to use it.



FIGURE 4-7. SLASH VERSUS BACKSLASH KEYS

Basic Document Setup

Figure 4-8 shows the recommended minimal skeleton of an HTML document. I say “recommended” because the only element that is *required* in HTML is the `title`. But I feel it is better, particularly for beginners, to explicitly organize documents into metadata (`head`) and content (`body`) areas.

1 ———— `<!DOCTYPE html>`

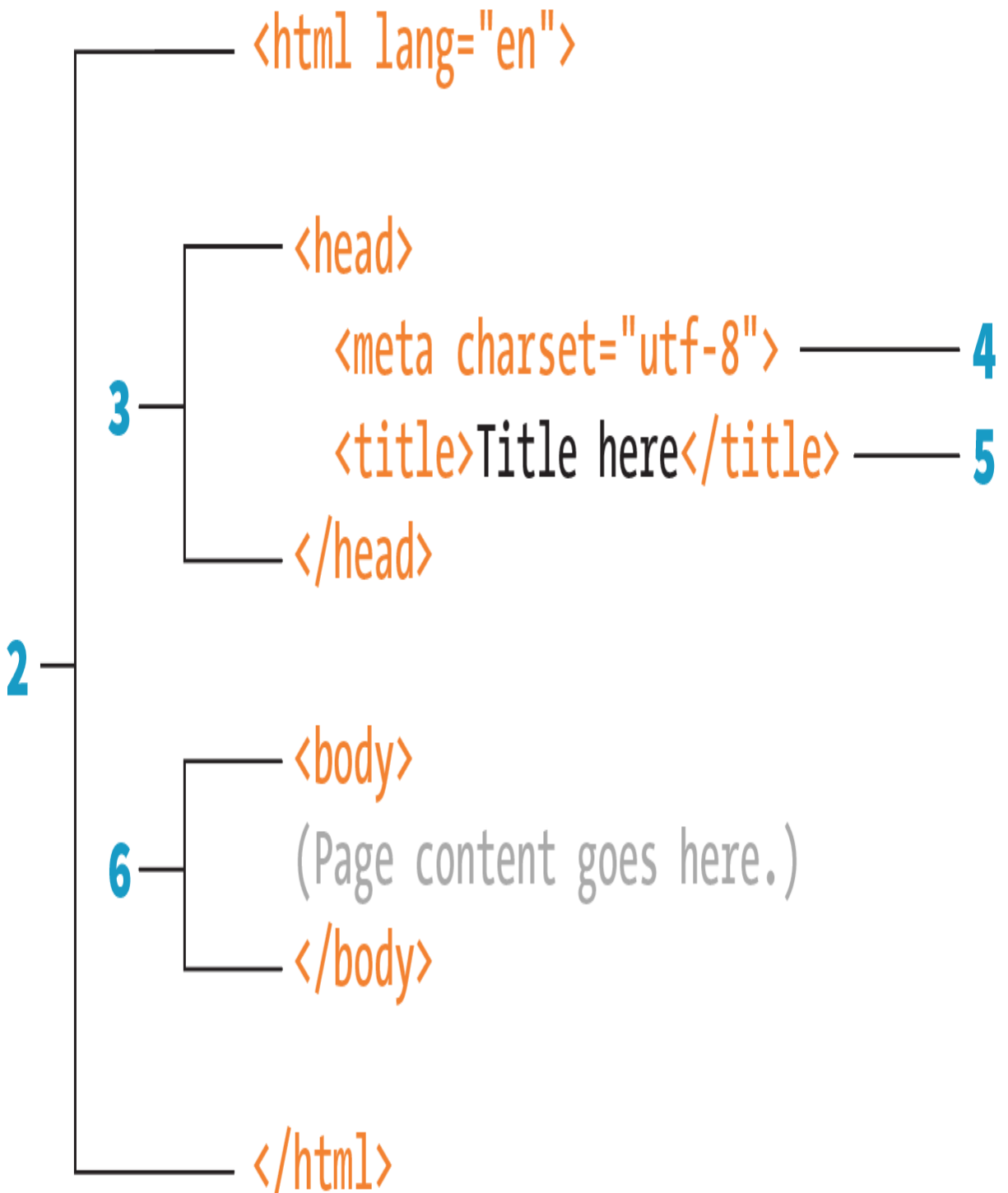


Figure 4-8. The minimal structure of an HTML document includes head and body elements contained within the html root element.

Let's take a look at what's going on in this minimal markup example:

1. Not to confuse things, but the first line in the example isn't an element at all. It's a *document type declaration* (also called a *DOCTYPE declaration*), which is required to let modern browsers know that the document is written in compliance with the current HTML specification (see Note).

NOTE

In the past, the W3C maintained many versions of HTML concurrently, each requiring a different level of strictness in how the document was authored (see the article “HTML+ to HTML5” for details). In those days, the DOCTYPE let the browser know which of those specifications to use. Thankfully, it's more straightforward now.

2. The entire document is contained within an `html` element. The `html` element is called the *root element* because it contains all the elements in the document. The `html` element may not be contained within any other element. The `lang` attribute specifies the language in which the document is written. It is not required by the spec, but including it is a best practice that aids accessibility. In the example, `en` is a *language tag* that indicates our content is written in English, which is also the default if the `lang` attribute is omitted. The full list of language abbreviations is maintained by the Internet Assigned Numbers Authority

(IANA) at iana.org/assignments/language-subtag-registry/language-subtag-registry.

3. Within the `html` element, the document is divided into a *head* and a *body*. The `head` element contains elements that pertain to the document that are not rendered as part of the content, such as its title, style sheets, scripts, and metadata.
4. `meta` elements provide document *metadata*, which is information about the document. In this case, the `meta` element specifies the *character encoding* (a standardized collection of letters, numbers, and symbols) used in the document as Unicode version UTF-8 (see the sidebar “Introducing Unicode”). I don’t want to go into too much detail on this right now, but know that there are many good reasons for specifying the `charset` in every document, so I have included it as part of the minimal document markup. Other types of metadata that can be provided by the `meta` element are the author, keywords, publishing status, and a description that can be used by search engines.
5. Also in the `head` is the mandatory `title` element. According to the HTML specification, every document must contain a descriptive title.
6. Finally, the `body` element contains everything that we want to show up in the browser window.

Are you ready to start marking up the Black Goose Bistro home page?

Open the `index.html` document in your text editor and move on to Exercise 4-2.

INTRODUCING UNICODE

All the characters that make up languages are stored in computers as numbers. A standardized collection of characters with their reference numbers (code points) is called a coded character set, and the way in which those characters are converted to bytes for use by computers is the character encoding. In the early days of computing, computers used limited character sets such as ASCII that contained 128 characters (letters from Latin languages, numbers, and common symbols). The early web used the Latin-1 (ISO 8859-1) character encoding that included 256 Latin characters from most Western languages—but given that the web was “worldwide,” this was clearly not sufficient.

Enter Unicode. Unicode (also called the Universal Character Set) is a super-character set that contains over 150,000 characters (letters, numbers, symbols, ideograms, logograms, etc.) from all active modern languages. You can read all about it at [Unicode.org](https://unicode.org). Unicode has three standard encodings—UTF-8, UTF-16, and UTF-32—that differ in the number of bytes used to represent the characters (1, 2, or 4, respectively).

HTML uses the UTF-8 encoding by default, which allows wide-ranging languages to be mixed within a single document. It is always a good idea to declare the character encoding for a document with the meta element, as shown in the previous example. Your server also needs to be configured to identify HTML documents as UTF-8 in the HTTP header (information

about the document that the server sends to the user agent). You can ask your server administrator to confirm the encoding of the HTML documents.

EXERCISE 4-2. ADDING MINIMAL MARKUP

1. Open the new index.html document if it isn't open already, and add the DOCTYPE declaration:

```
<!DOCTYPE html>
```

2. Put the entire document in an HTML root element by adding an <html> start tag after the DOCTYPE and an </html> end tag at the very end of the text. Add the lang attribute to indicate the document is written in English.
3. Next, create the document head that contains the title for the page. Insert <head> and </head> tags before the content. Within the head element, add information about the character encoding, <meta charset="utf-8">, and the title, "Black Goose Bistro", surrounded by opening and closing <title> tags.
4. Finally, define the body of the document by wrapping the text content in <body> and </body> tags. When you are done, the source document should look like this (the markup is shown in color to make it stand out):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Black Goose Bistro</title>
</head>
```

```
<body>
Black Goose Bistro

...(remaining content)...

</body>
</html>
```

5. Save the document in the bistro directory, so that it overwrites the old version. Open the file in the browser, or hit Refresh or Reload if it is open already. Figure 4-9 shows how it should look now.

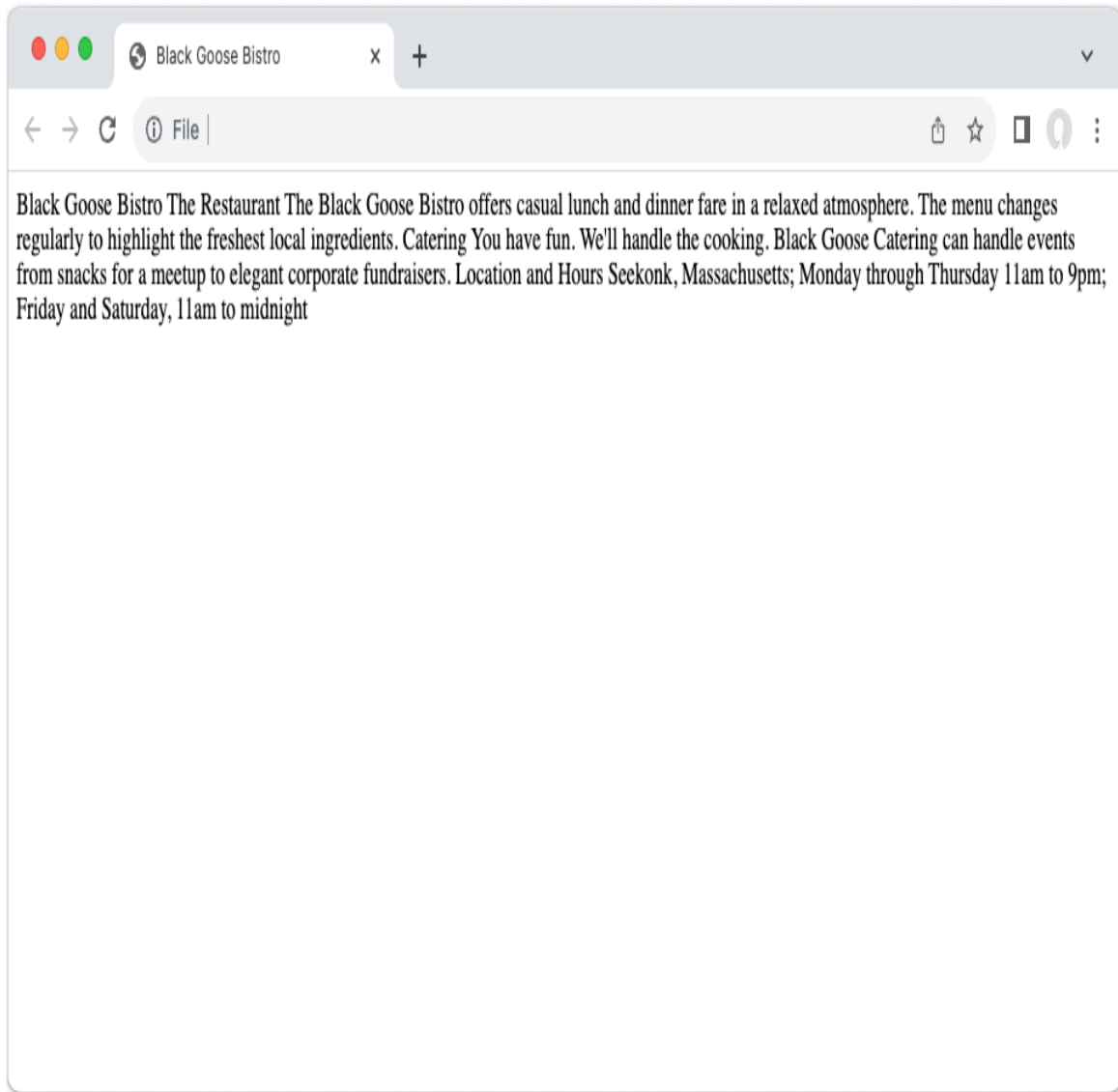


FIGURE 4-9. THE PAGE IN A BROWSER AFTER THE DOCUMENT STRUCTURE ELEMENTS HAVE BEEN DEFINED.

Not much has changed in the Bistro page after setting up the document, except that the browser now displays the title of the document in the top bar or tab (Figure 4-9). If someone were to bookmark this page, that title would be added to their Bookmarks or Favorites list as well (see the sidebar

“Don’t Forget a Good Title”). But the content still runs together, because we haven’t given the browser any indication of how it should be structured. We’ll take care of that next.

DON’T FORGET A GOOD TITLE

A title element is not only required for every document but also quite useful. The title is what displays in a user’s Bookmarks or Favorites list and on tabs in desktop browsers. Descriptive titles are also a key tool for improving accessibility, as they are the first things a person hears when using a screen reader. Search engines rely heavily on document titles as well.

For these reasons, it’s important to provide thoughtful and descriptive titles for all your documents and avoid vague titles, such as “Welcome” or “My Page.” You may also want to keep the length of your titles in check so they are able to display in the browser’s title area. Knowing that users typically have a number of tabs open or a long list of Bookmarks, put your most uniquely identifying information in the first 20 or so characters.

Step 3: Identify Elements

With a little markup experience under your belt, it should be easy to add the markup for headings and subheads (`h1` and `h2`), paragraphs (`p`), and

emphasized text (`em`) to our content, as we'll do in Exercise 4-3. However, before we begin, let's take a moment to talk about what we're doing and not doing when marking up content with HTML.

NOTE

The purpose of HTML is to add meaning and structure to the content.

Mark It Up Semantically

The purpose of HTML is to add meaning and structure to the content. It is *not* intended to describe how the content should look (its presentation; see Note).

NOTE

In the early days of the web, before we had style sheets, HTML was extended to give authors ways to change the appearance of fonts, colors, and alignment using markup alone. Those presentational extras (like the `font` element) were kicked to the curb long ago and are not in the current spec, but you may run across them if you view the source of an archived site or in the results of a Google search. In this book, you'll learn HTML the right way.

Your job when marking up content is to choose the HTML element that provides the most meaningful description of the content at hand. In the biz, we call this *semantic markup*. For example, the most important heading at the beginning of the document should be marked up as an `h1` because it is

the most important heading on the page. Don't worry about what it looks like; you can easily change that with a style sheet. The important thing is that you choose elements based on what makes the most sense for the content.

It is worth noting that you can't make up your own elements in the markup; you need to choose from the elements defined in the HTML standard (see Note).

NOTE

Technically, there is a way to create your own custom elements and define their behaviors, not in HTML, but by using JavaScript as part of the Web Components specification. That's a topic well beyond the scope of this book, but perhaps you'll get to try it one day as a seasoned web developer.

In addition to adding meaning to content, the markup creates the document's structure. The way elements follow each other or nest within one another creates relationships between them. You can think of this structure as an outline (its technical name is the *DOM*, for *Document Object Model*).

There are a lot of benefits to having a clear, well-defined document hierarchy. The underlying hierarchy gives browsers, and accessibility technology like screen readers, cues on how to handle the content. The document hierarchy is also the foundation upon which we add presentation instructions with style sheets and behaviors with JavaScript.

OK, enough lecturing. It's time to get to work on that content in Exercise 4-3.

EXERCISE 4-3. DEFINING TEXT ELEMENTS

1. Open the document index.html in your text editor, if it isn't open already.
2. The first line of text, "Black Goose Bistro," is the main heading for the page, so we'll mark it up as a heading level 1 (h1) element. Put the opening tag, <h1>, at the beginning of the line and the closing tag, </h1>, after it, like this:

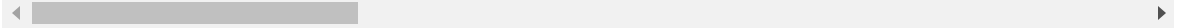
```
<h1>Black Goose Bistro</h1>
```

3. Our page also has three subheads. Mark them up as heading level 2 (h2) elements in a similar manner. I'll do the first one here; you do the same for "Catering" and "Location and Hours":

```
<h2>The Restaurant</h2>
```

4. Each h2 element is followed by a brief paragraph of text, so let's mark those up as paragraph (p) elements in a similar manner. Here's the first one; you do the rest:

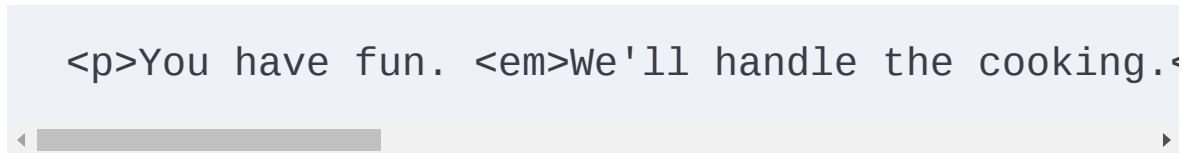
```
<p>The Black Goose Bistro offers casual lunch a
```



5. Finally, in the Catering section, I want to emphasize that visitors should just leave the cooking to us. To turn that sentence into "emphasized

text,” mark it up in an emphasis (em) element, as shown here:

```
<p>You have fun. <em>We'll handle the cooking.</em>
```



6. Now that we’ve marked up the document, let’s save it as we did before, and open (or reload) the page in the browser. You should see a page that looks much like the one in Figure 4-10. If it doesn’t, check your markup to be sure that you aren’t missing any angle brackets or a slash in a closing tag.

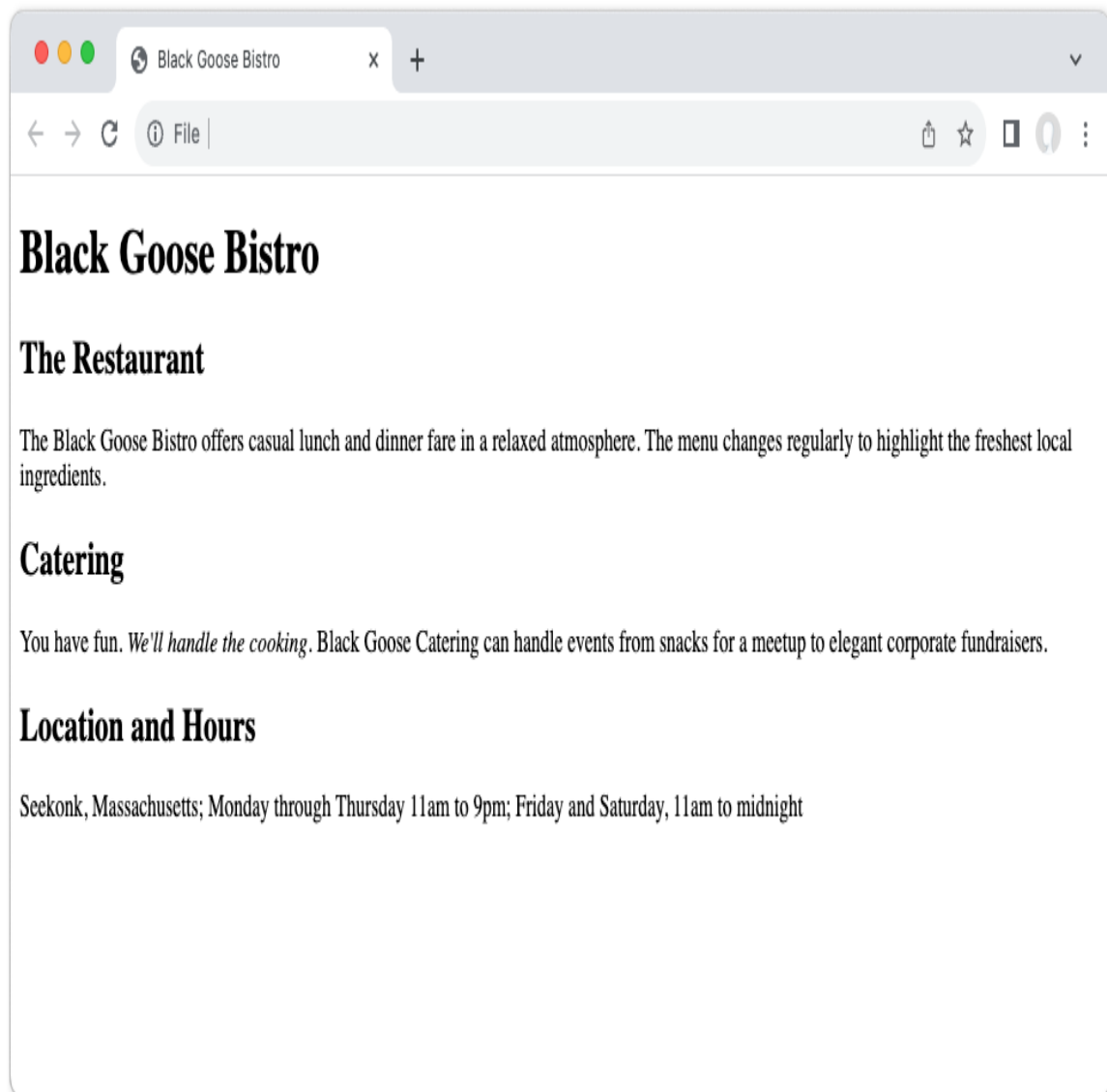


FIGURE 4-10. THE HOME PAGE AFTER THE CONTENT HAS BEEN MARKED UP WITH HTML ELEMENTS

Now we're getting somewhere. With the elements properly identified, the browser can now display the text in a more meaningful manner. There are a

few significant things to note about what's happening in Figure 4-10; we'll look at those in the following sections.

Block and Inline Elements

Although it may seem like stating the obvious, it's worth pointing out that the heading and paragraph elements in our example start on new lines and do not run together as they did before. That's because by default, headings and paragraphs display as *block elements*. Browsers treat block elements as though they are in little rectangular boxes, stacked up in the page. Each block element begins on a new line, and some space is also usually added above and below the entire element by default. In Figure 4-11, the edges of the block elements are outlined in red.

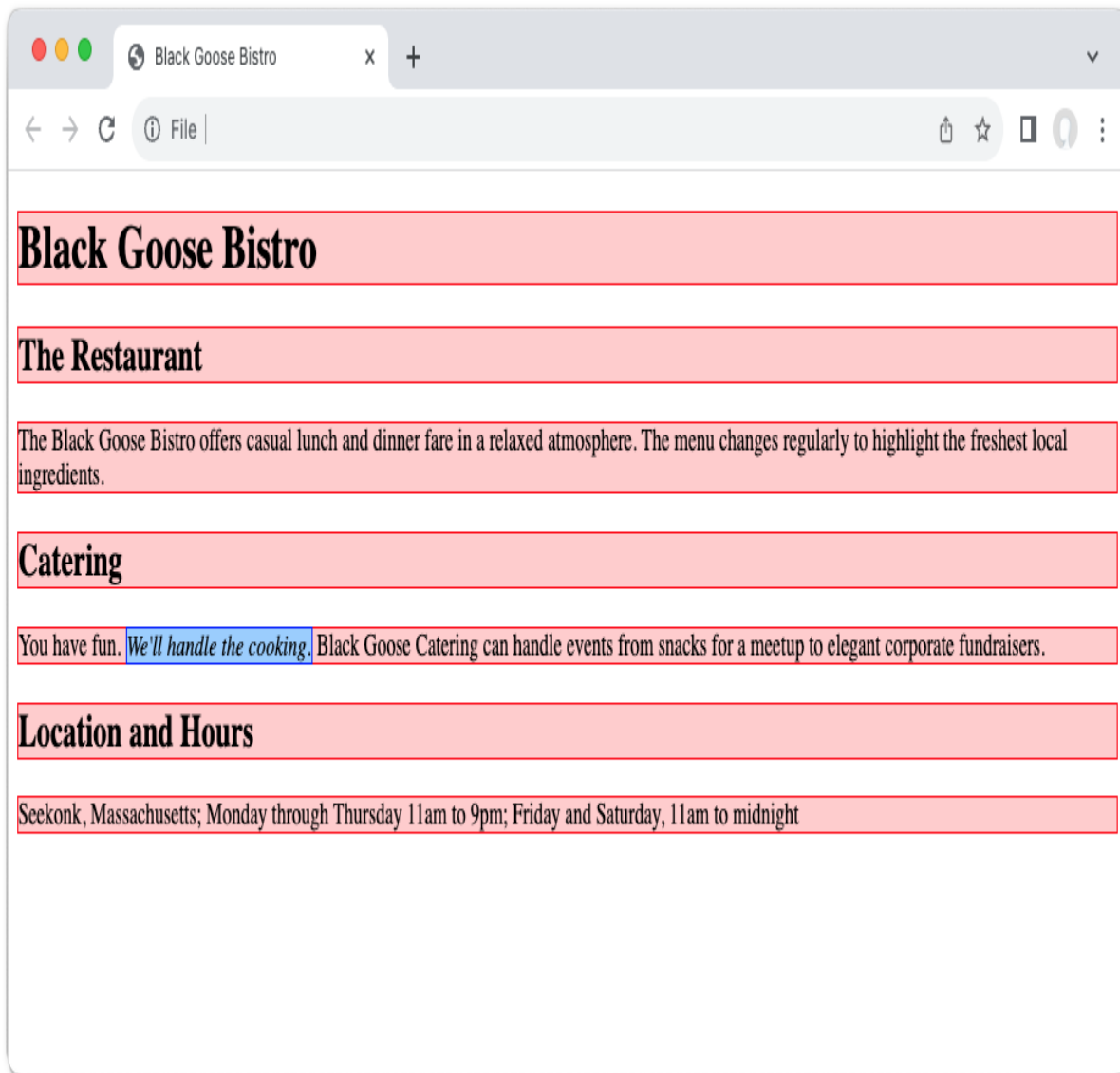


Figure 4-11. The outlines show the structure of the elements in the home page.

By contrast, look at the text we marked up as emphasized (`em`, outlined in blue in Figure 4-11). It does not start a new line, but rather stays in the flow of the paragraph. That's because the `em` element is a *text-level element* (also called a *phrasing element* or *inline element*). By default, text-level elements do not start new lines; they just go with the flow.

ADDING HIDDEN COMMENTS

You can leave notes in the source document for yourself and others by marking them up as comments. Anything you put between comment tags (<!-- and -->) will not display in the browser and will not have any effect on the rest of the source:

```
<!-- This is a comment -->  
<!-- This is a  
    multiple-line comment  
    that ends here. -->
```

Comments are useful for labeling and organizing long documents, particularly when they are shared by a team of developers. In this example, comments are used to point out the section of the source document that contains the navigation:

```
<!-- start global nav -->  
<ul>  
    ...  
</ul>  
<!-- end global nav -->
```

Bear in mind that although the browser will not display comments in the web page, readers can see them if they “view source,” so be sure that the comments you leave are appropriate for everyone.

Default Styles

The other thing that you will notice about the marked-up page in Figures 4-10 and 4-11 is that the browser makes an attempt to give the page some visual hierarchy by making the first-level heading the biggest and boldest thing on the page, with the second-level headings slightly smaller, and so on.

How does the browser determine what an `h1` should look like? It uses a style sheet! All browsers have their own built-in style sheets (called *user agent style sheets* in the HTML spec) that describe the default rendering of elements. The default rendering is similar from browser to browser (for example, `h1`s are always big and bold), but there are some variations (the `blockquote` element for long quotes may or may not be indented, for instance).

If you think the `h1` is too big and clunky as the browser renders it, change it with your own style sheet rule. Resist the urge to mark up the heading with another element just to get it to look better—for example, using an `h3` instead of an `h1` so it isn't as large. You should always choose elements based on how accurately they describe the content, and don't worry about the browser's default rendering.

We'll fix the presentation of the page with style sheets in a moment, but first, let's add an image to the page.

Step 4: Add an Image

What fun is a web page with no images? In Exercise 4-4, we'll add an image to the page with the `img` element. Images will be discussed in more detail in Chapter 7, Adding Images, but for now, they give me an opportunity to introduce two more basic markup concepts: void elements and attributes.

Void Elements

So far, nearly all of the elements we've used in the Black Goose Bistro home page have followed the syntax shown in Figure 4-6: a bit of text content surrounded by start and end tags.

A handful of elements, however, do not have content because they are used to provide simple directives. These elements are said to be *void* (previously called *empty*, so you may also hear that term). The image element (`img`) is an example of a void element. It tells the browser to get an image file from the server and embed it at that spot in the flow of the text. Other void elements include the line break (`br`), thematic breaks (`hr`, a.k.a. “horizontal rules”), and elements that provide information about a

document but don't affect its displayed content, such as the `meta` element that we used earlier.

NOTE

The complete list of void elements as defined in the spec is `area`, `base`, `br`, `col`, `embed`, `hr`, `img`, `input`, `link`, `meta`, `source`, `track`, and `wbr`.

Figure 4-12 shows the very simple syntax of a void element (compare it to Figure 4-6).

The diagram shows the syntax of a void element, which is a single tag without a closing tag. It is represented as `<element-name>` inside a light green rectangular box.

Example: The `br` element inserts a line break.

`<p>1005 Gravenstein Highway North<br>Sebastopol, CA 95472</p>`

Figure 4-12. Void element structure

WHAT IS THAT EXTRA SLASH?

If you poke around in source documents for existing web pages, you may see void elements with extra slashes at the end, like so: `<img />`, `<br />`, `<meta />`, and `<hr />`. That indicates the document was written according to the stricter rules of XHTML. In XHTML, *all* elements, including void elements, must be closed (or terminated, to use the proper term). You terminate void elements by adding a trailing slash before the closing bracket. The preceding character space is not required but was used for backward compatibility with browsers that did not have XHTML parsers, so `<img/>`, `<br/>`, and so on are also valid. It is no longer required to self-close void elements, so I will not be doing so in this book. Some developers continue to do so out of habit or preference.

Attributes

Let's get back to adding an image with the `img` element. Obviously, an `<img>` tag is not very useful by itself—it doesn't indicate which image to use. That's where attributes come in. *Attributes* are instructions that clarify or modify an element. For the `img` element, the `src` (short for “source”) attribute is required, and specifies the location (URL) of the image file. The `alt` attribute is also required for providing a description of the image (we'll talk more about alternate text in Chapter 7).

NOTE

Attributes are instructions that clarify or modify an element.

The syntax for an attribute is as follows:

```
attributename="value"
```

Attributes go after the element name, separated by a space. In content (non-void) elements, attributes go in the opening tag only:

```
<element attributename="value">  
<element attributename="value">Content</element>
```

You can also put more than one attribute in an element, in any order. Just keep them separated with spaces:

```
<element attribute1="value1" attribute2="value2">
```

Figure 4-13 shows an `img` element with its required attributes labeled.

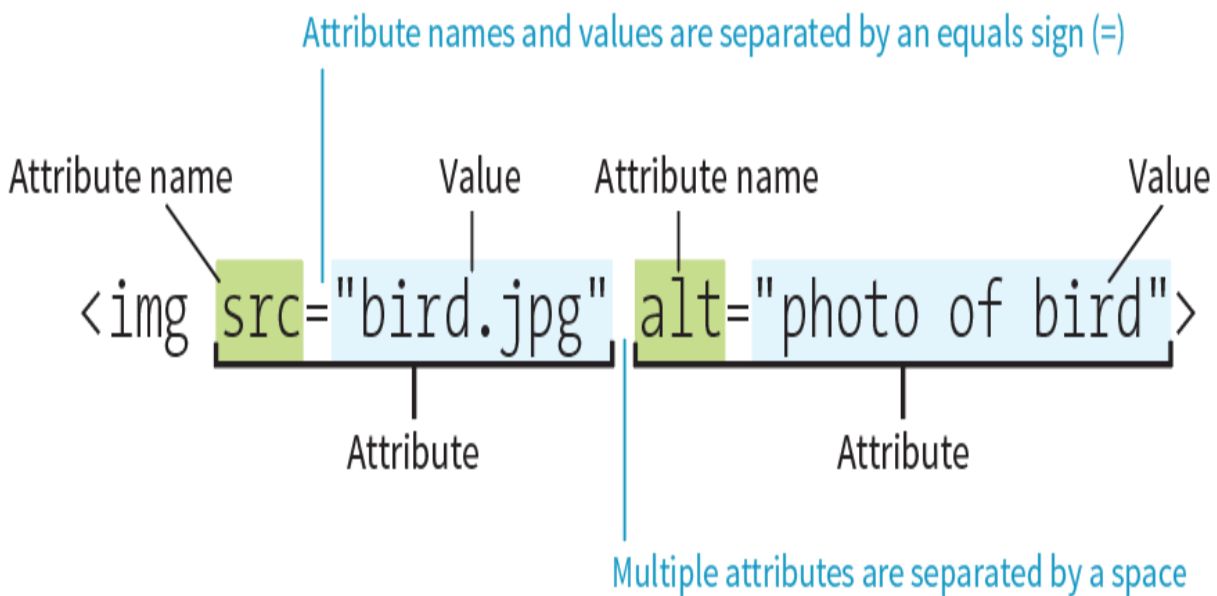


Figure 4-13. An `img` element with two attributes

Here's what you need to know about attributes:

- Attributes go after the element name in the opening tag only, never in the closing tag.
- There may be several attributes applied to an element, separated by spaces in the opening tag. Their order is not important.
- Most attributes take values, which follow an equals sign (=). In HTML, some attribute values are single descriptive words. For example, the `checked` attribute, which makes a form checkbox checked when the form loads, is equivalent to `checked="checked"`. You may hear this type of attribute called a *Boolean attribute* because it describes a feature that is either on or off.

- A value might be a number, a word, a string of text, a URL, or a measurement, depending on the purpose of the attribute.
- Wrapping attribute values in double quotation marks is a strong convention, but note that quotation marks are not required and may be omitted. In addition, either single or double quotation marks are acceptable, as long as the opening and closing marks match. Note that quotation marks in HTML files need to be straight (“), not curly (”).
- The attribute names and values available for each element are defined in the HTML specification; in other words, you can’t make up an attribute for an element.
- Some attributes are required, such as the `src` and `alt` attributes in the `img` element. The HTML specification also defines which attributes are required in order for the document to be valid.

Now you should be more than ready to try your hand at adding the `img` element with its attributes to the Black Goose Bistro page in Exercise 4-4. We’ll throw a few line breaks in there as well.

EXERCISE 4-4. ADDING AN IMAGE

1. If you're working along, the first thing you'll need to do is get a copy of the image file on your hard drive so you can see it in place when you open the file locally. The image file is provided with the materials for this chapter. You can also get the image file by saving it right from the sample web page online, at learningwebdesign.com/6e/materials/bistro. Right-click (or Control-click on a Mac) the goose image and select "Save image" (or similar) from the pop-up menu, as shown in Figure 4-14. Name the file `blackgoose.png`. Be sure to save it in the bistro folder with `index.html`.
2. Once you have the image, insert it before the first-level heading by typing in the `img` element and its attributes as shown here:

```

<h1>Black Goose Bistro</h1>
```



Windows: Right-click on the image to access the pop-up menu.

Mac (shown): Control-click on the image to access the pop-up menu. The options may vary by browser.

FIGURE 4-14. SAVING AN IMAGE FILE FROM A PAGE ON THE WEB

The src attribute provides the location and name of the image file that should be inserted, and the alt attribute provides text that should be displayed if the image is not available. Both of these attributes are required in every img element in order for the document to be valid.

3. Make it clear that the image and heading work together as the header of the document by wrapping the elements in opening and closing <header> tags. This also allows us to apply styles to the elements as a unit in the next exercise:

```
<header>
  
  <h1>Black Goose Bistro</h1>
</header>
```

4. Let's break up the last paragraph into three lines for better clarity. Drop line break (
) tags at the spots where you'd like the line breaks to occur. Try to match the screenshot in Figure 4-15.
5. Now save index.html and open or refresh it in the browser window. The page should look like the one shown in Figure 4-15. If it doesn't, check to make sure that the image file, blackgoose.png, is in the same directory as index.html. If it is, then check to make sure that you aren't missing any characters, such as a closing quote or bracket, in the img element markup.

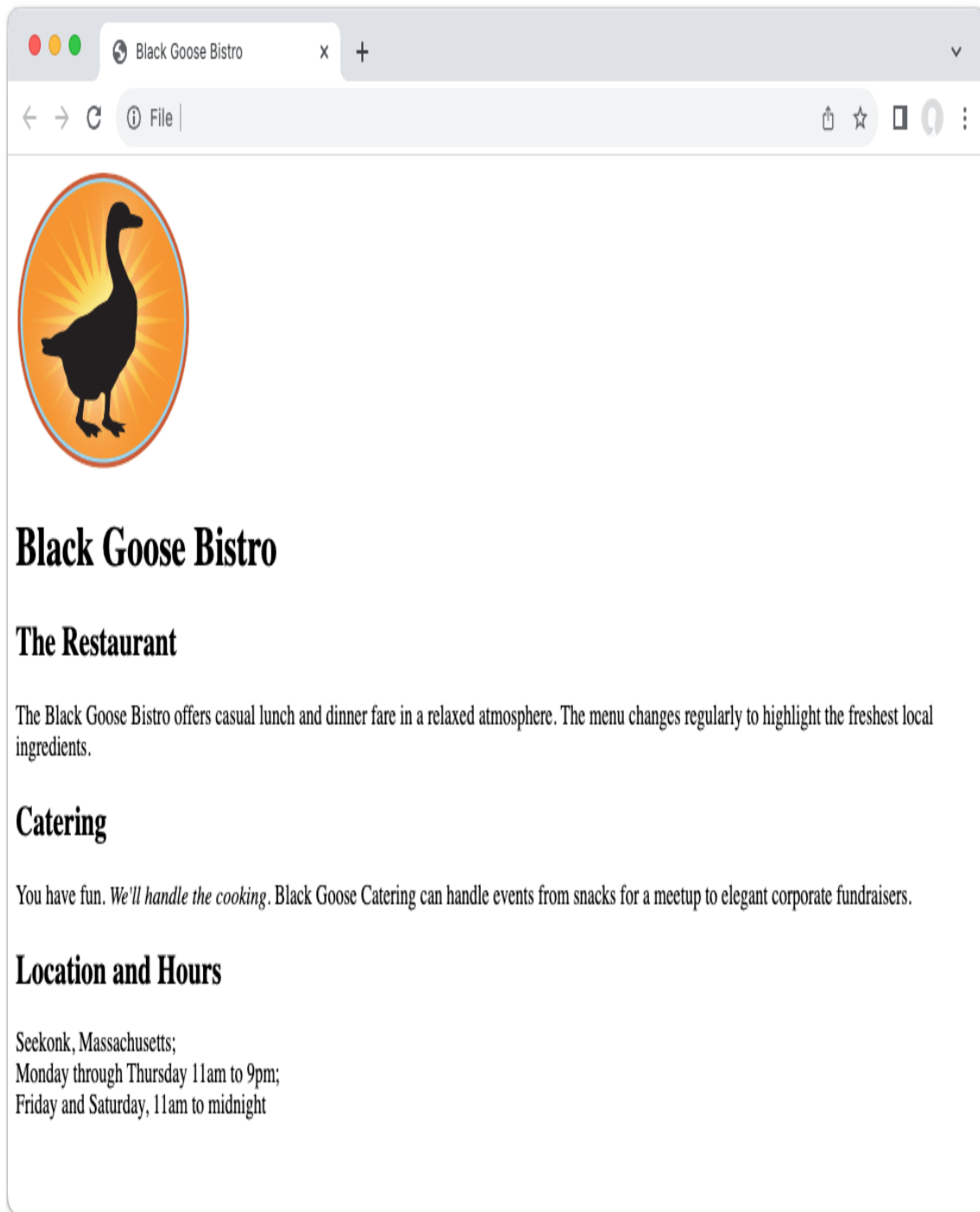


FIGURE 4-15. THE BLACK GOOSE BISTRO PAGE WITH THE LOGO IMAGE

Step 5: Change the Look with a Style Sheet

Depending on the content and purpose of your website, you may decide that the browser's default rendering of your document is perfectly adequate.

However, we're going to pretty up the Black Goose Bistro home page a bit to make a good first impression on potential patrons. "Prettying up" is just my way of saying that I'd like to change its presentation, which is the job of Cascading Style Sheets (CSS).

In Exercise 4-5, we'll change the appearance of the text elements and the page background by using some simple style sheet rules. Don't worry about understanding them all right now. I'll get into CSS in more detail in Part III, CSS for Presentation. But I want to at least give you a taste of what it means to add a "layer" of presentation onto the structure we've created with our markup.

EXERCISE 4-5. ADDING A STYLE SHEET

1. Open `index.html` if it isn't open already. We're going to use the `style` element to apply a very simple embedded style sheet to the page. This is just one of the ways to add a style sheet; the others are covered in Chapter 11, *Introducing Cascading Style Sheets*.

2. The style element is placed inside the document head. Start by adding the style element to the document as shown here:

```
<head>
  <meta charset="utf-8">
  <title>Black Goose Bistro</title>
  <style>
  </style>
</head>
```

3. Next, type the following style rules within the style element just as you see them here. Don't worry if you don't know exactly what's going on (although it's fairly intuitive); you'll learn all about style rules in Part III. You can also copy and paste these style rules from the finished exercise document in the materials folder.

```
<style>
  body {
    margin: 0 10%;
    font: 1.25rem/1.75rem sans-serif;
    color: floralwhite;
    background-color: darkslategray;
  }
  header {
    background-color: #faf2e4;
    text-align: center;
```

```
padding: 12px;
margin-bottom: 36px;
}
h1 {
color: darkslategray;
font: bold 1.75rem sans-serif;
text-transform: uppercase;
letter-spacing: 5px;
margin: 0;
}
h2 {
color: orange;
font: bold 1.25rem sans-serif;
text-align: center;
}
</style>
```

4. Now it's time to save the file and take a look at it in the browser. It should look like the page in Figure 4-16. If it doesn't, go over the style sheet to make sure you didn't miss a semicolon or a curly brace. Compare the way the page looks with our styles to how it looked with the browser's default styles (Figure 4-15).



BLACK GOOSE BISTRO

The Restaurant

The Black Goose Bistro offers casual lunch and dinner fare in a relaxed atmosphere. The menu changes regularly to highlight the freshest local ingredients.

Catering

You have fun. *We'll handle the cooking.* Black Goose Catering can handle events from snacks for a meetup to elegant corporate fundraisers.

Location and Hours

Seekonk, Massachusetts;

Monday through Thursday 11am to 9pm;

Friday and Saturday, 11am to midnight

FIGURE 4-16. THE BLACK GOOSE BISTRO PAGE AFTER CSS STYLE RULES HAVE BEEN APPLIED

We're finished with the Black Goose Bistro page. Not only have you written your first web page, complete with a style sheet, but you've also learned about elements, attributes, void elements, block and inline elements, the basic structure of an HTML document, and the correct use of markup along the way. Not bad for one chapter!

When Good Pages Go Bad

The previous demonstration went smoothly, but it's easy for small things to go wrong when you're typing out HTML markup by hand. Unfortunately, one missed character can break a whole page. Let's try breaking some elements on purpose so you can see what happens.

What if we had neglected to type the slash in the closing emphasis tag (`</em>`) in Exercise 4-3? With just one character out of place, the remainder of the document displays in emphasized (italic) text, as shown in Figure 4-17. That's because without that slash, there's nothing telling the browser to "turn off" the emphasized formatting, so it just keeps going (see Note).

```
<h2>Catering</h2>
```

```
<p>You have fun. <em>We'll handle the cooking.<em> Black Goose  
Catering can handle events from snacks for a meetup to elegant  
corporate fundraisers.</p>
```

g.

Catering

You have fun. We'll handle the cooking. Black Goose Catering can handle events from snacks for a meetup to elegant corporate fundraisers.

Location and Hours

*Seekonk, Massachusetts;
Monday through Thursday 11am to 9pm;
Friday and Saturday, 11am to midnight*

Figure 4-17. When a slash is omitted, the browser doesn't know when the element ends, as is the case in this example.

NOTE

Omitting the slash in the closing tag (or even omitting the closing tag itself) for block elements, such as headings or paragraphs, may not be so dramatic. Browsers interpret the start of a new block element to mean that the previous block element is finished.

We've fixed the slash, but this time, let's see what would have happened if we had accidentally omitted a bracket from the end of the first `<h2>` tag (Figure 4-18).

```
<h2The Restaurant</h2>
```

```
<p>The Black Goose Bistro offers casual lunch and dinner fare  
in a relaxed atmosphere. The menu changes regularly to highlight  
the freshest local ingredients.</p>
```

```
<h2The
```

Without the bracket, all the following characters
are interpreted as part of the tag, and “The
Restaurant” disappears from the page.

Missing subhead



Figure 4-18. A missing end bracket makes the browser think the following characters are part of the tag, and therefore the headline text doesn’t display.

See how the headline is missing? That’s because without the closing tag bracket (>), the browser assumes that all the following text—all the way

up to the next closing bracket it finds—is part of the `<h2>` opening tag. Browsers don't display any text within a tag, so the heading disappeared. The browser just ignored the long, foreign-looking element name and moved on to the next element.

Making mistakes in your first HTML documents and fixing them is a great way to learn. If you write your first pages perfectly, try fiddling with the code to see how the browser reacts to various changes. This can be extremely useful in troubleshooting pages later. The sidebar “Having Problems?” lists some common problems. Note that these issues are not specific to beginners. Little stuff like this goes wrong all the time, even for the pros.

HAVING PROBLEMS?

The following are some typical problems that crop up when you are creating web pages and viewing them in a browser:

I've changed my document, but when I reload the page in my browser, it looks exactly the same.

It could be you didn't save your document before reloading, or you may have saved it in a different directory.

Half my page disappeared.

This could happen if you are missing a closing bracket (>) or a quotation mark within a tag. This is a common error when you're writing HTML by hand.

I put in an image by using the `img` element, but all that shows up is a broken image icon.

The broken icon could mean a couple of things. First, it might mean that the browser is not finding the image file. Make sure that the URL to the file is correct (we'll discuss URLs further in Chapter 6, Adding Links), and that the image file is actually in the directory you've specified. If the file is there, make sure it is in one of the formats that web browsers can display (such as PNG, JPEG, WebP, GIF, or SVG) and that it is named with the proper suffix (.png, .jpg, .jpeg, .webp, .gif, or .svg, respectively).

Validating Your Documents

One way that professional web developers catch errors in their markup is by validating their documents. What does that mean? To *validate* a document is to check your markup to make sure that you have abided by all the rules of HTML. Documents that are error-free are said to be *valid*. It is strongly recommended that you validate your documents, especially for professional sites. Valid documents are more consistent on a variety of browsers, display more quickly, and are more accessible.

Browsers are very forgiving and don't require documents to be valid (in other words, they'll do their best to display them, errors and all), but anytime you stray from the standard, you introduce unpredictability in the way the page is handled by browsers or alternative devices.

So how do you make sure your document is valid? You could check it yourself or ask a friend, but humans make mistakes, and you aren't expected to memorize every minute rule in the specifications. Instead, use a *validator*, software that checks your source against the HTML version you specify. These are some things validators check for:

- The inclusion of a DOCTYPE declaration
- The inclusion of required rules and attributes
- Nonstandard elements
- Mismatched tags

- Nesting errors (incorrectly putting elements inside other elements)
- Typos and other minor errors

Developers use a variety of helpful tools for checking and correcting errors in HTML documents. Many code editors, frameworks, and other developer build tools have validators built in. Browsers such as Safari and Chrome feature a number of developer tools, including validators, so you can check your work on the fly. Also, a lot of code editors nest and close elements and attributes automatically as you type, so there is less room for error in the writing process itself.

The Nu Html Checker (validator.nu) is a popular online validator. You can upload a file, or provide a link to a page that is already online. Figure 4-19 shows the report the validator generates when I upload the version of the `Bistro index.html` file that doesn't have any markup. It shows that there are a number of missing elements that keep this document from being valid. The report also highlights the problem source code and provides an explanation of how the code should appear. Pretty darned handy!

(X)HTML5 validation results for ex4-1 index.html

Validator Input

File Upload  Choose File no file selected

☐ Show Image Report

☐ Show Source

Group Messages

1. **Error:** The character encoding was not declared. Proceeding using windows-1252.
2. **Error:** Non-space characters found without seeing a doctype first. Expected <!DOCTYPE html>.
From line 1, column 1; to line 11, column 75
Black Goose Bistro↵↵The Restaurant↵The Black Goose Bistro offers casual lunch and dinner fare in a relaxed atmosphere. The menu changes regularly to highlight the freshest local ingredients.↵↵Catering↵You have fun. We'll handle the cooking. Black Goose Catering can handle events from snacks for a meetup to elegant corporate fundraisers.↵↵Location and Hours↵Seekonk, Massachusetts;↵Monday through Thursday 11am to 9pm; Friday and Saturday, 11am to midnight↵↵
3. **Error:** Element [head](#) is missing a required instance of child element [title](#).
From line 1, column 1; to line 11, column 75
Black Goose Bistro↵↵The Restaurant↵The Black Goose Bistro offers casual lunch and dinner fare in a relaxed atmosphere. The menu changes regularly to highlight the freshest local ingredients.↵↵Catering↵You have fun. We'll handle the cooking. Black Goose Catering can handle events from snacks for a meetup to elegant corporate fundraisers.↵↵Location and Hours↵Seekonk, Massachusetts;↵Monday through Thursday 11am to 9pm; Friday and Saturday, 11am to midnight↵↵
Content model for element [head](#):
If the document is an [iframe srcdoc document](#) or if title information is available from a higher-level protocol: Zero or more elements of [metadata content](#), of which no more than one is a [title](#) element and no more than one is a [base](#) element.
Otherwise: One or more elements of [metadata content](#), of which exactly one is a [title](#) element and no more than one is a [base](#) element.
A [head](#) element's [start tag](#) can be omitted if the element is empty, or if the first thing inside the [head](#) element is an element.
A [head](#) element's [end tag](#) can be omitted if the [head](#) element is not immediately followed by a [space character](#) or a [comment](#).

There were errors. (Tried in the text/html mode.)

The Content-Type was text/html. Used the HTML parser.

Total execution time 2 milliseconds.

[About this Service](#) • [More options](#)

Figure 4-19. Using the Nu Html Checker to validate an HTML document

HTML Resources

Now you have a little hands-on experience marking up a document and a feel for what HTML is all about. In the upcoming chapters, I will present collections of HTML elements organized by topic: text content, links, images, tables, forms, and embedded media. By the end of Part II, you will have gotten a thorough introduction to the majority of the elements in the HTML specification—certainly the ones you are most likely to need when you’re just starting out (and many you may never use at all). If you feel you need more explanation or more examples than I was able to include here, I encourage you to get familiar with these sites, which are excellent resources for students as well as experienced developers:

MDN Web Docs (developer.mozilla.org)

This site is many developers’ go-to for looking up information about HTML, CSS, JavaScript, and other web technologies. It goes into deeper technical detail than I do here, so it will help you take your skills to the next level and help you out when you’re stuck. It is published by Mozilla, the creators of the Firefox browser.

Web.dev (web.dev)

Google’s Chrome Developer Relations team publishes comprehensive courses and articles about HTML, CSS, and JavaScript.

W3Schools ([w3schools.com](https://www.w3schools.com))

W3Schools publishes an HTML element reference ([w3schools.com/tags/](https://www.w3schools.com/tags/)) as well as succinct tutorials on HTML and many other web-related topics. Exercises and quizzes are available if you'd like to put your knowledge to the test, and W3Schools also offers courses and certification exams for a fee.

The HTML Living Standard (html.spec.whatwg.org)

Get the definitive answer on all matters of HTML in the current spec.

Element Review: HTML Document Setup

This chapter introduced the following elements that establish the metadata and content portions of an HTML document. The remaining elements introduced in the exercises will be treated in more depth in upcoming chapters.

Element	Description
body	Identifies the body of the document that holds the content
head	Identifies the head of the document that contains information about the document itself
html	Is the root element that contains all the other elements
meta	Provides information about the document
title	Gives the page a title

Chapter 5. Marking Up Text

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 5th chapter of the final book.

If you’d like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at jbleiel@oreilly.com.

In this chapter:

- Choosing the best element for your content
- Paragraphs and headings
- Three types of lists
- Organizing content into sections
- Text-level elements
- Generic elements, div and span
- Special characters

Once your content is ready to go (you’ve proofread it, right?) and you’ve added the markup to structure the document (`<!DOCTYPE>` , `html` ,

`head`, `title`, `meta` `charset`, and `body`), you're ready to identify the elements in the content. This chapter introduces the elements you have to choose from for marking up text. There probably aren't as many of them as you might think, and really just a handful that you'll use with regularity. That said, this chapter is a big one and covers a lot of ground.

As we begin our tour of elements, I want to reiterate how important it is to choose elements *semantically*—that is, in a way that most accurately describes the content's meaning. If you don't like how it looks, change it with a style sheet. A semantically marked-up document ensures your content is available and accessible in the widest range of browsing environments, from desktop computers and mobile devices to assistive screen readers. It also allows non-human readers, such as search engine indexing programs, to correctly parse your content and make decisions about the relative importance of elements on the page.

With these principles in mind, it's time to meet the HTML text elements, starting with the most basic element of them all, the humble paragraph.

NOTE

This book covers markup according to the HTML Living Standard maintained by the WHATWG (whatwg.org).

Paragraphs

```
<p>...</p>
```

Paragraph element

Paragraphs are the most rudimentary elements of a text document. Indicate a paragraph with the `p` element by inserting an opening `<p>` tag at the beginning of the paragraph and a closing `</p>` tag after it, as shown in this example:

```
<p>Serif typefaces have small slabs at the ends of  
the letters. The most common serif typeface is Times  
New Roman. It is a serif typeface because it has  
slabs on the ends of the letters. The most common  
sans-serif font is Helvetica. It is a sans-serif  
font because it does not have slabs on the ends  
of the letters. The most common monospace font  
is Courier New. It is a monospace font because  
all the letters are the same width. The most  
common variable font is Verdana. It is a variable  
font because it can be stretched or compressed.  
<p>Sans-serif fonts do not have serif slabs; the
```

Visual browsers nearly always display paragraphs on new lines, with a bit of space between them by default (to use a term from CSS, they are displayed as a *block*). Paragraphs may contain text, images, and text-level elements (also called *phrasing content*), but they may *not* contain headings, lists, sectioning elements, or any elements that typically display as blocks by default.

Technically, it is OK to omit the closing `</p>` tag because it is not required in order for the document to be valid. A browser just assumes it is closed when it encounters the next element with a “block” display type.

Most web developers prefer to close paragraphs and all elements for the sake of consistency and clarity. I recommend folks who are just learning markup do the same.

NO NAKED TEXT!

It is strongly recommended that all text be enclosed in some sort of element. Text that is not contained within tags is called *naked* or *anonymous* text. Although the document will still be valid, it is a best practice not to leave content bare.

Headings

```
<h1>...</h1>
<h2>...</h2>
<h3>...</h3>
<h4>...</h4>
<h5>...</h5>
<h6>...</h6>
```

Heading elements

In the last chapter, we used the `h1` and `h2` elements to indicate headings for the Black Goose Bistro page. There are actually six levels of headings, from `h1` to `h6`. When you add headings to content, the browser uses them to create a *document outline* for the page. Assistive reading devices

such as screen readers use the document outline to help users quickly scan and navigate through a page. In addition, search engines look at heading levels as part of their algorithms, potentially giving information in higher heading levels more weight. For these reasons, it is a best practice to start with the level 1 heading (`h1`) and work down in numerical order, creating a logical document structure and outline.

This example shows the markup for four heading levels. Additional heading levels would be marked up in a similar manner:

```
<h1>Type Design</h1>

<h2>Serif Typefaces</h2>
<p>Serif typefaces have small slabs at the ends of the letters.</p>

<h3>Baskerville</h3>

<h4>Description</h4>
<p>Description of the Baskerville typeface.</p>

<h4>History</h4>
<p>The history of the Baskerville typeface.</p>

<h3>Georgia</h3>
<p>Description and history of the Georgia typeface.</p>

<h2>Sans-serif Typefaces</h2>
<p>Sans-serif typefaces do not have slabs at the ends of the letters.</p>
```

```
<p>Sans-serif typefaces do not have slabs at the
```

The markup in this example would create the following document outline:

1. Type Design
 1. Serif Typefaces
 - + text Paragraph
 - 1. Baskerville
 1. Description
 - + text paragraph
 2. History
 - + text paragraph
 - 2. Georgia
 - + text paragraph
 2. Sans-serif Typefaces
 - + text paragraph

By default (that is, using the browser's built-in style sheet), the headings in the example display in bold text, starting in very large type for **h1**s, with each consecutive level in smaller text, as shown in Figure 5-1. You can use a style sheet to change their appearance.

h1 — **Type Design**

h2 — **Serif Typefaces**

Serif typefaces have small slabs at the ends of letter strokes. In general, serif fonts can make large amounts of text easier to read.

h3 — **Baskerville**

h4 — **Description**

Description of the Baskerville typeface.

h4 — **History**

The history of the Baskerville typeface.

h3 — **Georgia**

Description and history of the Georgia typeface.

h2 — **Sans-serif Typefaces**

Sans-serif typefaces do not have slabs at the ends of strokes.

Figure 5-1. The default rendering of four heading levels

Thematic Breaks (Horizontal Rules)

```
<hr>
```

Thematic break (horizontal rule)

If you want to indicate that one topic has completed and another one is beginning, you can insert what the spec calls a “paragraph-level thematic break” with the `hr` element. The `hr` element adds a logical divider between sections of a page or paragraphs, without introducing a new heading level.

In older HTML versions, `hr` was defined as a “horizontal rule” because it inserts a horizontal line on the page. Browsers still render `hr` as a 3D shaded rule and put it on a line by itself with some space above and below by default, but in the current HTML spec, it has a semantic name and definition. If a decorative line is all you’re after, it is better to create a rule by specifying a colored border before or after an element with CSS.

`hr` is a void element, meaning it has no content—you just drop it into place where you want the thematic break to occur, as shown in this example and Figure 5-2:

```
<h3>Times</h3>
<p>Description and history of the Times typeface
<hr>
```

```
<h3>Georgia</h3>
<p>Description and history of the Georgia typeface</p>
```

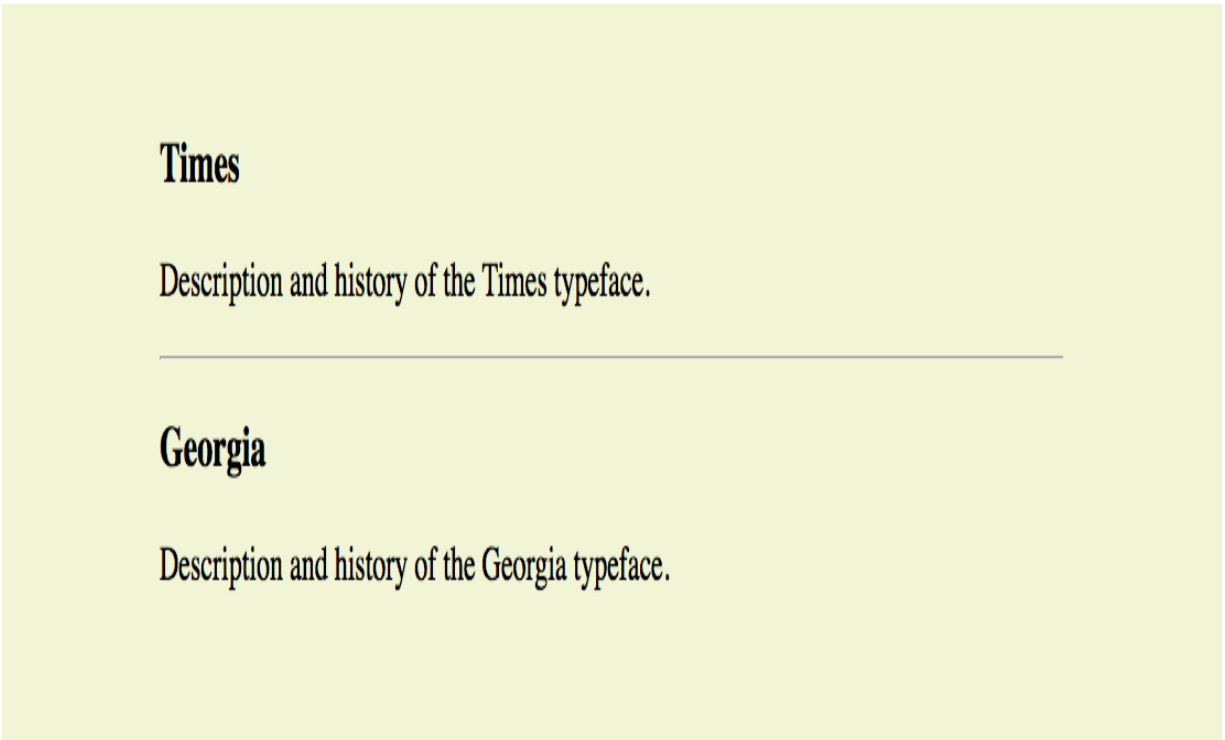


Figure 5-2. The default rendering of a thematic break (horizontal rule)

Lists

Humans are natural list makers, and HTML provides elements for marking up three types of lists:

Unordered lists

Collections of items that appear in no particular order

Ordered lists

Lists in which the sequence of the items is important

Description lists

Lists that consist of name and value pairs, including but not limited to terms and definitions

All list elements—the lists themselves and the items that go in them—are displayed as block elements by default, which means they start on a new line and have some space above and below, but that may be altered with CSS. In this section, we'll look at each list type in detail.

Unordered Lists

```
<ul>...</ul>
```

Unordered list

```
<li>...</li>
```

List item within an unordered list

Just about any list of examples, names, components, thoughts, or options qualifies as an unordered list. In fact, most lists fall into this category. By default, unordered lists display with a bullet before each list item, but you can change that with a style sheet, as you'll see in a moment.

TIP

Unordered lists are ideal for marking up links in navigation toolbars.

To identify an unordered list, mark it up as a `ul` element. The opening `<ul>` tag goes before the first list item, and the closing `</ul>` tag goes after the last item. Then, to mark up each item in the list as a list item (`li`), enclose it in opening and closing `<li>` tags, as shown in this example. Notice that there are no bullets in the source document; the browser adds them automatically (Figure 5-3):

```
<ul>
  <li>Serif</li>
  <li>Sans-serif</li>
  <li>Script</li>
  <li>Display</li>
  <li>Dingbats</li>
</ul>
```

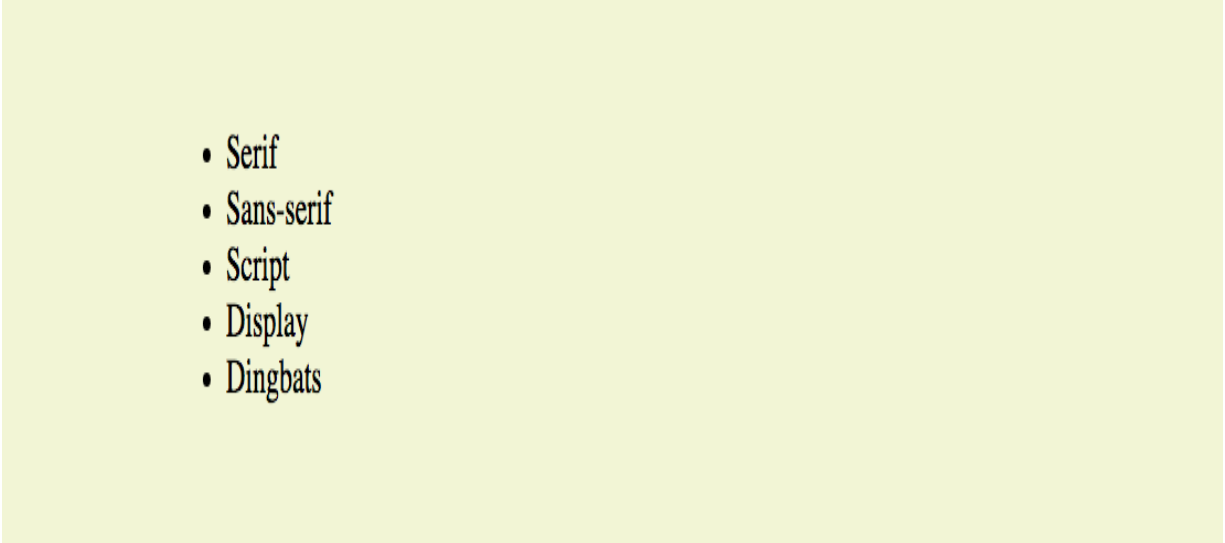

- 
- Serif
 - Sans-serif
 - Script
 - Display
 - Dingbats

Figure 5-3. The default rendering of the sample unordered list—the browser adds the bullets automatically

NOTE

The only thing that is permitted within an unordered list (that is, between the start and end `<ul>` tags) is one or more list items. You can't put other elements in there, and there may not be any untagged text. However, you can put any type of content element within a list item (`li`).

But here's the cool part. We can take that same unordered list markup and radically change its appearance by applying different style sheets, as shown in Figure 5-4. In the figure, I've turned off the bullets, added my own bullet icons, lined up the items horizontally, and even made them look like graphical buttons. The markup stays exactly the same.

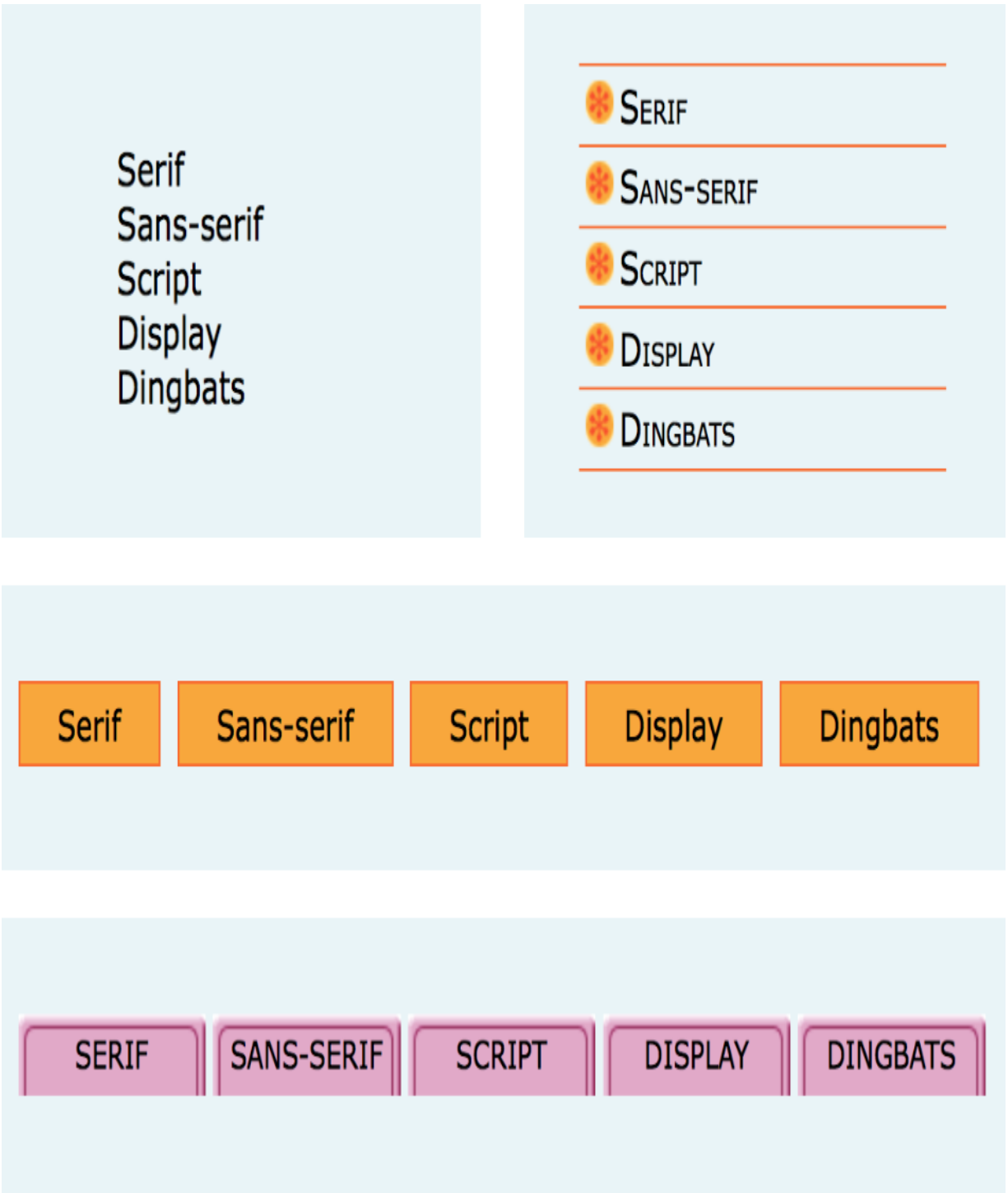


Figure 5-4. With style sheets, you can give the same unordered list many looks.

NESTING LISTS

Any list can be nested within another list; it just has to be placed within a list item. Nested lists are commonly used as the markup for hierarchical navigation links. This example shows the structure of an unordered list nested in the second item of an ordered list:

```
<ol>
  <li></li>
  <li>
    <ul>
      <li></li>
      <li></li>
      <li></li>
    </ul>
  </li>
</ol>
```

When you nest an unordered list within another unordered list, the browser automatically changes the bullet style for the second-level list.

Unfortunately, the numbering style is not changed by default when you nest ordered lists. You need to set the numbering styles yourself with CSS rules.

Ordered Lists

```
<ol>...</ol>
```

Ordered list

```
<li>...</li>
```

List item within an ordered list

Ordered lists are for items that occur in a particular order, such as step-by-step instructions or driving directions. They work just like the unordered lists described previously, but they are defined with the `ol` element (for “ordered list,” of course). Instead of bullets, the browser automatically inserts numbers before ordered list items (see Note), so you don’t need to number them in the source document. This makes it easy to rearrange list items without renumbering them.

NOTE

If something is logically an ordered list, but you don’t want numbers to display, remember that you can always remove the numbering with style sheets. So, go ahead and mark up the list semantically as an `ol` and adjust how it displays with a style rule.

Ordered list elements must contain one or more list item elements, as shown in this example and Figure 5-5:

```
<ol>
  <li>Gutenberg develops moveable type (1450s)</li>
  <li>Linotype is introduced (1890s)</li>
  <li>Photocomposition catches on (1950s)</li>
  <li>Type goes digital (1980s)</li>
```

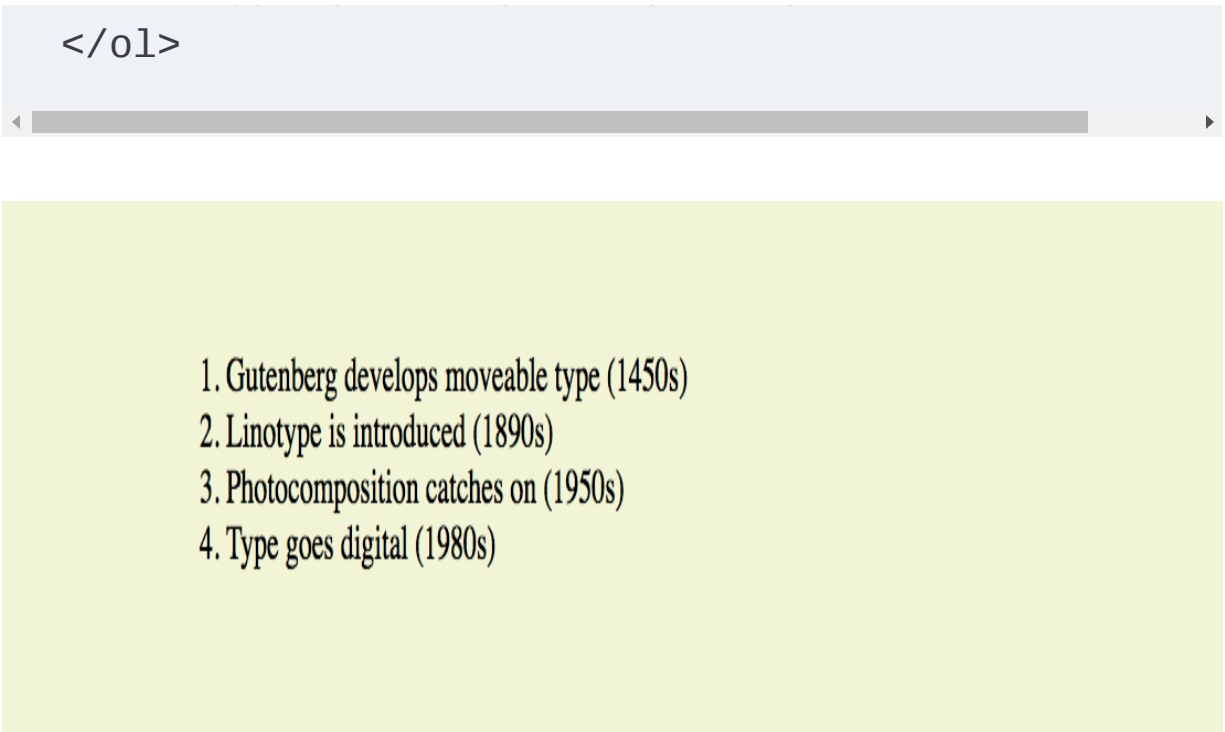
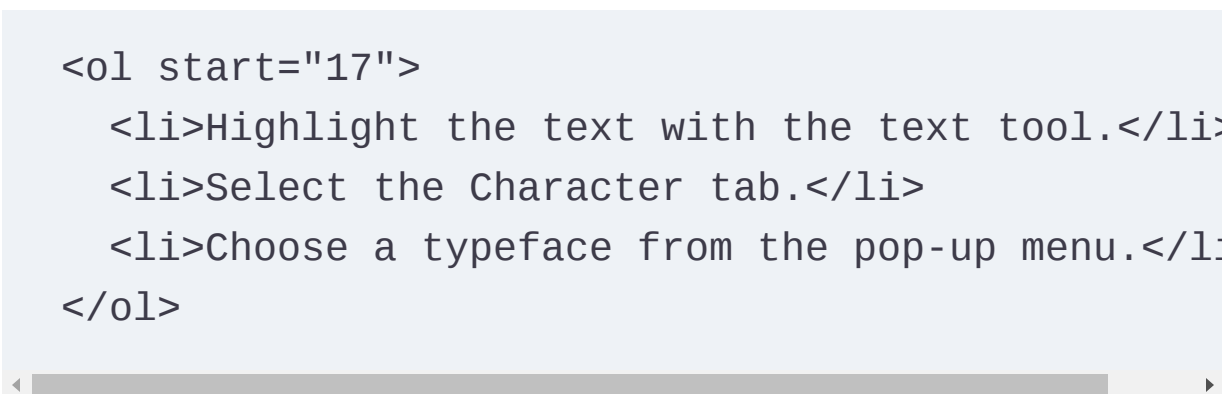


Figure 5-5. The default rendering of an ordered list—the browser adds the numbers automatically

If you want a numbered list to start at a number other than 1, you can use the `start` attribute in the `ol` element to specify another starting number, as shown here:



The resulting list items would be numbered 17, 18, and 19, consecutively.

CHANGING BULLETS AND NUMBERING

You can use the `list-style-type` style sheet property to change the bullets and numbers for lists. For example, for unordered lists, you can change the shape from the default dot to a square or an open circle, substitute your own image, or remove the bullet altogether. For ordered lists, you can change the numbers to Roman numerals (I, II, III or i, ii, iii), letters (A, B, C or a, b, c), and several other numbering schemes. In fact, as long as the list is marked up semantically, it doesn't need to display with bullets or numbering at all. Changing the style of lists with CSS is covered in Chapter 12, *Formatting Text*.

Description Lists

```
<dl>...</dl>
```

Description list

```
<dt>...</dt>
```

A name, such as a term or label

```
<dd>...</dd>
```

A value, such as a description or definition

Description lists are used for name/value pairs, such as terms and their definitions, questions and answers, or any other types of text and associated information. Their structure is a bit different from the other two lists that we

just discussed. The whole description list is marked up as a `dl` element. The content of a `dl` is some number of `dt` elements indicating the names and `dd` elements for their respective values. You might find it helpful to think of them as “terms” (to remember the “t” in `dt`) and “definitions” (for the “d” in `dd`), even though that is only one use of description lists.

Here is an example of a list that associates forms of typesetting with their descriptions (Figure 5-6):

```
<dl>
  <dt>Linotype</dt>
  <dd>Line-casting allowed type to be selected, u

  <dt>Photocomposition</dt>
  <dd>Typefaces are stored on film then projected

  <dt>Digital type</dt>
  <dd><p>Digital typefaces store the outline of t
    <p>Postscript emerged as a standard due to :
  </dd>
</dl>
```

Linotype

Line-casting allowed type to be selected, used, then recirculated into the machine automatically. This advance increased the speed of typesetting and printing dramatically.

Photocomposition

Typefaces are stored on film then projected onto photo-sensitive paper. Lenses adjust the size of the type.

Digital type

Digital typefaces store the outline of the font shape in a format such as Postscript. The outline may may be scaled to any size for output.

Postscript emerged as a standard due to its support of graphics and its early support on the Macintosh computer and Apple laser printer.

Figure 5-6. The default rendering of a definition list—definitions are set off from the terms by an indent

The `dl` element is allowed to contain only `dt` and `dd` elements or some number of `div`s (we'll get to the `div` element later in this chapter). You cannot put headings or content grouping elements (like paragraphs) in names (`dt`), but the values (`dd`) can contain any type of flow content. For example, the last `dd` element in the previous example contains two paragraph elements (the awkward default spacing could be cleaned up with

a style sheet). It is permitted to have multiple definitions with one term and vice versa.

More Content Elements

We’ve covered paragraphs, headings, and lists, but there are a few more special text elements to add to your HTML toolbox: long quotations (`blockquote`), preformatted text (`pre`), and figures (`figure` and `figcaption`). One thing these elements have in common is that they are categorized as “grouping elements” in the HTML spec, along with `p` , `hr` , the list elements, and `main` and the generic `div` , covered later in this chapter. The other thing they share is that browsers typically display them as block elements by default.

Long Quotations

```
<blockquote>...</blockquote>
```

Lengthy, block-level quotation

If you have a long quotation, a testimonial, or a section of copy from another source, mark it up as a `blockquote` element. It is recommended that content within `blockquote` elements be contained in other elements, such as paragraphs, headings, or lists, as shown in this example:

```
<p>Renowned type designer, Matthew Carter, has th
```

```
<blockquote>
  <p>Our alphabet hasn't changed in eons; there :
  <p>Much like a piece of classical music, the s
</blockquote>
```

NOTE

There is also a text-level element, `q`, for short quotations in the flow of text. I'll talk about that later in this chapter.

Figure 5-7 shows the default rendering of the `blockquote` example. This can be altered with CSS.

Renowned type designer, Matthew Carter, has this to say about his profession:

Our alphabet hasn't changed in eons; there isn't much latitude in what a designer can do with the individual letters.

Much like a piece of classical music, the score is written down. It's not something that is tampered with, and yet, each conductor interprets that score differently. There is tension in the interpretation.

Figure 5-7. The default rendering of a `blockquote` element

Preformatted Text

```
<pre>...</pre>
```

Preformatted text

In the previous chapter, you learned that browsers ignore whitespace such as line returns and character spaces in the source document. But in some types of information, such as code examples or certain poems, the whitespace is important for conveying meaning. For content in which whitespace is semantically significant, use the preformatted text (`pre`) element. This is a unique element, in that its content is displayed exactly as it is typed—including all the carriage returns and multiple character spaces.

By default, preformatted text is also displayed in a constant-width font (one in which all the characters are the same width, also called *monospace*), such as Courier; however, you can easily change the font with a style sheet rule.

NOTE

The `white-space:pre` CSS property can also be used to preserve spaces and line returns in the source if the effect is stylistic rather than semantically important.

The `pre` element in this example displays as shown in Figure 5-8. The second part of the figure shows the same content marked up as a paragraph (`p`) element for comparison:

The figure shows two code snippets side-by-side, illustrating the difference between the `pre` and `p` HTML elements. The top snippet uses `<pre>` and `</pre>` tags, resulting in text that is displayed in a monospace font with all spaces and line breaks preserved exactly as written in the source code. The bottom snippet uses `<p>` and `</p>` tags, resulting in text that is displayed in a standard serif font where spaces are normalized and line breaks are ignored, treating the content as a single paragraph.

```
<pre>
This is          an          example of
    text with a      lot of
                        curious
                        whitespace.
</pre>

<p>
This is          an          example of
    text with a      lot of
                        curious
                        whitespace.
</p>
```

```
This is          an          example of
      text with a      lot of
                           curious
                           whitespace.
```

This is an example of text with a lot of curious whitespace.

Figure 5-8. Preformatted text is unique in that the browser displays the whitespace exactly as it is typed into the source document. Compare it to the paragraph element, in which multiple line returns and character spaces are reduced to a single space.

Figures

```
<figure>...</figure>
```

Related image or resource

```
<figcaption>...</figcaption>
```

Text description of a figure

The `figure` element identifies content that illustrates or supports some point in the text. A figure may contain an image, a video, a code snippet, text, or even a table—pretty much anything that can go in the flow of web content. Content in a `figure` element should be treated and referenced as a self-contained unit. That means if a figure is removed from its original

placement in the main flow (to a sidebar or appendix, for example), both the figure and the main flow should continue to make sense.

Although you can simply add an image to a page, wrapping it in `<figure>` tags makes its purpose clear semantically. It also works as a hook for applying special styles to figures but not to other images on the page:

```
<figure>
  
```

If you want to provide a text caption for the figure, use the `figcaption` element above or below the content inside the `figure` element, as shown here—this is a more semantically rich way to mark up the caption than using a simple `p` element:

```
<figure>
  <pre>
    <code>
      body {
        background-color: #000;
        color: yellow;
      }
    </code>
```

```
</pre>  
<figcaption>Sample CSS rule.</figcaption>  
</figure>
```

In Exercise 5-1, you'll get a chance to mark up a document yourself and try out the basic text elements we've covered so far.

EXERCISE 5-1. MARKING UP A RECIPE

The owners of the Black Goose Bistro have decided to share recipes and news on their site. In the exercises in this chapter, we'll assist them with content markup.

In this exercise, you will start with the raw text of a recipe. It's up to you to decide which element is the best semantic match for each chunk of content. You'll use paragraphs, headings, lists, and at least one special content element.

You can write the tags right on this page. Or, if you want to use a text editor and see the results in a browser, this text file, as well as the final version with markup, is available at learningwebdesign.com/6e/materials.

Tapenade (Olive Spread)

This is a really simple dish to prepare and it's

"Make this the night before so that the flavors l

Ingredients

1 8oz. jar sundried tomatoes

2 large garlic cloves

2/3 c. kalamata olives

1 t. capers

Instructions

Combine tomatoes and garlic in a food processor.

Add capers and olives. Pulse the motor a few times.

Serve on thin toast rounds with goat cheese and tomatoes.

Organizing Page Content

So far, the elements we've covered handle very specific tidbits of content: a paragraph, a heading, a figure, and so on. Prior to HTML5, there was no way to group these bits into larger parts other than wrapping them in a generic division (`div`) element (we'll cover `div` in more detail later). HTML5 introduced new elements that give semantic meaning to sections of a typical web page or application (see Note), including main content (`main`), headers (`header`), footers (`footer`), sections (`section`), articles (`article`), navigation (`nav`), and tangentially related or complementary content (`aside`). These elements (with the exception of `main` , which is considered grouping content) are categorized as *sectioning elements* in the HTML spec. Curiously, the spec lists the old `address` element as a sectioning element as well, so we'll look at that one here too.

NOTE

The element names are based on a Google study in 2005 that looked at the top 20 names that developers assigned to generic division (div) elements.

Main Content

```
<main>...</main>
```

Primary content area of page or app

Web pages these days are loaded with different types of content: mastheads, sidebars, ads, footers, more ads, even more ads, and so on. Explicitly pointing out the primary content of a page or application with the `main` element helps screen readers and other assistive technologies know where the main content of the page begins and replaces the “Skip to main content” links that have been utilized in the past.

The content of a `main` element should be unique to that page. In other words, headers, sidebars, and other elements that appear across multiple pages in a site should not be included in the `main` section:

```
<body>
  <header>...</header>
  <main>
    <h1>Humanist Sans Serif</h1>
    <!-- code continues -->
```

```
</main>  
</body>
```

The HTML specification states that pages should have only one `main` section and that it should not be nested within an `article`, `aside`, `header`, `footer`, or `nav`. Doing so will cause the document to be invalid.

Headers and Footers

```
<header>...</header>
```

Introductory material for page, section, or article

```
<footer>...</footer>
```

Footer for page, section, or article

Because web authors had been labeling header and footer sections in their documents for years, it was obvious that the full-fledged `header` and `footer` elements added in HTML5 would come in handy. Let's start with headers.

Headers

The `header` element is used for introductory material that typically appears at the beginning of a web page or at the top of a section or article (we'll get to those elements soon). There is no specified list of what a

header must or should contain; anything that makes sense as the introduction to a page or section is acceptable. In the following example, the document header includes a logo image, the site title, and navigation:

```
<body>
  <header>
    
    <h1>Nuts about Web Fonts</h1>
    <nav>
      <ul>
        <li><a href="/index.html">Home</a></li>
        <li><a href="/blog.html">Blog</a></li>
        <li><a href="/shop.html">Shop</a></li>
      </ul>
    </nav>
  </header>

  <!-- page content -->

</body>
```

NOTE

The `<a href=""></a>` code in the examples is the markup for adding links to other web pages. We'll take on links in Chapter 6, Adding Links. The value is the URL for the page, as shown here, but I'll be using a simple slash as a space-saving measure going forward.

When used in an individual article, the **header** might include the article title, author, and publication date, as shown here:

```
<article>
  <header>
    <h1>More about WOFF</h1>
    <p>by Jennifer Robbins, <time datetime="2023-
  </header>
  <!-- article content here -->
</article>
```

NOTE

Neither header nor footer elements are permitted to contain nested header or footer elements.

Footers

The **footer** element is used to indicate the type of information that typically comes at the end of a page or an article, such as its author, copyright information, related documents, or navigation. The **footer** element may apply to the entire document, or it could be associated with a particular section or article. If the footer is contained directly within the **body** element, either before or after all the other **body** content, then it applies to the entire page or application. If it is contained in a sectioning element (**section** , **article** , **nav** , or **aside**), it is parsed as the

footer for just that section. Note that although it is called “footer,” there is no requirement that it appear last in the document or sectioning element. It could also appear at or near the beginning, if that makes sense.

In this simple example, we see the typical information listed at the bottom of an article marked up as a `footer` :

```
<article>
  <header>
    <h1>More about WOFF</h1>
    <p>by Jennifer Robbins, <time datetime="2024-01-01">2024-01-01
  </header>
  <!-- article content here -->
  <footer>
    <p><small>Copyright ©2024 Jennifer Robbins</small>
    <nav>
      <ul>
        <li><a href="/">Previous</a></li>
        <li><a href="/">Next</a></li>
      </ul>
    </nav>
  </footer>
</article>
```

NOTE

The time element will be discussed in the section “Dates and times” later in this chapter.

Sections and Articles

```
<section>...</section>
```

Thematic group of content

```
<article>...</article>
```

Self-contained, reusable composition

Long documents are easier to use when they are divided into smaller parts. For example, books are divided into chapters, and newspapers have sections for local news, sports, comics, and so on. To divide long web documents into thematic sections, use the aptly named `section` element. Sections typically include a heading (inside the `section` element) plus content that has a meaningful reason to be grouped together.

NOTE

The HTML spec recommends using the generic `div` element instead of `section` if the purpose for grouping the elements is simply to provide a hook for styling.

The `section` element has a broad range of uses, from dividing a whole page into major sections to identifying thematic sections within a single

article. A `section` should include a heading (`h1` – `h6`) that identifies the content, although it is not strictly required in order to be valid. In the following example, a document with information about typography resources has been divided into two sections based on resource type:

```
<section>
  <h2>Typography Books</h2>
  <ul>
    <li>...</li>
  </ul>
</section>

<section>
  <h2>Online Tutorials</h2>
  <p>These are the best tutorials on the web.</p>
  <ul>
    <li>...</li>
  </ul>
</section>
```

Use the `article` element for self-contained works that could stand alone or be reused in a different context (such as syndication). It is useful for magazine or newspaper articles, blog posts, comments, or other items that could be extracted for external use. You can think of it as a specialized `section` element that answers “yes” to the question “Could this appear on another site and make sense?”

A long **article** could be broken into a number of sections, as shown here:

```
<article>
  <h1>Get to Know Helvetica</h1>
  <section>
    <h2>History of Helvetica</h2>
    <p>...</p>
  </section>

  <section>
    <h2>Helvetica Today</h2>
    <p>...</p>
  </section>
</article>
```

Conversely, a **section** in a web document might be composed of a number of articles:

```
<section id="essays">
  <h1>For Further Reading</h1>
  <article>
    <h1>A Fresh Look at Futura</h1>
    <p>...</p>
  </article>
  <article>
    <h1>Getting Personal with Humanist</h1>
```

```
<p>...</p>  
</article>  
</section>
```

The `section` and `article` elements are easily confused, particularly because it is possible to nest one in the other and vice versa. Keep in mind that if the content is self-contained and could appear outside the current context, it is best marked up as an `article`.

Aside (Sidebars)

```
<aside>...</aside>
```

Tangentially related material

The `aside` element identifies content that is separate from, but tangentially related to, the surrounding content. In print, its equivalent is a sidebar, but the spec doesn't call it "sidebar" because putting something on the "side" is a presentational description, not semantic. Nonetheless, a sidebar is a good mental model for using the `aside` element. `aside` can be used for pull quotes, background information, lists of links, callouts, or anything else that might be associated with (but not critical to) a document.

In this example, an `aside` element is used for a list of links related to the main article:

```
<h1>Web Typography</h1>
<p>Back in 1997, there were competing font formats
<p>We now have a number of methods for using beautiful
<aside>
  <h2>Web Font Resources</h2>
  <ul>
    <li><a href="http://variablefonts.io">Variable Fonts</a>
    <li><a href="http://fonts.google.com">Google Fonts</a>
  </ul>
</aside>
```

The `aside` element has no default rendering, so you will need to set its display type to adjust its appearance and layout with style sheet rules.

Navigation

```
<nav>...</nav>
```

Primary navigation links

The `nav` element gives developers a semantic way to identify navigation for a site. Earlier in this chapter, we saw an unordered list that might be used as the top-level navigation for a font catalog site. Wrapping that list in a `nav` element describes its purpose accurately:

```
<nav>
  <ul>
    <li><a href="#">Home</a></li>
    <li><a href="#">About</a></li>
    <li><a href="#">Contact</a></li>
    <li><a href="#">Fonts</a></li>
    <li><a href="#">Tools</a></li>
    <li><a href="#">Blog</a></li>
  </ul>
```

```
<li><a href="/serif.html">Serif</a></li>
<li><a href="/sansserif.html">Sans-serif</a></li>
<li><a href="/script.html">Script</a></li>
<li><a href="/display.html">Display</a></li>
<li><a href="/dingbats.html">Dingbats</a></li>
</ul>
</nav>
```

Not all lists of links should be wrapped in a `nav` element, however. The spec makes it clear that `nav` should be used for links that provide primary navigation around a site or a lengthy section or article. The `nav` element may be especially helpful from an accessibility perspective.

Search-Related Content

```
<search>...</search>
```

Content related to searching or filtering

Just when you thought we had all the text elements we'd ever need, in 2023 the `search` grouping element was born! The `search` element is used to indicate a part of a page that contains form controls related to searching or filtering content. The searching functionality itself is handled by a `form` and its `input` elements, but marking up that form with the `search` element can make the search bar easier to find with assistive devices.

`search` is similar to `main` or `nav` in that its job is to add semantic meaning that can be helpful for accessibility. In fact, it was added to the spec in order to map to the “search” landmark role in ARIA (ARIA and roles are explained in the “Improving Accessibility with ARIA” section later in this chapter).

The `search` element has only been supported in browsers since 2023, but non-supporting browsers will just ignore it, so there is no harm adding it today.

Addresses

```
<address>...</address>
```

Contact information

Last and, well, least is the `address` element that is used to create an area for contact information for the author or maintainer of the document. It is generally placed at the end of the document or in a section or article within a document. An `address` would be right at home in a `footer` element. It is important to note that the `address` element should *not* be used for any other type of address on a page, such as a business’s mailing address. It is intended specifically for author contact information (although that could potentially be a mailing address). Following is an example of its intended use:

```
<address>
```

```
<p>Contributed by <a href=" ../authors/robbins/">.  
<a href="http://www.oreilly.com/">O'Reilly Media  
</address>
```

The Text-Level Element Roundup

Now that we've identified the larger chunks of content, we can provide semantic meaning to phrases within the chunks by using what the HTML specification calls *text-level semantic elements*. You may also hear them called *inline elements*, as that was the term used in earlier HTML specifications (because they display in the flow of text by default and do not cause any line breaks).

Text-Level Elements

Despite all the types of information you could add to a document, there are only a couple of dozen text-level semantic elements. [Table 5-1](#) lists all of them.

THE TEXT-LEVEL ELEMENTS BACKSTORY

Many of the text-level elements that have been around since the dawn of the web were introduced to change the visual formatting of text selections because of the lack of a style sheet system. If you wanted bolded text, you marked it as `b`. Italics? Use the `i` element. In fact, there was once a `font` element used solely to change the font, color, and size of text (the horror!). Not surprisingly, HTML5 kicked the purely presentational `font` element to the curb. However, many of the old-school presentational text-level elements (for example, `u` for underline and `s` for strikethrough) were kept and given new semantic definitions (`b` is now for “added visual attention,” `s` for “inaccurate text”).

Many text-level elements have the expected style rendering (bold for the `b` element, for example). Others are purely semantic (such as `abbr` or `time`) and don’t have default renderings. For any text-level elements, you can use CSS rules if you want to change the way they display.

HTML5 also declared the following text-level elements obsolete: `acronym`, `applet`, `basefont`, `big`, `center`, `dir` (directory), `font`, `isindex` (search box), `strike`, and `tt` (teletype). I mention them here in case you run across them when viewing the source of an old document or if you are using an older web authoring tool. You should not use them today.

Table 5-1. Text-level semantic elements

Element	Description
a	An anchor or hypertext link (see Chapter 6 for details)
abbr	Abbreviation
b	Added visual attention, such as keywords (bold)
bdi	Indicates text that may have directional requirements
bdo	Bidirectional override; explicitly indicates text direction (left to right, ltr, or right to left, rtl)
br	Line break
cite	Citation; a reference to the title of a work, such as a book title
code	Computer code sample

Element	Description
data	Machine-readable equivalent dates, time, weights, and other measurable values
del	Deleted text; indicates an edit made to a document
dfn	The defining instance or first occurrence of a term
em	Emphasized text
i	Alternative voice (italic) or alternate language
ins	Inserted text; indicates an insertion in a document
kbd	Keyboard; text entered by a user (for technical documents)
mark	Contextually relevant text

Element	Description
menu	Represents a toolbar consisting of its contents, in the form of an unordered list
q	Short, text-level quotation
ruby, rt, rp	Provides annotations or pronunciation guides under East Asian typography and ideographs
s	Incorrect text (strike-through)
samp	Sample output from programs
small	Small print, such as a copyright or legal notice (usually displayed in a smaller type size by default)
span	Generic phrase content
strong	Content of strong importance
sub	Subscript

Element	Description
sup	Superscript
time	Machine-readable time data
u	Indicates a formal name, misspelled word, or text that would be underlined
var	A variable or program argument (for technical documents)
wbr	Word break

Although it may be useful having all of the text-level elements listed together in a table, they certainly deserve more detailed explanations. Let's take a closer look at each of these elements and what they're used for.

Emphasized text

```
<em>...</em>
```

Stressed emphasis

Use the `em` element to indicate which part of a sentence should be stressed or emphasized. The placement of `em` elements affects how a sentence's meaning is interpreted. Consider the following sentences, which are identical except for which words are stressed:

```
<p><em>Arlo</em> is very smart.</p>  
<p>Arlo is <em>very</em> smart.</p>
```

The first sentence indicates *who* is very smart. The second example is about *how* smart he is. Notice that the `em` element has an effect on the meaning of the sentence.

Emphasized text (`em`) elements nearly always display in italics by default (Figure 5-9), but of course you can make them display any way you like with a style sheet. Screen readers may use a different tone of voice to convey stressed content, which is why you should use an `em` element only when it makes sense semantically, not just to achieve italic text.

Important text

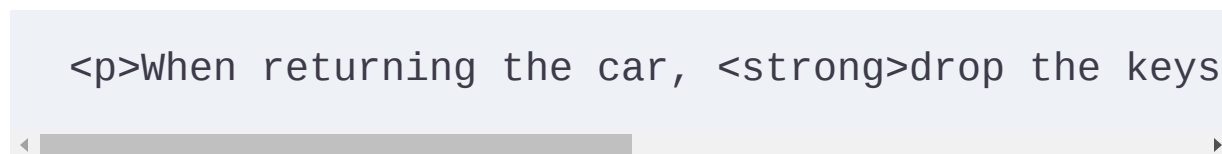
```
<strong>...</strong>
```

Strong importance

The `strong` element indicates that a word or phrase is important, serious, or urgent. In the following example, the `strong` element identifies the

portion of the instructions that requires extra attention. The **strong** element does not change the meaning of the sentence; it merely draws attention to the important parts:

```
<p>When returning the car, <strong>drop the keys
```



Visual browsers typically display **strong** text elements in bold text by default (Figure 5-9). Screen readers may use a distinct tone of voice for important content, so mark text as **strong** only when it makes sense semantically, not just to make it bold.

Arlo is very smart.

Arlo is *very* smart.

When returning the car, **drop the keys in the red box by the front desk.**

Figure 5-9. The default rendering of emphasized and strong text

Elements originally named for their presentational properties

```
<b>...</b>
```

Keywords or visually emphasized text (bold)

```
<i>...</i>
```

Alternative voice (italic)

```
<s>...</s>
```

Incorrect text (strike-through)

```
<u>...</u>
```

Annotated text (underline)

```
<small>...</small>
```

Legal text; small print (smaller type size)

As long as we're talking about bold and italic text, let's see what the old `b` and `i` elements are up to now. The elements `b`, `i`, `u`, `s`, and `small` were introduced in the old days of the web as a way to provide typesetting instructions (bold, italic, underline, strike-through, and smaller text, respectively). Despite their original presentational purposes, these elements are still around in the current HTML spec, with updated, semantic definitions. Browsers still render them by default as you'd expect (Figure 5-10 shows how the examples given here are typically displayed). However, if a type style change is all you're after, using a style sheet rule is the appropriate solution. Save these for when they are semantically appropriate.

Let's look at these elements and their correct usage, as well as the style sheet alternatives.

b

Indicates keywords, product names, or other phrases that need to stand out from the surrounding text without conveying added importance or emphasis (see Note). [*Old definition:* Bold.]

CSS property: For bold text, use `font-weight` (example: `font-weight: bold;`)

Example: `<p>The slabs at the ends of letter strokes are called`
`<b>serifs</b>.</p>`

NOTE

It helps me to think about how a screen reader would read the text. If I don't want the word read in a loud, emphatic tone of voice, but it really should be bold, then **b** may be more appropriate than **strong**.

i

Indicates text that is in a different voice or mood than the surrounding text, such as a phrase from another language, a technical term, or a thought. [*Old definition:* Italic.]

CSS property: For italic text, use `font-style` (example: `font-style: italic;`)

Example: `<p>Simply change the font and <i>Voila!</i>, a new personality!`
`</p>`

s

Indicates text that is incorrect. [*Old definition:* Strike-through text.]

CSS property: To draw a line through a selection of text, use `text-decoration` (example: `text-decoration: line-through`)

Example: `<p>Scala Sans was designed by <s>Eric Gill</s> Martin Majoor.
</p>`

u

Indicates text that should have some sort of non-text annotation supplied (such as an underline). There are a few instances when underlining has semantic significance, such as underlining a formal name in Chinese or indicating a misspelled word after a spell check, such as the misspelled “Helvetica” in the following example. Note that underlined text is easily confused with a link and should generally be avoided, except for a few niche cases. [*Old definition:* Underline.]

CSS property: For underlined text, use `text-decoration` (example: `text-decoration: underline`)

Example: `<p>New York subway signage is set in <u>Helviteca</u>.</p>`

small

Indicates an addendum or side note to the main text, such as the legal “small print” at the bottom of a document. [*Old definition:* Renders in font smaller than the surrounding text.]

CSS property: To make text smaller, use `font-size` (example: `font-size: 80%`)

Example: `<p><small>(This font is free for personal and commercial use.)`
`</small></p>`

b — The slabs at the ends of letter strokes are called **serifs**.

i — Simply change the font and *Voila!*, a new personality!

s — Scala Sans was designed by ~~Eric Gill~~ Martin Majoor.

u — New York subway signage is set in Helviteca.

small — (This font is free for personal and commercial use.)

Figure 5-10. Figure 5-10. The default rendering of b, i, s, u, and small elements

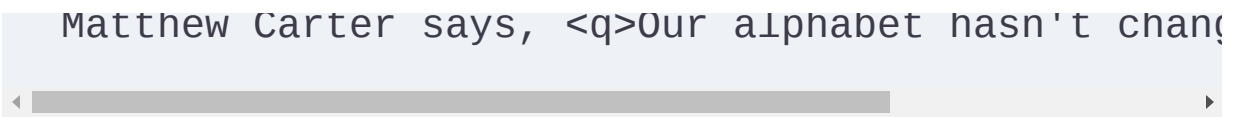
Short quotations

`<q>...</q>`

Short text-level quotation

Use the quotation (`q`) element to mark up short quotations, such as “To be or not to be,” in the flow of text, as shown in this example (Figure 5-11):

.....

A screenshot of a web browser's source code or developer tool. It shows a line of HTML code: `Matthew Carter says, <q>Our alphabet hasn't changed in eons.</q>`. Below the code is a horizontal scrollbar, indicating the text is part of a larger document.

According to the HTML spec, browsers should add quotation marks around `q` elements automatically, so you don't need to include them in the source document. Most major browsers—like Firefox, Chrome and Microsoft Edge—render curly quotes, which is preferable. Safari (which I used for this example) renders them as straight quotes, as shown in Figure 5-11.

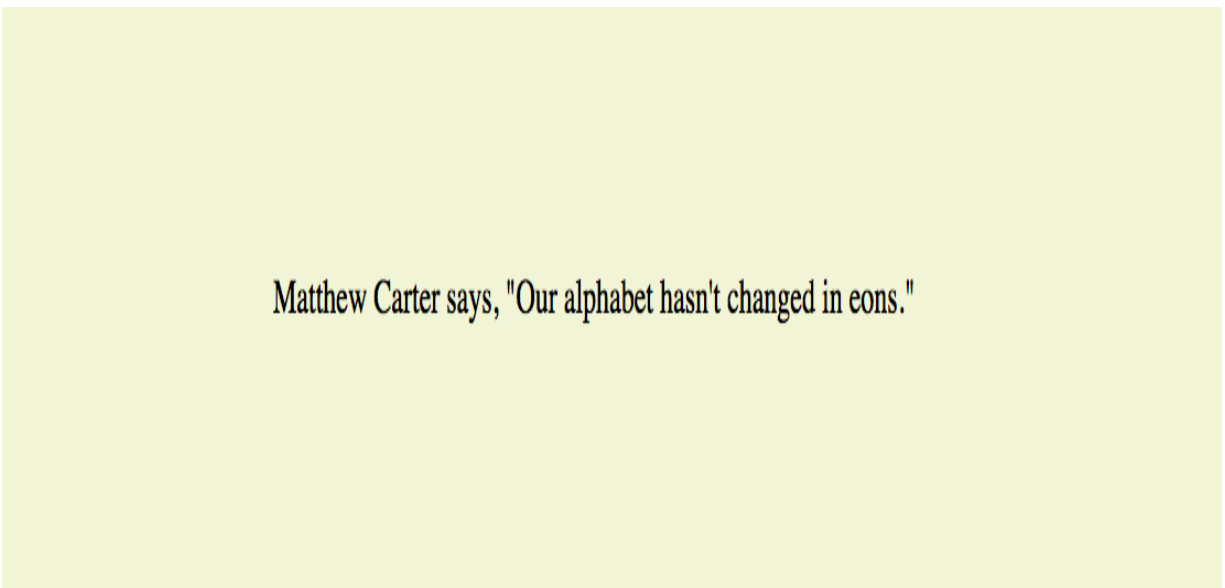


Figure 5-11. Browsers add quotation marks automatically around `q` elements.

The `q` element has an optional `cite` attribute for listing a URL for the source of the quote. The spec makes it clear that marking up short quotations with `q` is optional; they can also be conveyed by typing quotation marks right in the text.

Abbreviations and acronyms

```
<abbr>...</abbr>
```

Abbreviation or acronym

Marking up acronyms and abbreviations with the `abbr` element provides useful information for search engines, screen readers, and other devices.

Abbreviations are shortened versions of a word ending in a period (“Conn.” for “Connecticut,” for example). Acronyms are abbreviations generally formed by the first letters of the words in a phrase (such as NASA or USA). The `title` attribute provides the long version of the shortened term, as shown in this example:

```
<abbr title="Points">pts.</abbr>  
<abbr title="American Type Founders">ATF</abbr>
```

NESTING ELEMENTS

You can apply two elements to a string of text (for example, a phrase that is both a quote and in another language), but be sure they are nested properly. That means the inner element, including its closing tag, must be completely contained within the outer element:

```
<q><i>Je ne sais pas.</i></q>
```

Here is an example of elements that are nested incorrectly. Notice that the inner `i` element is not closed within the containing `q` element:

```
<q><i>Je ne sais pas.</q></i>
```

It's easy to spot the nesting error in an example that is this short, but when you're nesting long passages or nesting multiple levels deep, it's easy to end up with overlaps. One advantage to using an HTML code editor is that it can automatically close elements for you correctly, or point out when you've made a mistake.

Citations

```
<cite>...</cite>
```

Citation

The `cite` element is used to identify a reference to another document, such as a book, magazine, article title, and so on. Citations are typically rendered in italic text by default. Here's an example:

```
<p>Passages of this article were inspired by <cite>
```

Defining terms

```
<dfn>...</dfn>
```

Defining term

It is common to point out the first and defining instance of a word in a document in some fashion. In this book, defining terms are set in blue text. In HTML, you can identify them with the `dfn` element, as shown here, and format them visually using style sheets:

```
<p><dfn>Script typefaces</dfn> are based on handw
```

Program code elements

```
<code>...</code>
```

Code

```
<var>...</var>
```

Variable

```
<math>...</math>
```

Program sample

```
<code>...</code>
```

User-entered keyboard strokes

A number of text-level elements are used for describing the parts of technical documents, such as code (`code`), variables (var), program samples (`samp`), and user-entered keyboard strokes (`kbd`). It's a quaint reminder of HTML's origins in the scientific world (Tim Berners-Lee developed HTML to share documents at the CERN particle physics lab in 1989).

Code, sample, and keyboard elements typically render in a constant-width (also called *monospace*) font such as Courier by default. Variables usually render in italics.

Subscript and superscript

```
<sub>...</sub>
```

Subscript

```
<sup>...</sup>
```

Superscript

The subscript (`sub`) and superscript (`sup`) elements cause the selected text to display in a smaller size, positioned slightly below (`sub`) or above (`sup`) the baseline. These elements may be helpful for indicating chemical formulas or mathematical equations.

Figure 5-12 shows how these examples of subscript and superscript typically render in a browser:

```
<p>H<sub>2</sub>O</p>
```

```
<p>E=MC<sup>2</sup></p>
```

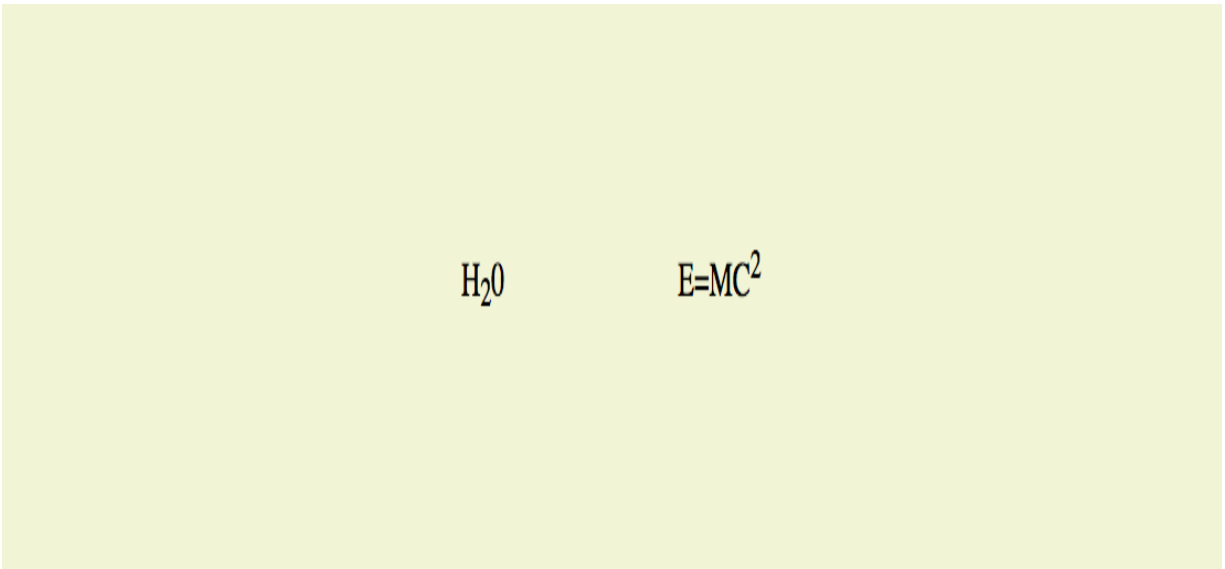


Figure 5-12. Subscript and superscript

Highlighted text

```
<mark>...</mark>
```


Contextually relevant text

The `mark` element indicates a word that may be considered especially relevant to the reader. One might use it to dynamically highlight a search term in a page of results, to manually call attention to a passage of text, or to indicate the current page in a series. Some designers (and browsers) give marked text a light-colored background, as though it were marked with a highlighter marker, as shown in Figure 5-13:

```
<p> ... PART I. ADMINISTRATION OF THE GOVERNMENT. TITLE IX.  
TAXATION. CHAPTER 65C. MASS. <mark>ESTATE TAX</mark>
```

... PART I. ADMINISTRATION OF THE GOVERNMENT. TITLE IX.
TAXATION. CHAPTER 65C. MASS. ESTATE TAX. Chapter 65C: Sect. 2.
Computation of estate tax.

Figure 5-13. In this example, search terms are identified with mark elements and given a yellow background with a style sheet so they are easier for the reader to find.

Dates and times

```
<time>...</time>
```

Time data

When we look at the phrase “noon on November 4,” we know that it is a date and a time. But the context might not be so obvious to a computer program. The `time` element allows us to mark up dates and times in a way that is comfortable for a human to read, but also encoded in a standardized way that computers can use (see Note). The content of the element presents the information to people, and the `datetime` attribute presents the same information in a machine-readable way.

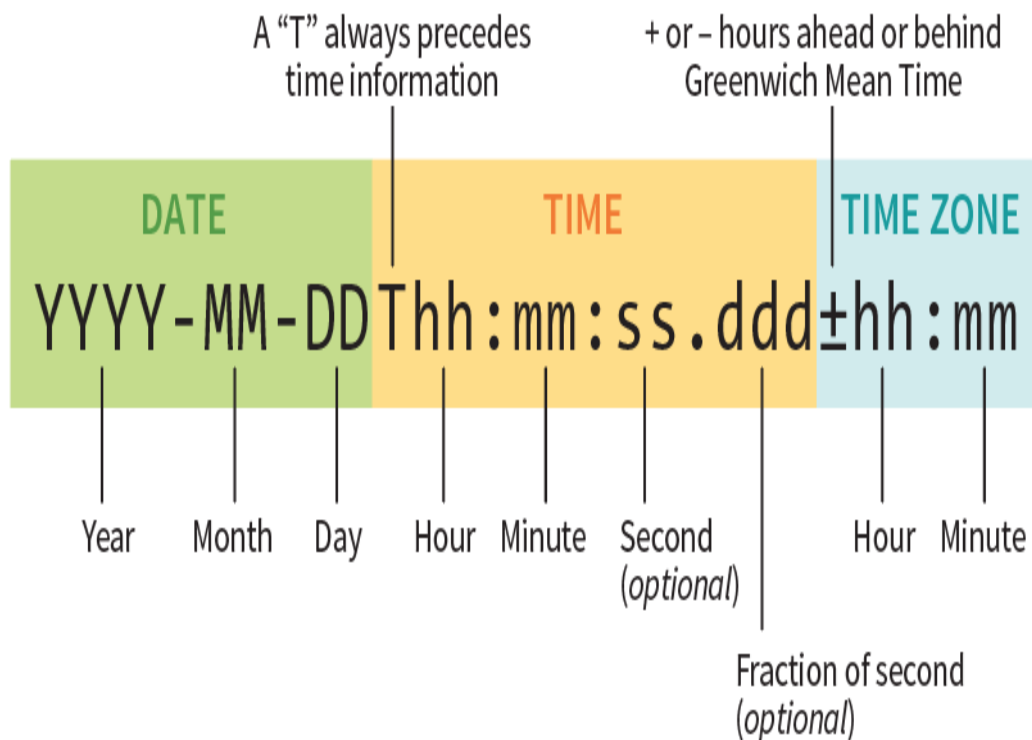
NOTE

The time element is not intended for marking up times for which a precise time or date cannot be established, such as “the end of last year” or “the turn of the century.”

The `time` element indicates dates, times, or date/time combos. It might be used, for example, to pass the date and time information to an application (such as for saving an event to a personal calendar), or by search engines to find the most recently published articles, or to restyle time information into an alternate format (e.g., changing 18:00 to 6 p.m.).

The `datetime` attribute specifies the date and/or time information in a standardized time format illustrated in Figure 5-14. The full time format

begins with the date (year–month–day). The time section begins with a letter “T” and lists hours (on the 24-hour clock), minutes, seconds (optional), and milliseconds (also optional). Finally, the time zone is indicated by the number of hours behind (–) or ahead of (+) Greenwich Mean Time (GMT). For example, “–05:00” indicates the Eastern Standard time zone, which is five hours behind GMT. When identifying dates and times alone, you can omit the other sections.



Example:

3pm PST on November 25, 2023

`2023-11-25T15:00-8:00`

Figure 5-14. Standardized date and time syntax

Here are a few examples of valid values for `datetime`:

- Time only: 9:30 p.m.

```
<time datetime="21:30">9:30 p.m.</time>
```

- Date only: June 19, 2016

```
<time datetime="2016-06-19">June 19, 2016</time>
```

- Date and time: Sept. 5, 1970, 1:11 a.m.

```
<time datetime="1970-09-05T01:11:00">Sept. 5, 1970, 1:11 a.m.</time>
```

- Date and time, with time zone information: 8:00 a.m. on July 19, 2023, in Providence, RI

```
<time datetime="2023-07-19T08:00:00-05:00">July 19, 2023, 8:00 a.m. in Providence, RI</time>
```

NOTE

You can also use the time element without the datetime attribute, but its content must be a valid date/time string:

```
<time>2016-06-19</time>
```

FURTHER READING

For more information on the intricate ins and outs of specifying dates and times, with examples, check out the time element entry in the HTML specification (html.spec.whatwg.org/multipage/text-level-semantics.html#the-time-element).

Machine-readable information

```
<data>...</data>
```

Machine-readable data

The `data` element is another tool for helping computers make sense of content. It can be used for all sorts of data, including dates, times, measurements, weights, microdata, and so on. The required `value` attribute provides the machine-readable information. Here are a couple of examples:

```
<data value="12">Twelve</data>
<data value="978-1-449-39319-9">CSS: The Definit
```



I'm not going to go into more detail on the `data` element, because as a beginner, you are unlikely to be dealing with machine-readable data quite yet. But it is interesting to see how markup can be used to provide usable information to computer programs and scripts as well as to your fellow humans.

Inserted and deleted text

```
<ins>...</ins>
```

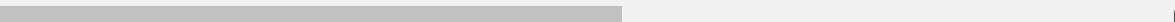
Inserted text

```
<del>...</del>
```

Deleted text

The `ins` and `del` elements are used to mark up edits indicating parts of a document that have been inserted or deleted (respectively):

```
Chief Executive Officer: <del title="retired">Pe
```



These elements rely on style rules for presentation (i.e., there is no dependable browser default). Both the `ins` and `del` elements can contain either text-level or grouping elements, depending on what type of content they contain.

Adding Breaks

Line breaks

```
<br>
```

Line break

Occasionally, you may need to add a line break within the flow of text.

We’ve seen how browsers ignore line breaks in the source document, so we need a specific directive to tell the browser to “add a line break here.”

The line break element (`br`) does exactly that. It can be used, for example, to break up lines of addresses or poetry. `br` is a void element, which means it does not have content. Add the element in the flow of text where you want a break to occur, as shown here and in Figure 5-15:

```
<p>So much depends <br>upon <br><br>a red wheel <
```

So much depends
upon

a red wheel
barrow

Figure 5-15. Line breaks are inserted at each `br` element (example extracted from “The Red Wheelbarrow” by William Carlos Williams)

Unfortunately, the `br` element is easily abused. Be careful that you aren’t using `br` elements to force breaks into text that really ought to be a list. For example, don’t do this:

```
<p>Times<br>  
Georgia<br>  
Garamond  
</p>
```

If it’s a list, use the semantically correct unordered list element instead, and turn off the bullets with style sheets:


```
<ul>
  <li>Times</li>
  <li>Georgia</li>
  <li>Garamond</li>
</ul>
```

Word breaks

```
<wbr>
```

Word break

The word break (`wbr`) element lets you mark the place where a word should break (a “line break opportunity,” according to the spec) should there not be enough room for the whole word (Figure 5-16). It takes some of the guesswork away from the browser and allows authors to control the best spot for the word to be split over two lines (without `wbr` , the word stays together, and if there is not enough room, the whole word wraps to the next line). If there is enough room, the word stays in one piece. Note that the browser does not add a hyphen when the word breaks over two lines (see Note). The `wbr` behaves as though it were a character space in the middle of the word:

```
<p>The biggest word you've ever heard and this is
<em>supercali<wbr>fragilistic<wbr>expialidocious</em>
```

The biggest word you've ever heard and this is how it goes: *supercalifragilistic
expialidocious!*

Figure 5-16. When there is not enough room for a word to fit on a line, it will break at the location of a wbr element.

NOTE

If you'd like a hyphen to appear when the word breaks onto two lines, you can insert a soft hyphen character (­) at the spot you'd like the word to break. If the word stays on one line, the soft hyphen stays hidden, but if the word breaks, a hyphen character displays at the end of the line. Character entities like ­ are discussed at the end of this chapter.

You've been introduced to 32 new elements since your last exercise. I say it's time to give some of the text-level elements a try in Exercise 5-2.

ACCOMMODATING NON-WESTERN LANGUAGES

If the web is to reach a truly worldwide audience, it needs to be able to support the display of all the languages of the world, with all their unique alphabets, symbols, directionality, and specialized punctuation. The W3C's efforts for internationalization (often referred to as "i18n," because it starts with an i, ends with an n, and has 18 letters in between) ensure that the formats and protocols defined in web technologies are usable worldwide.

Internationalization efforts include the following:

- Using the Unicode character encoding, which contains the characters, glyphs, symbols, ideographs, and the like from all active, modern languages.
- Declaring the primary language of a document by using a two-letter language code from the ISO 639-1 standard (available at www.loc.gov/standards/iso639-2/php/code_list.php). For example, English is "EN," Czech is "CS," and German is "DE." Use the lang attribute in the html element to declare the language for the whole document, or in individual elements that require clarification.
- Accommodating the various writing directions of languages. In HTML, the dir attribute explicitly sets the direction for the document or an element to ltr (left to right) or rtl (right to left). On phrase-level elements, it also creates a bidirectional isolation, preventing text within the element from influencing the ordering of text outside it. (This can be an important consideration when you are embedding user-generated text.)

For example, to include a passage of Hebrew in an English document, use the `dir` attribute to indicate that the phrase should be displayed right to left:

```
<p>This is how you write Shalom: <span dir="rtl">שלום</span></p>
```

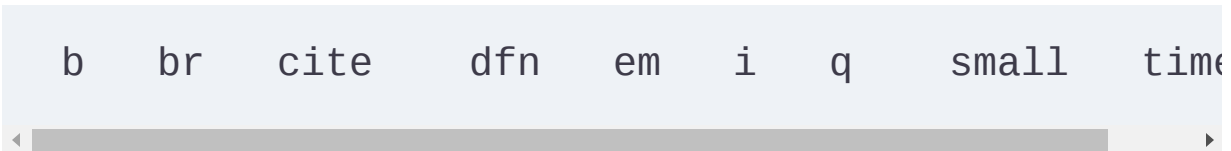
- Providing a system that allows for ruby annotation, notes that typically appear above ideographs from East Asian languages to give pronunciation clues or translations (ruby, rt, and rp elements). See the spec for details if this is something you need to do.
- Optionally providing alternative links to translated versions of your page, using the link element with the hreflang attribute in the head of the document. For example, this link element points to a separate version of your page, translated into German:

```
<link rel="alternate" title="Das Seite" href="http://www.w3.org/International/techniques/authoring-german.html">
```

The W3C internationalization techniques page (www.w3.org/International/techniques/authoring-html) provides a thorough collection of HTML and CSS authoring techniques and resources to help with your internationalization efforts.

EXERCISE 5-2. IDENTIFYING TEXT-LEVEL ELEMENTS

This little post for the Black Goose Bistro News page will give you an opportunity to identify and mark up a variety of text-level elements. See if you can find phrases to mark up accurately with the following elements:



Because markup is always somewhat subjective, your resulting markup may not look exactly like my final markup, but there is an opportunity to use all of the preceding elements in the article. For extra credit, there is a phrase that could have two elements applied to it. (Hint: look for a term in another language.) Remember to nest them properly by closing the inner element before you close the outer one.

You can write the tags right on this page. Or, if you want to use a text editor and see the results in a browser, this text file is available online at learningwebdesign.com/6e/materials along with the resulting code:

```
<article>

<header>
<p>posted by BGB, November 15, 2024</p>
</header>

<h2>Low and Slow</h2>
```

```
<p>This week I am extremely excited about a new c
<p>Next month, we will be serving Sous Vide Salmon

<p>blackgoose@example.com
555-336-1800</p>

<p>Warning: Sous vide cooked salmon is not pasteurized
</article>
```

Generic Elements (div and span)

```
<div>...</div>
```

Generic block-level element

```
<span>...</span>
```

Generic text-level element

What if none of the elements we've talked about so far accurately describes your content? After all, there are endless types of information in the world but, as you've seen, not all that many semantic elements. Fortunately, HTML provides two generic elements that can be customized to describe your content perfectly. The `div` element indicates a division of content, and `span` indicates a word or phrase for which no text-level element

currently exists. The generic elements are given meaning and context with the `id` and `class` attributes, which we'll discuss in a moment.

The `div` and `span` elements have no inherent presentation qualities of their own, but you can use style sheets to format them however you like. In fact, generic elements are a primary tool in standards-based web design because they enable authors to accurately describe content and offer plenty of “hooks” for adding style rules. They also allow elements on the page to be accessed and manipulated by JavaScript.

We're going to spend a little time on `div` and `span` elements, as well as the `id` and `class` attributes, to learn how authors use them to structure content.

Divide It Up with a `div`

The `div` element is used to create a logical grouping of content or elements on the page. It indicates that its content belongs together in a conceptual unit or should be treated as a unit by CSS or JavaScript. By marking related content as a `div` and giving it a unique `id` or indicating that it is part of a `class`, you give context to the elements in the grouping. The use of `div`s to structure content for styling, layout, and scripting is a key feature of modern web development, so chances are you'll be using a lot of them.

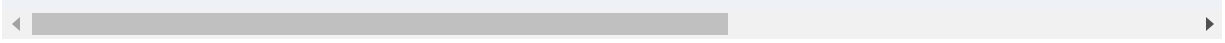
Let's look at a few examples of `div` elements.

TIP

It is possible to nest `div` elements within other `div` elements, but don't go overboard. Always strive to keep your markup as simple as possible, so add a `div` element only if it is necessary for logical structure, styling, or scripting. You want to avoid what developers call “divitis” or “div soup.”

In this example, a `div` element is used as a container to group an image and two paragraphs into a product “listing”:

```
<div class="listing">
  
  <p><cite>The Complete Manual of Typography</cite>
  <p>A combination of type history and examples of
</div>
```



By putting those elements in a `div`, we've made it clear that they are conceptually related. It also allows us to style `p` elements within listings differently than other `p` elements in the document.

Another common use of a `div` is to break a page into sections for layout purposes. In this example, a heading and several paragraphs are enclosed in a `div` and identified as the “news” division:

```
<div id="news">
  <h1>New This Week</h1>
  <p>We've been working on...</p>
```



```
<p>And last but not least, ...</p>
</div>
```

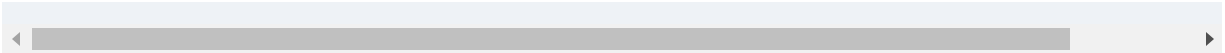
Now we have a custom element that we've given the name “news.” You might be thinking, “Hey, couldn't you use a `section` element for that?” You could! In fact, authors may turn to generic `div`s less often now that we have better semantic sectioning elements.

Define a Phrase with `span`

A `span` offers the same benefits as the `div` element, except it is used for phrase elements and does not introduce line breaks. Because `span`s are text-level elements, they may contain only text and other text-level elements (in other words, you cannot put headings, lists, content grouping elements, and so on in a `span`). Let's get right to some examples.

There is no `telephone` element, but we can use a `span` to give meaning to telephone numbers. In this example, each telephone number is marked up as a `span` and classified as “tel”:

```
<ul>
  <li>John: <span class="tel">999.8282</span></li>
  <li>Paul: <span class="tel">888.4889</span></li>
  <li>George: <span class="tel">888.1628</span></li>
  <li>Ringo: <span class="tel">999.3220</span></li>
</ul>
```



You can see how the classified `span`s add meaning to what otherwise might be a random string of digits. As a bonus, the `span` element enables us to apply the same style to phone numbers throughout the site (for example, ensuring line breaks never happen within them by using a CSS `white-space: nowrap` declaration).

id and class Attributes

In the previous examples, we used the `id` and `class` attributes to provide context to generic `div` and `span` elements. `id` and `class` have different purposes, however, and it's important to know the difference.

Identification with id

The `id` attribute is used to assign a *unique* identifier to an element in the document. In other words, the value of `id` must be used only once in the document. This makes it useful for assigning a name to a particular element, as though it were a piece of data. See the sidebar “id and class Values” for information on providing values for the `id` attribute.

ID AND CLASS VALUES

The values for id and class attributes must contain at least one character (that is, they may not be empty), and id values may not contain any character spaces. You can use pretty much any character in the value.

Earlier versions of HTML had restrictions on id values (for example, they needed to start with a letter), but those restrictions were removed in HTML5.

In this example, each book listing `div` is identified by the book's unique ISBN number (see Note) as the value of the `id` attribute. The values could also have been something simple (`product1` , `product2` , for example), as long as no two `id` s in the document had the same value.

NOTE

An ISBN (International Standard Book Number) is a unique number assigned to a published book or other piece of media. Look for one on the copyright page or in the bar code on the back cover.

```
<div id="ISBN0321127307">
  
  <p><cite>The Complete Manual of Typography</cite>
  <p>A combination of type history and examples of
  </p>
</div>
```

```
<div id="ISBN0881792063">
  
  <p><cite>The Elements of Typographic Style</cite></p>
  <p>This lovely, well-written book is concerned
</div>
```

Web authors also use `id` when identifying the various sections of a page. In the following example, there may not be more than one element with the `id` of “links” or “news” in the document:

```
<section id="news">
  <!-- news items here -->
</section>

<aside id="links">
  <!-- list of links here -->
</aside>
```

Classification with class

The `class` attribute classifies elements into conceptual groups; therefore, unlike the `id` attribute, a `class` name may be shared by multiple elements. Making elements part of the same class enables you to easily apply styles to all of the labeled elements at once with a single style rule, or

manipulate them all with a script. Let's start by classifying some elements in the earlier book example. In this first example, we've added `class` attributes to classify each `div` as a "listing" and to classify paragraphs as "descriptions":

```
<div id="ISBN0321127307" class="listing">
  <header>
    
    <p><cite>The Complete Manual of Typography</cite>
  </header>
  <p class="description">A combination of type h
</div>

<div id="ISBN0881792063" class="listing">
  <header>

    
    <p><cite>The Elements of Typographic Style</cite>
    </p>
  </header>
  <p class="description">This lovely, well-writte
</div>
```

Notice how the same element may have both a `class` and an `id`. It's also possible for elements to belong to multiple classes. When there is a list of `class` values, simply separate them with character spaces. In this

example, we’ve classified each `div` as a “book” to set them apart from possible “audiobook” or “dvd” listings elsewhere in the document:

```
<div id="ISBN0321127307" class="listing book">
  
  <p><cite>The Complete Manual of Typography</cite>
  <p class="description">A combination of type h
</div>

<div id="ISBN0881792063" class="listing book">
  <cite>The Elements of Typographic Style</cite>
  </p>
  <p class="description">This lovely, well-written
  foremost with creating beautiful typography.</p>
</div>
```

TIP

Use the `id` attribute to *identify*.

Use the `class` attribute to *classify*.

Identify and Classify All the Elements

The `id` and `class` attributes are not limited to just `div` and `span` — they are two of the *global attributes* in HTML, which means you may use them with any HTML element (see Note). For example, you could give an ordered list an `id` of “directions” instead of wrapping it in a `div`:

```
<ol id="directions">
  <li>...</li>
  <li>...</li>
  <li>...</li>
</ol>
```

NOTE

Other examples of global attributes are `style`, `tabindex`, and `title`. See Appendix B, HTML Global Attributes, for a list of all the available global attributes with their values and definitions.

This section should have given you a good introduction to how to use the `class` and `id` attributes to add meaning and organization to documents. We’ll work with them more in the style sheet chapters in Part III, CSS for Presentation.

Improving Accessibility with ARIA

As web designers, we must always consider the experience of users with assistive technologies for navigating pages and interacting with web

applications. Your users may be listening to the content on the page read aloud by a screen reader or using keyboards, joysticks, voice commands, or other non-mouse input devices to navigate through the page.

Many HTML elements are plainly understood when you look at (or read) only the HTML source. Elements like the title, headings, lists, images, and tables have implicit meanings in the context of a page; however, generic elements like `div` and `span` lack the semantics necessary to be interpreted by an assistive device. In rich web applications, especially those that rely heavily on JavaScript, the markup alone does not provide enough clues as to how elements are being used or whether a form control is currently selected, required, or in some other state.

Fortunately, we have *ARIA (Accessible Rich Internet Applications)*, a standardized set of attributes for making pages easier to navigate and interactive features easier to use. The specification was created and is maintained by a Working Group of the Web Accessibility Initiative (WAI), which is why you'll also hear it referred to as WAI-ARIA. ARIA defines *roles*, *states*, and *properties* that developers can add to markup and scripts to provide richer semantic information. The HTML specification lists which roles and properties apply in the description of every element.

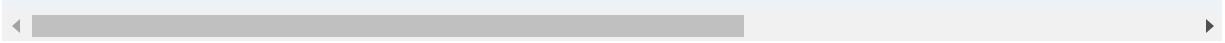
Roles

Roles describe an element’s function or purpose in the context of the document. Examples include `alert`, `button`, `dialog`, `slider`, and `menubar`, to name only a few. For instance, as you saw earlier, you can turn an unordered list into a tabbed menu of options using style sheets, but what if you can’t see that it is styled that way? Adding `role="toolbar"` to the list makes its purpose clear:

```
<ul id="tabs" role="toolbar">
  <li>A-G</li>
  <li>H-O</li>
  <li>P-T</li>
  <li>U-Z</li>
</ul>
```

Here’s another example that reveals that the “status” `div` is used as an alert message:

```
<div id="status" role="alert">You are no longer c
```



Some roles describe “landmarks” that help readers find their way through the document, such as `navigation`, `banner`, `contentinfo`, `complementary`, and `main`. You may notice that some of these sound similar to the page structuring elements that were added in HTML5, and that’s no coincidence. One of the benefits of having improved semantic

section elements is that they can be used as landmarks, so, for example, we can replace `<div id="main" role="main">` with `<main>`.

TIP

It can be tempting to lean on ARIA rather than to focus on building a solid document structure through semantic markup, but that leads to many accessibility issues. Treat ARIA as a way of supplementing a solid foundation of semantic markup.

Most current browsers already recognize the implicit roles of the new elements, but some developers explicitly add ARIA roles until all browsers comply. The sectioning elements pair with the ARIA landmark roles in the following way:

```
<nav role="navigation">
<header role="banner"> (see Note)
<main role="main">
<aside role="complementary">
<footer role="contentinfo">
```

NOTE

The banner role is used when the header applies to the whole page, not just a section or article.

States and Properties

ARIA also defines a long list of states and properties that apply to interactive elements such as form widgets and dynamic content. States and properties are indicated with attributes prefixed with `aria-`, such as `aria-disabled`, `aria-describedby`, and many more.

The difference between a state and a property is subtle. For properties, the value of the attribute is more likely to be stable; examples include `aria-labelledby`, which associates labels with their respective form controls, and `aria-haspopup`, which indicates the element has a related pop-up menu. States have values that are more likely to be changed as the user interacts with the element, such as `aria-selected`.

Further Reading

Obviously, this is not enough ARIA coaching to allow you to start confidently using it today, but it should give you a good feel for its value. When you are ready to dig in and take your skills to a professional level, here are some recommended resources:

The WAI-ARIA Working Draft ([w3.org/TR/wai-aria-1.3/](https://www.w3.org/TR/wai-aria-1.3/))

This is the current Working Draft of the specification as of this writing.

ARIA in HTML ([w3.org/TR/html-aria/](https://www.w3.org/TR/html-aria/))

This W3C Working Draft helps developers use ARIA attributes with HTML correctly. It features a great list of every HTML element, whether

it has an implicit role (in which case ARIA should *not* be used), and what roles, states, and properties apply.

ARIA resources at MDN Web Docs (developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA)

This site features lots of links to ARIA-related and up-to-date resources. It's a good starting point for exploration.

HTML5 Accessibility (html5accessibility.com)

This site tests which new HTML5 features are accessibly supported by major browsers.

Character Escapes

There's just one more text-related topic to cover before I close out this chapter. The title of this section makes it sound like someone left the gate open and all the characters got out. The real meaning is more mundane, albeit useful to know.

You already know that as a browser parses an HTML document, when it runs into a `<` symbol, it interprets it as the beginning of a tag. But what if you just need a less-than symbol in your text? Characters that might be misinterpreted as code need to be *escaped* in the source document.

Escaping means that instead of typing in the character itself, you represent it by its numeric or named *character entity reference*. When the browser

sees the character reference, it substitutes the proper character in that spot when the page is displayed.

There are two ways of referring to (escaping) a specific character:

- Using a predefined abbreviated name for the character (called a *named entity*). There are hundreds of named entities defined in HTML, and you can't make up your own.
- Using an assigned numeric value that corresponds to its position in a coded character set (*numeric entity*). Numeric values may be in decimal or hexadecimal format.

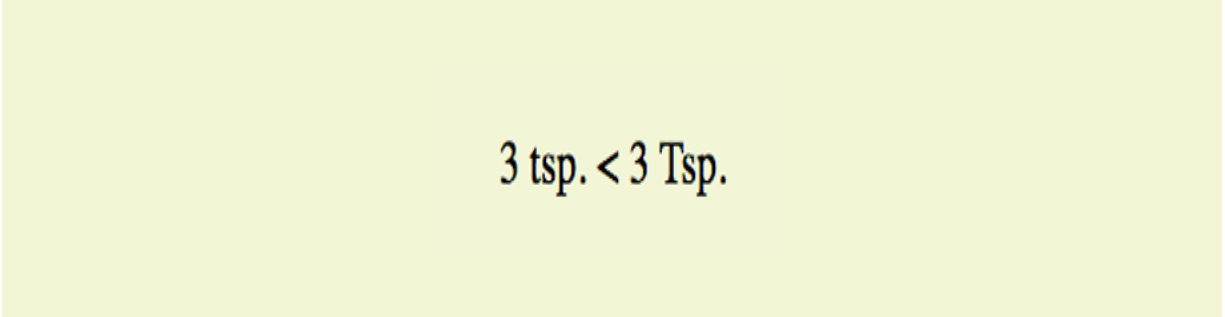
All character references begin with an `&` (ampersand) and end with a `;` (semicolon).

An example should make this clear. If, as mentioned previously, you want to use a less-than symbol in your text, you must use the named entity (`<`) or its numeric equivalent (`<`) where you want the symbol to appear (Figure 5-17). For example, you'd need to write:

```
<p>3 tsp. &lt; 3 Tsp.</p>
```

or:

```
<p>3 tsp. &#060; 3 Tsp.</p>
```



3 tsp. < 3 Tsp.

Figure 5-17. The special character is substituted for the character reference when the document is displayed in the browser.

When to Escape Characters

The majority of special characters do not need to be escaped if you are using the UTF-8 character encoding for your document (declared with a `meta` element in the `head` of the document). The browser will display most special characters in the source document just fine. However, there are a few instances in which you may need or want to use a character reference.

Characters from HMTL code syntax

The `<`, `>`, `&`, `"`, and `'` characters have special syntax meanings in HTML and may be misinterpreted as code. Therefore, the spec recommends that you escape `<`, `>`, and `&` characters in content. The most important is `<`, because it indicates the beginning of a tag. The ampersand (`&`) is generally safe unless the characters directly after it indicate a character reference.

If attribute values contain single or double quotes, escaping the quote characters in the values is advised. Quote marks are fine in the content and

do not need to be escaped. [Table 5-2](#) lists the entity names and decimal and hexadecimal numeric equivalents for each of these characters.

Table 5-2. Syntax characters and their character references

Character	Description	Entity name	Decimal no.
<	Less-than symbol	<	<
>	Greater-than symbol	>	>
"	Quotation mark	"	"
'	Apostrophe	'	'
&	Ampersand	&	&



TIP

These character entities are useful when you need to show an example of HTML markup on a web page.

Invisible or ambiguous characters

Some characters have no graphic display (see Note) and are difficult to see in the markup. These characters (listed in Table 5-3) include the non-breaking space (` `), which is used to ensure that a line doesn't break between two words. So, for instance, if I mark up my name like this:

```
Jennifer&nbsp;Robbins
```

I can be sure that my first and last names will always stay together on a line. Another use for non-breaking spaces is to separate digits in a long number, such as 32 000 000.

Zero-width spaces can be inserted in languages that do not use spaces between words to indicate where lines should break. A zero-width joiner is a non-printing space that causes neighboring characters to display in their connected forms (common in Arabic and Indic languages). Zero-width non-joiners prevent neighboring characters from joining to form ligatures or other connected forms.

NOTE

Some code editors do display invisible characters. For example, IntelliJ editors indicate non-breaking spaces with “NBSP”.

Table 5-3. Invisible characters and their character references

Character	Description	Entity name	Decimal no.
(Non-printing)	Non-breaking space	 	
(Non-printing)	En space	 	 
(Non-printing)	Em space	 	 
(Non-printing)	Zero-width space	(None)	​
(Non-printing)	Zero-width non-joiner	‌	‌
(Non-printing)	Zero-width joiner	‍	‍

Input limitations

If your keyboard or editing software does not include the character you need (or if you simply can't find it), you can use a character entity to make sure you get the character you want. The spec doesn't endorse this practice, so use the proper character in your source if you are able. Table 5-4 lists some special characters that may be less straightforward to type into the source.

Table 5-4. Special characters and their character references

Character	Description	Entity name	Decimal no.
...	Horizontal ellipsis	…	…
©	Copyright	©	©
®	Registered trademark	®	®
™	Trademark	™	™
£	Pound	£	£
¥	Yen	¥	¥
€	Euro	€	€
—	En dash	–	–
—	Em dash	—	—



A complete list of HTML named entities and their Unicode code points can be found as part of the HTML specification at html.spec.whatwg.org/multipage/named-characters.html#named-character-references. For a more user-friendly listing of named and numerical entities, I recommend this archived page at the Web Standards Project: webstandards.org/learn/reference/charts/entities.

Putting It All Together

So far, you've learned how to mark up elements, and you've met all of the HTML elements for adding structure and meaning to text content. Now it's just a matter of practice. Exercise 5-3 gives you an opportunity to try out everything we've covered so far: document structure elements, grouping (block) elements, text-level elements, sectioning elements, and character entities.

EXERCISE 5-3. THE BLACK GOOSE BISTRO NEWS PAGE

Now that you’ve been introduced to all of the text elements, you can put them to work by marking up the News page for the Black Goose Bistro site. You can find the starter text and finished markup files at learningwebdesign.com/6e/materials. The resulting page is shown in Figure 5-18.

NOTE

The “Low and Slow” paragraph is already marked up with the text-level elements from Exercise 5-2.

MARKUP TIPS

- Choose the element that best fits the meaning of the selected text.
- Don’t forget to close elements with closing tags.
- Put all attribute values in quotation marks for clarity.
- Copy and paste is your friend when adding the same markup to multiple elements. Just be sure what you copied is correct before you paste it throughout the document.

Here’s the text that we’ll be marking up:

The Black Goose Bistro

News

Home
Menu
News
Contact

Summer Menu Items

posted by BGB, June 18, 2024

Our chef has been busy putting together the perfect

Appetizers

Black bean purses

Spicy black bean and a blend of Mexican cheeses w

Southwestern napoleons with lump crab -- new item

Layers of light lump crab meat, bean and corn sal

Main courses

Shrimp sate kebabs with peanut sauce

Skewers of shrimp marinated in lemongrass, garlic

Jerk rotisserie chicken with fried plantains -- r

Tender chicken slow-roasted on the rotisserie, fi

Low and Slow

posted by BGB, November 15, 2024

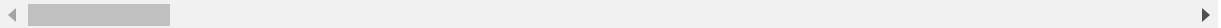
<p>This week I am extremely excited about

<p>Next month, we will be serving <i>Sous Vide

Location: Baker's Corner, Seekonk, MA

Hours: Tuesday to Saturday, 11am to 11pm

All content copyright 2024, Black Goose Bistro and



The Black Goose Bistro News

- [Home](#)
- [Menu](#)
- [News](#)
- [Contact](#)

Summer Menu Items

posted by BGB, June 18, 2024

Our chef has been busy putting together the perfect menu for the summer months. Stop by to try these appetizers and main courses while the days are still long.

Appetizers

Black bean purses

Spicy black bean and a blend of Mexican cheeses wrapped in sheets of phyl-lo and baked until golden. \$3.95

Southwestern napoleons with lump crab — **new item!**

Layers of light lump crab meat, bean and corn salsa, and our handmade flour tortillas. \$7.95

Main courses

Shrimp sate kebabs with peanut sauce

Skewers of shrimp marinated in lemongrass, garlic, and fish sauce then grilled to perfection. Served with spicy peanut sauce and jasmine rice. \$12.95

Jerk rotisserie chicken with fried plantains — **new item!**

Tender chicken slow-roasted on the rotisserie, flavored with spicy and fragrant jerk sauce and served with fried plantains and fresh mango. \$12.95

Low and Slow

posted by BGB, November 15, 2016

This week I am *extremely* excited about a new cooking technique called *sous vide*. In *sous vide* cooking, you submerge the food (usually vacuum-sealed in plastic) into a water bath that is precisely set to the target temperature you want the food to be cooked to. In his book, *Cooking for Geeks*, Jeff Potter describes it as “ultra-low-temperature poaching.”

Next month, we will be serving ***Sous Vide* Salmon with Dill Hollandaise**. To reserve a seat at the chef table, contact us before November 30.

Location:

Baker's Corner, Seekonk, MA

Hours:

Tuesday to Saturday, 11am to 11pm

All content copyright © 2024, Black Goose Bistro

FIGURE 5-18. THE FINISHED MENU PAGE

To mark it up so it looks like Figure 5-18, follow these instructions:

1. Start by adding the DOCTYPE declaration to tell browsers this is a standards-compliant HTML document.
2. Add all the document structure elements first (html, head, meta, title, and body). Give the document the title “The Black Goose Bistro News.”
3. Identify the top-level heading and the list of links as the header for the document by wrapping them in a header element (don’t forget the closing tag). Within the header, the headline should be an h1 and the list of links should be an unordered list (ul). Don’t worry about making the list items links; we’ll get to linking in the next chapter. Give the list more meaning by identifying it as the primary navigation for the site (nav).
4. The News page has two posts, titled “Summer Menu Items” and “Low and Slow.” Mark up each one as an article.
5. Now, get the first article into shape. Let’s create a header for this article that contains the heading (h2 this time, because we’ve moved down in the document hierarchy) and the publication information (p). Identify the publication date for the article with the time element, just like in Exercise 5-2.
6. The content after the header is a simple paragraph. However, there are some interesting things going on in the menu. It’s divided into two conceptual sections (Appetizers and Main Courses), so mark those up as

section elements. Make sure that the final closing `</section>` tag appears before the closing `</article>` tag, so the elements are nested correctly and don't overlap. Finally, identify the sections with id attributes. Name the first one "appetizers" and the second "maincourses".

7. With our sections in place, now we can mark up the content. We're down to h3 for the headings in each section. Choose the most appropriate list elements to describe the menu item names and their descriptions. Mark up the lists and each item within the lists.
8. Now we can add a few fine details. Classify each price as "price" using span elements.
9. Two of the dishes are new items. Change the double hyphens to an em dash character and mark up "new item!" as "strongly important." Classify the title of each new dish as "newitem" (use the existing dt element; there's no need to add a span this time). This allows us to target menu titles with the "newitem" class and style them differently than other menu items.
10. That takes care of the first article. The second article is already mostly marked up from the previous exercise, but you should mark up the header with the appropriate heading and publication date information.
11. So far, so good, right? Now make the remaining content that applies to the whole page a footer. Mark each line of content within the footer as a paragraph.
12. Next, give the location and hours information some context by putting them in a div named "about". Make the labels "Location:" and "Hours:"

appear on lines by themselves by adding line breaks after them. Mark up the hours with the time element (you don't need the date or time zone portions).

13. Finally, copyright information is typically “small print” on a document, so mark it up accordingly. As the final touch, add a copyright symbol after the word “copyright” using the keyboard or the © character entity.

Save the document as `bistro_news.html`, and check your page in a modern browser. You can also use the Nu Html Checker (validator.nu) to make sure it's valid (this is a great way to spot mistakes). How did you do?

WANT MORE PRACTICE?

Try marking up your own résumé. Start with the raw text and add document structure elements, content grouping elements, and text-level elements, as you did in Exercise 5-3. If you don't see an element that matches your information just right, try creating one using a `div` or a `span`.
